

Sri Lanka Institute of information Technology

Programming Applications and Frameworks
IT3030



NELLi
..Network for Equipment, Labs and Location Incidents..
Smart Campus Operations Hub
2026

Group Details

IT Number	Name	Email	Contact Number
IT23183018	Hirusha D G A D	it23183018@my.sliit.lk	077 2424 521
IT23191006	Cooray Y H	it23191006@my.sliit.lk	070 6080 877

Submitted On : 2026-04-28

Table of Contents

1. INTRODUCTION 5

2. AIM AND OBJECTIVES 6

2.1. Primary Aim 6
2.2. Objectives 6
2.3. Stakeholders Overview 7
2.3.1. Primary Stakeholders 7
2.3.2. Secondary Stakeholders 8

3. SOLUTION OVERVIEW 10

3.1. Functional Scope 10
3.2. Technology Overview 11

4. SYSTEM ARCHITECTURE 12

4.1. Overall System Architecture 12
4.2. Backend Architecture 13
4.3. Frontend Architecture 15

5. SYSTEM DEPENDENCIES 17

5.1. Backend Maven Dependencies 17
5.2. Frontend npm Dependencies 18

6. DATABASE ARCHITECTURE 19

6.1. Database Platform 19
6.2. Entity Relationship Overview 19
6.3. Core Entity Specifications 21
6.3.1. Users Table 21
6.3.2. Facilities Table 22
6.3.3. Bookings Table 22
6.3.4. Tickets Table 23
6.3.5. Supporting Tables 24
6.4. Key Database Constraints & Indexes 25

7. PROJECT TIMELINE - GANTT CHART 26

8. IMPLEMENTED COMPONENTS 30

8.1. Module A : Facilities & Assets Catalogue 30
8.1.1. Backend Components 30
8.1.2. Frontend Components 32
8.1.3. Non-Functional Requirements of Module A 33
8.2. Module B : Booking Management 34
8.2.1. Backend Components 34
8.2.2 Frontend Components 37
8.2.2. Non-Functional Requirements of Module B 37
8.3. Module C : Maintenance & Incident Ticketing 38
8.3.1. Backend Components 38

8.3.2. Frontend Components 41
8.3.3. Non-Functional Requirements of Module C 42
8.4. Module D : Notifications & Email Service 42
8.4.1. Backend Components 43
8.4.2. Frontend Components 45
8.4.3. Non-Functional Requirements of Module D 46
8.5. Module E : Authentication & Authentication 46
8.5.1. Backend Components 47
8.5.2. Frontend Components 49
8.5.3. Non-Functional Requirements of Module E 50
8.6. Module F - QR Code Check-In for Approved Bookings (Innovative Feature) 50
8.6.1. Backend Implementation 50
8.6.2. Frontend Implementation 51
8.6.3. Non-Functional Requirements of Module F 51
8.7. Module G - Admin Analytics Dashboard (Innovative Feature) 52
8.7.1. Backend Components 52
8.7.2. Frontend Components 54
8.7.3. Non-Functional Requirements of Module G 55

9. REST API ARCHITECTURE 56

9.1. API Design Principle 56
9.2. Complete Endpoint Inventory 56
9.3. Standardized Error Response Format 57
9.4. API Documentation 58

10. TESTING & QUALITY MANAGEMENT W/ POSTMAN 63

10.1. Testing Strategy 63
10.2. Backend Unit & Integration Tests 63
10.2.1. Service Layer Unit Tests 63
10.2.2. Controller Integration Tests 63
10.3. Postman Testing 64
10.3.1. Collection Structure 64
10.3.2. Test Scripts 65
10.3.3. Test Coverage Summary 65
9.4. Input Validation & Error Handling 66

11. VERSION CONTROL W/ GITHUB 67

11.1. Repository Structure (Source Code Links) 67
11.2. Branching Strategy 68
11.3. Complete Branch Registery 68
11.4. Commit Message Convention 71
11.5. GitFlow WorkFlow 72
11.6. Continuous Integration Pipeline 77
11.6.1. Github Actions Workflow 77
11.6.2. Workflow Configuration 77
11.3. CI Pipeline Stages 79
11.4. Secrets Management 80

12. INDIVIDUAL CONTRIBUTION 81

12.1. HIRUSHA D G A D - Contribution Summary 81
12.1.1. Backend Contributions 81
12.1.2. Frontend Contributions 82
12.1.3. Key Technical Achievements 82
12.2. COORAY Y H - Contribution Summary 83
12.2.1. Backend Contributions 83
12.2.2. Frontend Contributions 84
12.2.3. Key Technical Achievements 84

13. CONCLUSION 86

14. APPENDIX 87

14.1. The Landing Page 87
14.2. Login & Register Pages 89
14.3. User Dashboard 90
14.4. Facilities Component 90
14.5. Booking Component 92
14.6. Ticketing Component 95
14.7. QR Code Verification Component 98
14.8. Admin Management Components 99
14.9. Admin Analytics Components 102

15. REFERENCES 104

1. Introduction

NELLi - Smart Campus Operations Hub is a full-stack, production-inspired web system developed for the IT3030 Programming Applications and Frameworks module at SLIIT. It addresses a real-world university challenges such as managing physical resources, bookings, and maintenance operations through a single, unified digital platform.

Modern universities manage dozens of facilities lecture halls, computer laboratories, meeting rooms, and specialist equipment across multiple buildings and departments. Traditionally, these are administered through fragmented spreadsheets, email chains, or disparate legacy systems. NELLi consolidates all these workflows into one cohesive platform, enabling role-based access, transparent audit trails, and real-time operational visibility.

The system was architected and implemented by a two-member team (Dineth and Yashodha) within a four-week sprint (24 March - 27 April 2026), delivering the full scope originally designed for a four-member group. The resulting application covers seven functional modules: **Facilities Catalogue, Booking Management, Incident Ticketing, Notifications & Email, Authentication & Authorization, QR Code Check-In, and an Admin Analytics Dashboard.**

The backend is built with Java Spring Boot following layered RESTful architecture principles, while the frontend is implemented with React (Vite), consuming the API through Axios service modules. Data persistence is provided by PostgreSQL hosted on Neon (cloud-managed), with JWT and OAuth 2.0 (Google Sign-In) securing all endpoints. Version control is managed via GitHub with structured feature branching and CI/CD via GitHub Actions.

2. Aim and Objectives

2.1. Primary Aim

To design, develop, and demonstrate a production-quality web platform that digitizes and streamlines campus facility management, booking workflows, and maintenance operations for a university environment adhering to industry-standard RESTful API design, role-based access control, and modern frontend engineering practices.

2.2. Objectives

- Implement a comprehensive Facilities & Assets Catalogue supporting CRUD operations with search and filtering by type, capacity, and location.
- Develop a Booking Management system enforcing workflow states (PENDING → APPROVED/REJECTED → CANCELLED) with server-side conflict detection for overlapping time ranges.
- Build a Maintenance & Incident Ticketing system supporting full ticket lifecycle management, technician assignment, image attachments, and comment threads.
- Deliver a real-time Notification system integrated with an Email Service (Gmail SMTP) to alert users on booking outcomes, ticket status changes, and comment activity.
- Implement secure Authentication and Authorization using OAuth 2.0 (Google Sign-In) and JWT tokens, with role-based access control (USER / ADMIN / TECHNICIAN) enforced at every endpoint.
- Provide a QR Code Check-In system that generates unique QR codes for approved bookings, enabling physical verification at facility entry points.
- Develop an Admin Analytics Dashboard offering usage insights including top resources, peak booking hours, and ticket resolution metrics.

- ❖ Maintain a clean, traceable Git history with structured feature / debug / enhancement and automated CI checks via GitHub Actions.

- ❖ Produce comprehensive Postman test collections covering all REST endpoints with environment-specific variables.

- ❖ Deliver a well-structured, documented codebase that clearly separates backend and frontend responsibilities, enabling independent assessment of each team member's contributions.

2.3. Stakeholders Overview

The NELLi Smart Campus Operations Hub involves multiple stakeholders who interact with the system at different levels. Each stakeholder has specific goals, responsibilities, and system interactions.

2.3.1. Primary Stakeholders

1. Students / General Users (USER role)

- Book facilities such as lecture halls, labs, and meeting rooms
- Submit maintenance or incident tickets
- View booking status and receive notifications
- Use QR codes for check-in to approved bookings

2. University Administrative Staff (ADMIN role)

- Manage facilities and assets (create, update, delete)
- Approve or reject booking requests
- Assign maintenance tickets to technicians
- Monitor system usage via analytics dashboard
- Configure system-level operations and workflows

3. Maintenance Staff / Technicians (TECHNICIAN role)

- View assigned tickets
- Update ticket status (IN_PROGRESS, RESOLVED, etc.)
- Add resolution notes and comments
- Ensure timely issue resolution

2.3.2. Secondary Stakeholders

4. University Management / Decision Makers

- Use analytics dashboard insights for decision-making
- Monitor resource utilization and operational efficiency
- Evaluate performance metrics (ticket resolution time, peak usage, etc.)

5. System Developers (Project Team)

- Design, develop, test, and maintain the system
- Ensure system scalability, security, and performance
- Manage CI/CD pipelines and deployments

6. IT Support & Infrastructure Team

- Maintain servers, database, and network infrastructure
- Ensure uptime, backups, and system security
- Handle technical issues and system monitoring

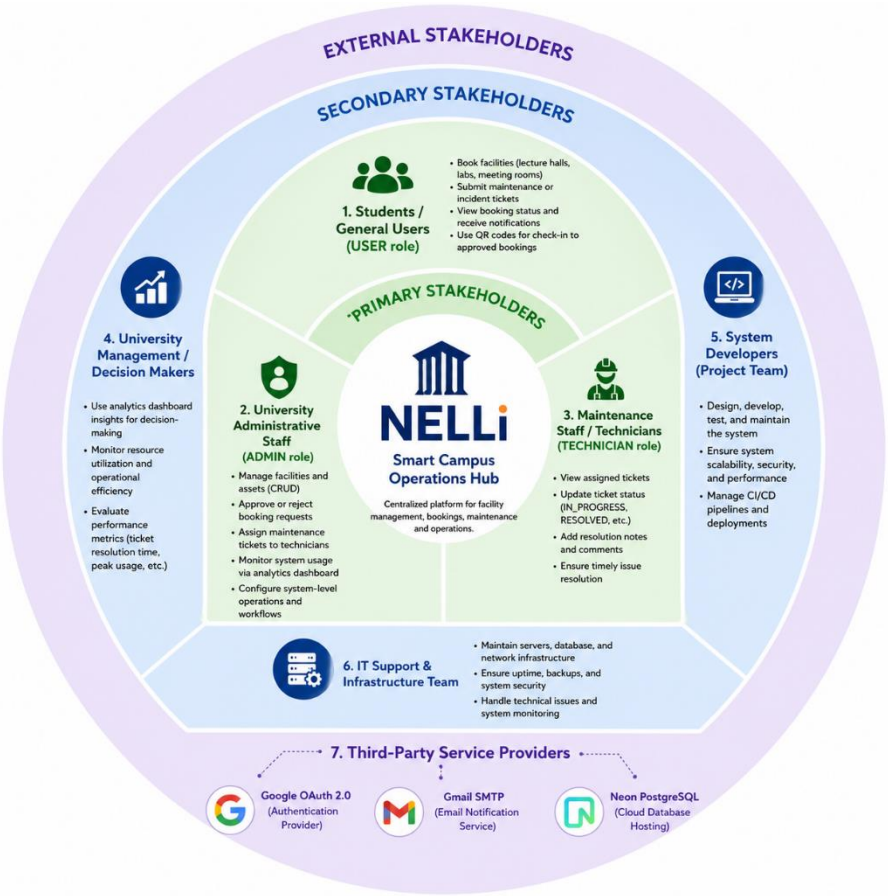
External Stakeholders

7. Third-Party Service Providers

- Google OAuth 2.0 (authentication provider)
- Gmail SMTP (email notification service)
- Neon PostgreSQL (cloud database hosting)

Stakeholder Summary Table

Stakeholder	Role	Key Responsibilities
Students / Users	USER	Booking, ticket submission, QR check-in
Admin Staff	ADMIN	Approvals, management, analytics
Technicians	TECHNICIAN	Ticket resolution
Management	Viewer	Strategic insights
Developers	System Builders	Development & maintenance
IT Support	Infrastructure	Hosting & reliability
External Services	Service Providers	Authentication, email, DB



3. Solution Overview

NELLi is structured as a three-tier system: a Spring Boot REST API (backend) , React/Vite single-page application (frontend) and Neon - Postgres SQL Serverless cloud-base database server (database). Communication between tiers is exclusively through the versioned JSON REST APIs. The platform defines three user roles such as USER, ADMIN, and TECHNICIAN each granted specific permissions across all modules.

3.1. Functional Scope

Module	Core Capability	Access Level
Module A Facilities Catalogue	Resource management with metadata, search, filtering, and status control	USER (read) / ADMIN (write)
Module B Booking Management	Resource reservation with workflow states, conflict checking, and admin approval	USER / ADMIN
Module C Incident Ticketing	Fault reporting with attachments, technician assignment, comments	USER / ADMIN / TECHNICIAN
Module D Notifications & Email	In-app and email notifications for all key events	USER / ADMIN
Module E Auth & Authorization	OAuth 2.0 + JWT with role-based endpoint protection	All roles
Module F QR Code Check-In	QR generation for approved bookings, scan-to-verify screen	USER / ADMIN
Module G Admin Analytics	Dashboard with resource usage, booking trends, ticket KPIs	ADMIN only

3.2. Technology Overview

The stack was chosen to align with the assignment requirements and to reflect industry-standard enterprise web development practices:

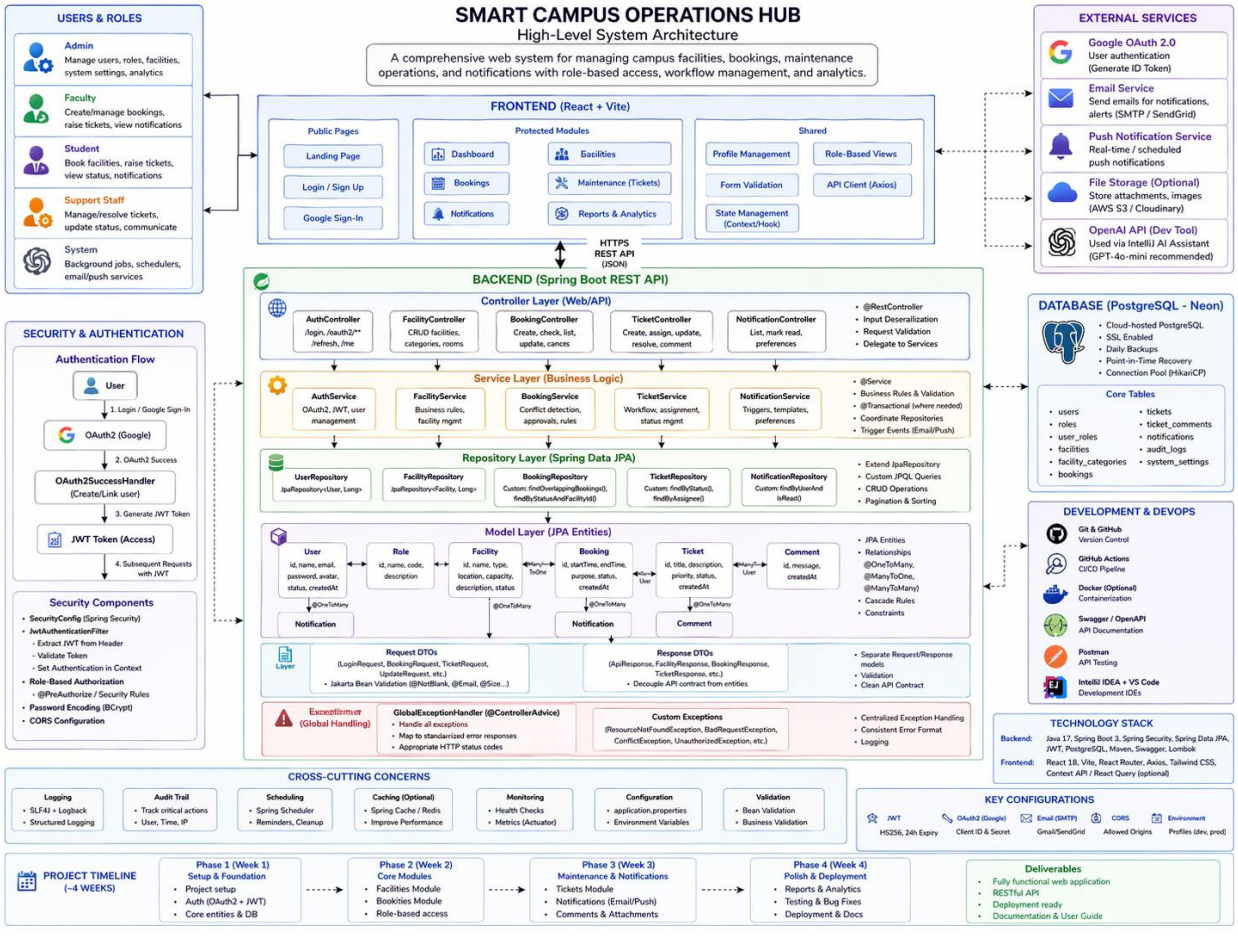
Layer	Technology Choice
Backend Framework	Java 21 SE + Spring Boot 4.0.5
API Style	RESTful with layered architecture (Controller → Service → Repository + DTOs and Models)
Security	Spring Security + OAuth 2.0 (Google) + JWT (JJWT library)
Database	PostgreSQL 15 (hosted on Neon cloud-managed platform)
ORM	Spring Data JPA with Hibernate
File Storage	Local file system / configurable cloud adapter
Email	Spring Mail + Gmail SMTP
Frontend Framework	React 19 + Vite 8 bundler
Styling	Tailwind CSS 3 with custom dark/light theme
State Management	React Context API + custom hooks
HTTP Client	Axios with interceptors for token refresh
Animation	GSAP (GreenSock Animation Platform)
QR Code	qrcode.react (frontend) + zxing (backend scanning)
Charts/Analytics	Recharts
Build & CI	Maven (backend) + Vite (frontend) + GitHub Actions
Version Control	Git with GitHub

Layer	Technology Choice
API Documentation	SpringDoc OpenAPI / Swagger UI

4. System Architecture

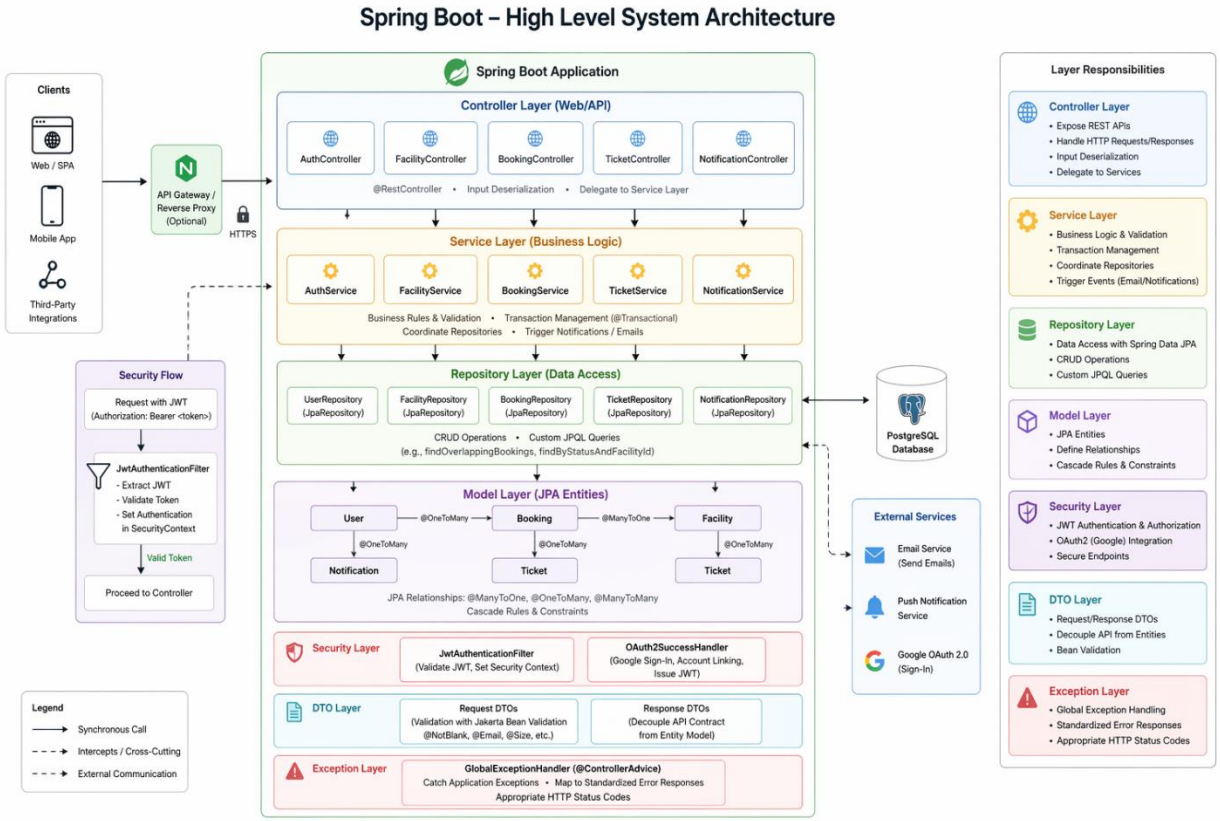
4.1. Overall System Architecture

NELLi follows a classic three-layer client-server architecture with clear separation of concerns across the presentation, application, and data layers.



4.2. Backend Architecture

The Spring Boot application follows a strict layered pattern, ensuring separation between web/API concerns, business logic, and data access:



❖ Controller Layer

- REST controllers annotated with @RestController expose API endpoints. Each controller is responsible for exactly one domain entity (AuthController, FacilityController, BookingController, TicketController, NotificationController). Controllers perform input deserialization and delegate all logic to the service layer.

❖ Service Layer

- Contains all business logic. Services validate rules (e.g., booking conflict detection), coordinate between multiple repositories, and trigger notification and email events. Services use @Transactional where appropriate.

❖ Repository Layer

- Spring Data JPA repositories extend JpaRepository<T, ID>, providing standard CRUD operations and custom JPQL queries (e.g., findOverlappingBookings, findByStatusAndFacilityId).

❖ Model Layer

- JPA entities mapped to PostgreSQL tables. Relationships are defined using @ManyToOne, @OneToMany, and @ManyToMany annotations with appropriate cascade rules.

❖ Security Layer

- JwtAuthenticationFilter intercepts requests to extract and validate JWT tokens. OAuth2SuccessHandler processes Google sign-in callbacks, creating or linking user accounts and issuing JWT tokens.

❖ DTO Layer

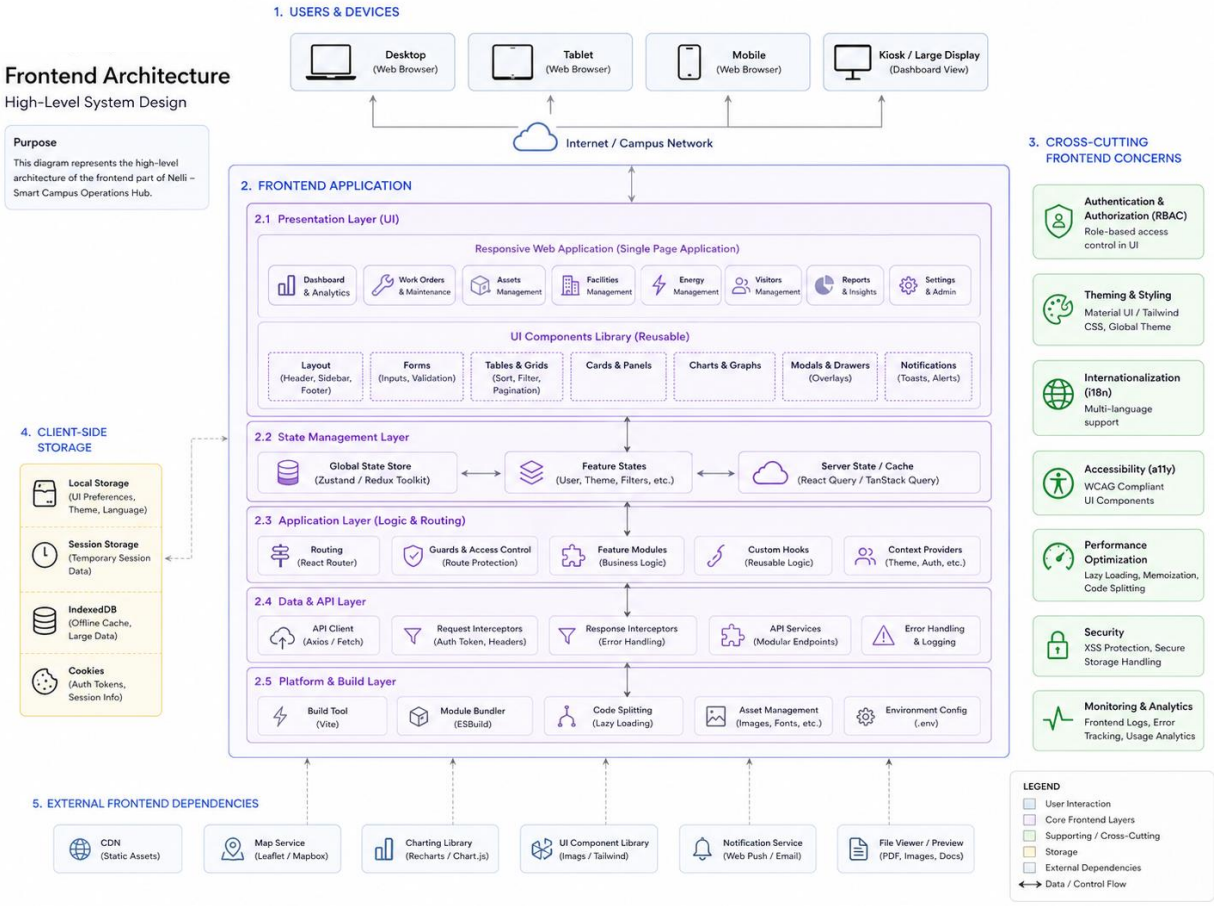
- Separate request and response DTOs decouple the API contract from the entity model. Uses Jakarta Bean Validation (@NotBlank, @Email, @Size, etc.) for input validation.

❖ Exception Layer

- GlobalExceptionHandler (@ControllerAdvice) catches application exceptions and maps them to standardized error responses with appropriate HTTP status codes.

4.3. Frontend Architecture

The React application is structured around domain-specific pages, reusable components, API service modules, and custom hooks:



- ❖ Pages
  - Route-level components (Home, Dashboard, Facilities, Bookings, Tickets, Notifications, Admin pages) act as containers that orchestrate data fetching and child components.

- Components
    - Domain components (FacilityCard, BookingForm, TicketCard, CommentSection) and common UI atoms (Button, Input, Modal, Badge, Spinner) are reused across pages.

- API Services
    - Each domain has a dedicated service module (facilityApi.js, bookingApi.js, ticketApi.js, etc.) using an Axios instance configured with base URL and JWT interceptors.

- Context
    - AuthContext manages authentication state (user, token, role) globally. NotificationContext polls for unread notifications.

- Custom Hooks
    - useAuth, useFacilities, useBookings, useNotifications encapsulate data-fetching logic with loading and error states.

- Routes
    - AppRoutes.jsx defines all application routes. PrivateRoute and AdminRoute HOCs enforce authentication and role-based frontend access.

5. System Dependencies

5.1. Backend Maven Dependencies

Dependency	Group ID	Version	Purpose
Spring Boot Starter Web	org.springframework.boot	4.0.5	REST controllers, JSON serialization
Spring Boot Starter Security	org.springframework.boot	4.0.5	Authentication & authorization
Spring Boot Starter OAuth2 Client	org.springframework.boot	4.0.5	Google OAuth 2.0 login
Spring Boot Starter Data JPA	org.springframework.boot	4.0.5	ORM & database access
Spring Boot Starter Validation	org.springframework.boot	4.0.5	Bean validation
Spring Boot Starter Mail	org.springframework.boot	4.0.5	Email notifications
PostgreSQL JDBC Driver	org.postgresql	42.x	PostgreSQL connectivity
JJWT API	io.jsonwebtoken	0.12.x	JWT creation & parsing
JJWT Impl	io.jsonwebtoken	0.12.x	JWT implementation
SpringDoc OpenAPI	org.springdoc	2.x	Swagger UI / API docs
Lombok	org.projectlombok	1.18.x	Boilerplate reduction
ZXing Core	com.google.zxing	3.5.x	QR code generation
MapStruct	org.mapstruct	1.5.x	DTO-Entity mapping
Spring Boot Test	org.springframework.boot	3.x	Testing framework

5.2. Frontend npm Dependencies

Package	Version	Category	Purpose
react	19.2.4	Core	UI component framework
react-dom	19.x	Core	DOM rendering
vite	8.0.1	Build	Fast development server & bundler
axios	1.6.2	HTTP	API calls with interceptors
react-router-dom	6.20.0	Routing	Client-side routing
tailwindcss	3.4.17	Styling	Utility-first CSS framework
gsap	3.12.4	Animation	Professional UI animations
Charts js	4.4.0	Charts	Analytics dashboard charts
qrcode.react	1.4.2	QR	QR code rendering in React
react-hook-form	7.x	Forms	Performant form management
date-fns	3.x	Utils	Date formatting & manipulation
React Icons	4.12.0	Icons	Consistent icon library
@headlessui/react	1.x	UI	Accessible UI primitives
React Toastify	11.1.0	UI	Toast notification UI

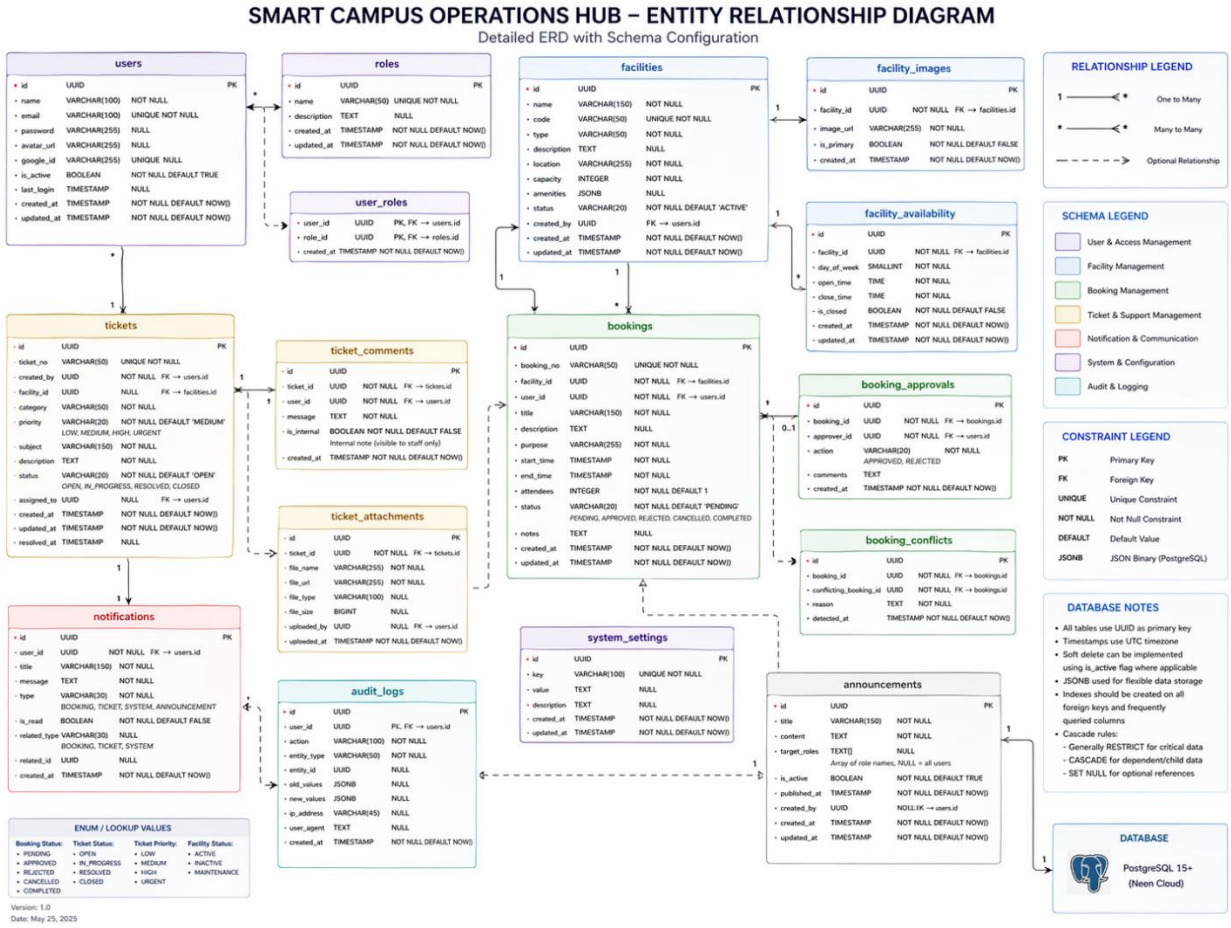
6. Database Architecture

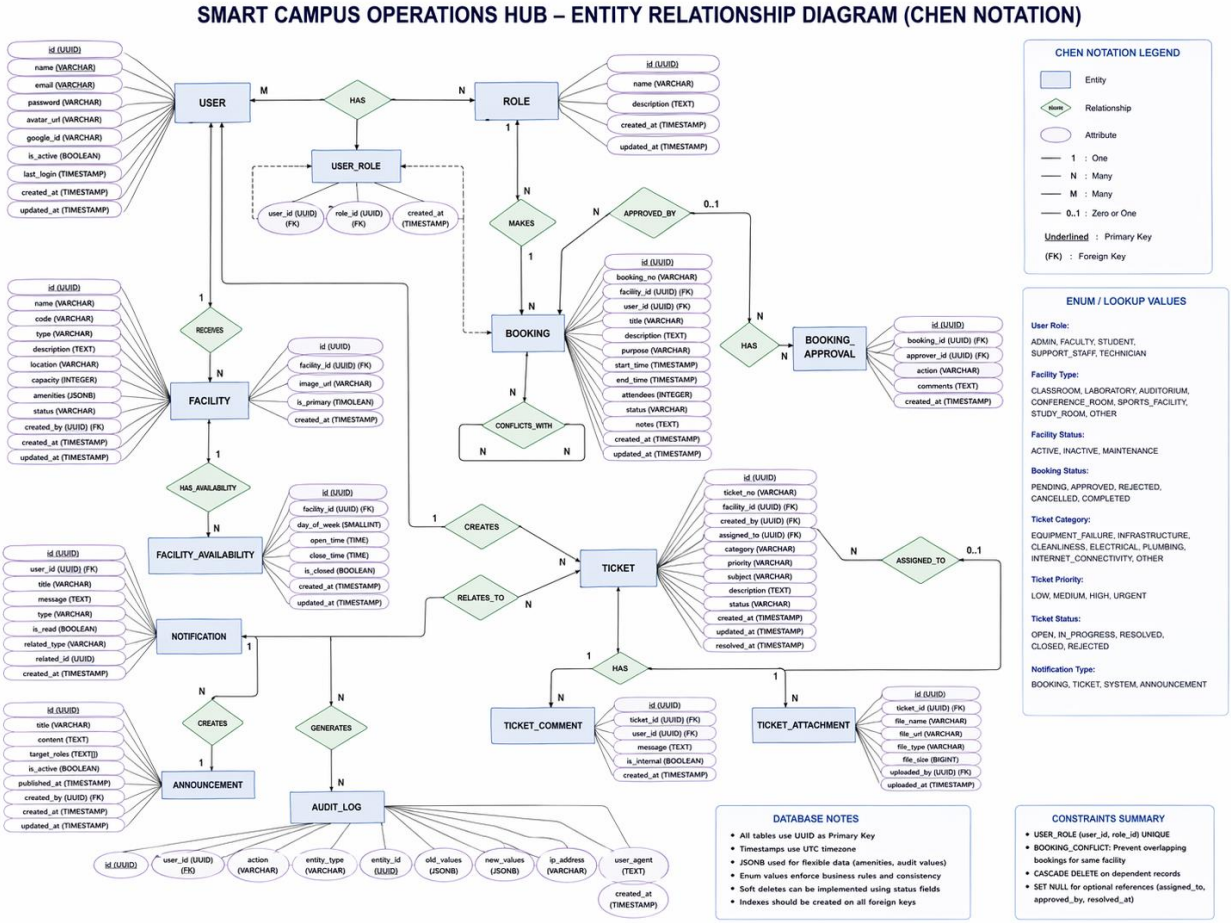
6.1. Database Platform

NELLi uses PostgreSQL 15 hosted on Neon, a serverless cloud-managed PostgreSQL service. Neon was selected over a local PostgreSQL instance for the following reasons:

- ❖ It eliminates local setup and maintenance overhead
- ❖ Provides a free tier sufficient for development and demonstration
- ❖ Enables both team members to connect to the same database instance remotely
- ❖ Offers built-in backups and point-in-time recovery.
- ❖ The connection is secured via SSL (sslmode=require)

6.2. Entity Relationship Overview





The database schema consists of ten primary tables, organized around five core domain entities.

6.3. Core Entity Specifications

6.3.1. Users Table

Column	Type & Constraints
id	UUID, PRIMARY KEY, DEFAULT gen_random_uuid()
email	VARCHAR(255), UNIQUE, NOT NULL
name	VARCHAR(255), NOT NULL
password_hash	VARCHAR(500), NULLABLE (NULL for OAuth-only users)
provider	VARCHAR(50), DEFAULT 'LOCAL' (LOCAL / GOOGLE)
provider_id	VARCHAR(255), NULLABLE (Google subject ID)
role	VARCHAR(50), NOT NULL, DEFAULT 'USER' (USER / ADMIN / TECHNICIAN)
is_active	BOOLEAN, DEFAULT TRUE
profile_picture_url	VARCHAR(500), NULLABLE
created_at	TIMESTAMP WITH TIME ZONE, DEFAULT CURRENT_TIMESTAMP
updated_at	TIMESTAMP WITH TIME ZONE, ON UPDATE

6.3.2. Facilities Table

Column	Type & Constraints
id	UUID, PRIMARY KEY
name	VARCHAR(255), NOT NULL
type	VARCHAR(100), NOT NULL (LECTURE_HALL / LAB / MEETING_ROOM / EQUIPMENT)
capacity	INTEGER, CHECK (capacity > 0), NULLABLE for equipment
location	VARCHAR(500), NOT NULL
description	TEXT, NULLABLE
status	VARCHAR(50), NOT NULL, DEFAULT 'ACTIVE' (ACTIVE / OUT_OF_SERVICE / MAINTENANCE)
availability_windows	JSONB - stores array of {dayOfWeek, startTime, endTime}
amenities	TEXT[], NULLABLE - e.g., {Projector, Whiteboard, AC}
image_url	VARCHAR(500), NULLABLE
created_by	UUID, FK → users(id)
created_at / updated_at	TIMESTAMP WITH TIME ZONE

6.3.3. Bookings Table

Column	Type & Constraints
id	UUID, PRIMARY KEY

Column	Type & Constraints
facility_id	UUID, NOT NULL, FK → facilities(id)
user_id	UUID, NOT NULL, FK → users(id)
start_datetime	TIMESTAMP WITH TIME ZONE, NOT NULL
end_datetime	TIMESTAMP WITH TIME ZONE, NOT NULL
purpose	VARCHAR(500), NOT NULL
expected_attendees	INTEGER, NULLABLE
status	VARCHAR(50), NOT NULL, DEFAULT 'PENDING' (PENDING/APPROVED/REJECTED/CANCELLED)
admin_note	TEXT, NULLABLE - reason for rejection
qr_token	VARCHAR(500), NULLABLE - generated on approval
contact_email	VARCHAR(255), NULLABLE
created_at / updated_at	TIMESTAMP WITH TIME ZONE

6.3.4. Tickets Table

Column	Type & Constraints
id	UUID, PRIMARY KEY
ticket_number	VARCHAR(50), UNIQUE - e.g., TCK-2026-001234
facility_id	UUID, NOT NULL, FK → facilities(id)
reported_by	UUID, NOT NULL, FK → users(id)

Column	Type & Constraints
assigned_to	UUID, NULLABLE, FK → users(id)
category	VARCHAR(100) (EQUIPMENT_FAILURE / STRUCTURAL / ELECTRICAL / PLUMBING / OTHER)
priority	VARCHAR(50) (LOW / MEDIUM / HIGH / CRITICAL)
status	VARCHAR(50), DEFAULT 'OPEN' (OPEN/IN_PROGRESS/RESOLVED/CLOSED/REJECTED)
description	TEXT, NOT NULL
resolution_notes	TEXT, NULLABLE
contact_email	VARCHAR(255), NULLABLE
contact_phone	VARCHAR(50), NULLABLE
first_response_at	TIMESTAMP, NULLABLE - SLA tracking
resolved_at	TIMESTAMP, NULLABLE
created_at / updated_at	TIMESTAMP WITH TIME ZONE

6.3.5. Supporting Tables

Table	Purpose & Key Columns
comments	Ticket comments: id, ticket_id (FK), user_id (FK), content TEXT, created_at, updated_at
attachments	Ticket image files: id, ticket_id (FK), file_name, file_url, file_size, uploaded_at

Table	Purpose & Key Columns
notifications	User alerts: id, user_id (FK), type ENUM, title, message, related_entity_type, related_entity_id, is_read BOOL, created_at

6.4. Key Database Constraints & Indexes

❖ Booking conflict check:

- UNIQUE partial index on (facility_id, start_datetime, end_datetime) - supplemented by JPQL overlap query in BookingRepository.

❖ Ticket number sequence:

- Database sequence or UUID-based generation with prefix TCK-YYYY-NNNNNN.

❖ Indexes:

- All foreign keys are indexed; additional composite index on bookings(facility_id, status) and tickets(status, priority) for efficient filtering queries.

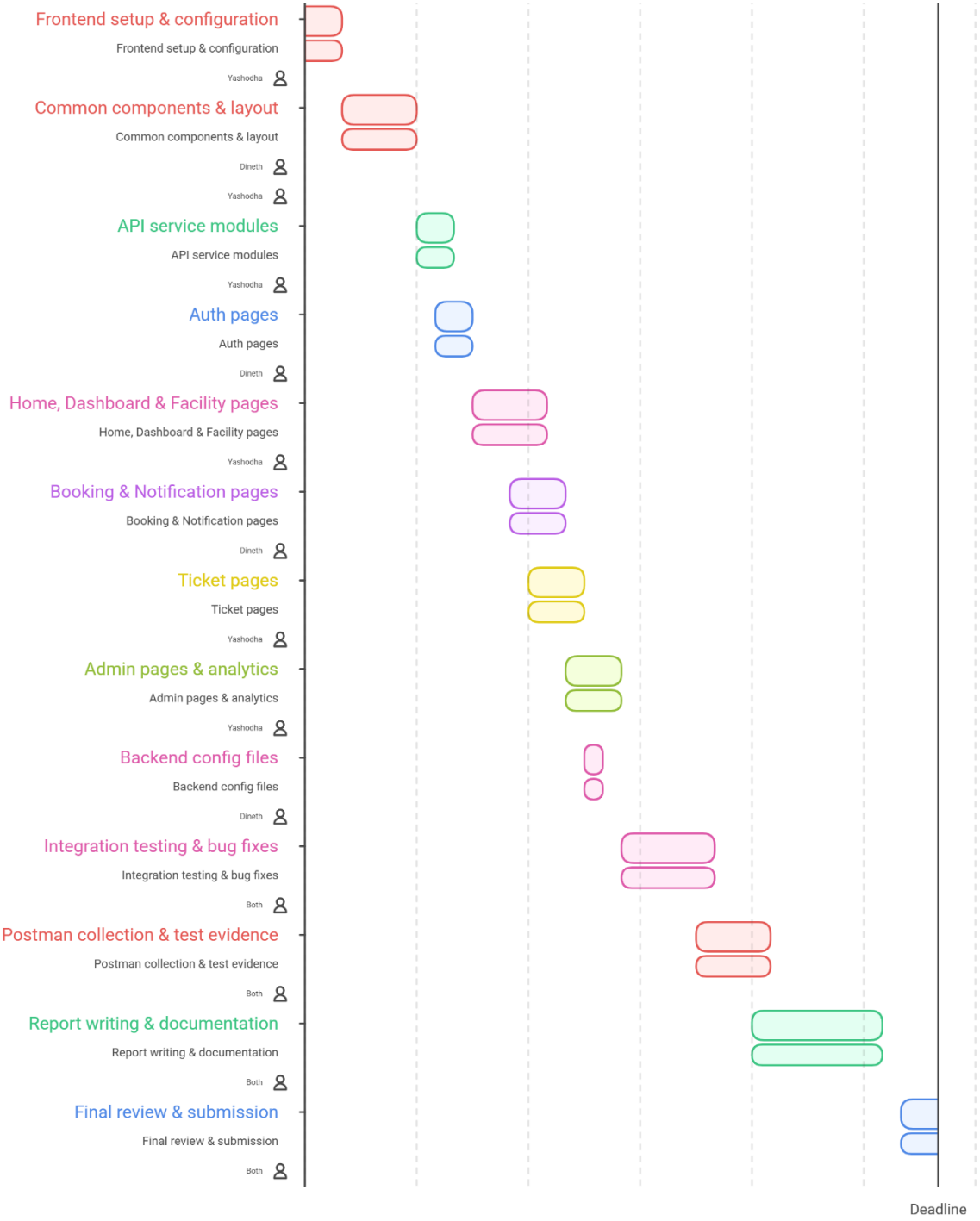
❖ Soft deletes:

- facilities uses status = 'DELETED' rather than physical deletion, preserving referential integrity with historical bookings.

7. Project Timeline - Gantt Chart

The project was executed over a four-week sprint from 24 March to 27 April 2026. The following table summarises the planned schedule. A visual Gantt chart is provided as a separate insert (Appendix A).




Frontend setup & configuration
Frontend setup & configuration
Yashodha
Common components & layout
Common components & layout
Dineth
Yashodha
API service modules
API service modules
Yashodha
Auth pages
Auth pages
Dineth
Home, Dashboard & Facility pages
Home, Dashboard & Facility pages
Yashodha
Booking & Notification pages
Booking & Notification pages
Dineth
Ticket pages
Ticket pages
Yashodha
Admin pages & analytics
Admin pages & analytics
Yashodha
Backend config files
Backend config files
Dineth
Integration testing & bug fixes
Integration testing & bug fixes
Both
Postman collection & test evidence
Postman collection & test evidence
Both
Report writing & documentation
Report writing & documentation
Both
Final review & submission
Final review & submission
Both
Deadline

Phase / Task	Owner	Start Date	End Date
Repository & Environment Setup	Both	24 Mar 2026	24 Mar 2026
Database design & schema planning	Both	24 Mar 2026	25 Mar 2026
OAuth2 + JWT authentication (feature/001, 002)	Dineth / Yashodha	24 Mar 2026	26 Mar 2026
Backend base entities & enums (feature/003, 005)	Dineth	25 Mar 2026	27 Mar 2026
Facility & Ticket entities (feature/006, 008)	Yashodha	26 Mar 2026	29 Mar 2026
Notification entity (feature/009)	Dineth	28 Mar 2026	29 Mar 2026
Facility & Ticket services (feature/010, 012)	Yashodha	30 Mar 2026	2 Apr 2026
Booking service & conflict logic (feature/011)	Dineth	30 Mar 2026	2 Apr 2026
Notification & Email service (feature/013)	Dineth	2 Apr 2026	4 Apr 2026
Security configuration (feature/014)	Dineth	3 Apr 2026	5 Apr 2026
Analytics DTOs & QR module (feature/015)	Yashodha	4 Apr 2026	6 Apr 2026
Frontend setup & configuration (feature/016)	Yashodha	24 Mar 2026	26 Mar 2026
Common components & layout (feature/017, 018)	Dineth / Yashodha	26 Mar 2026	30 Mar 2026
API service modules (feature/019)	Yashodha	30 Mar 2026	1 Apr 2026
Auth pages (feature/020)	Dineth	31 Mar 2026	2 Apr 2026

Phase / Task	Owner	Start Date	End Date
Home, Dashboard & Facility pages (feature/021, 022)	Yashodha	2 Apr 2026	6 Apr 2026
Booking & Notification pages (feature/023, 025)	Dineth	4 Apr 2026	7 Apr 2026
Ticket pages (feature/024)	Yashodha	5 Apr 2026	8 Apr 2026
Admin pages & analytics (feature/026)	Yashodha	7 Apr 2026	10 Apr 2026
Backend config files (feature/027)	Dineth	8 Apr 2026	9 Apr 2026
Integration testing & bug fixes	Both	10 Apr 2026	15 Apr 2026
Postman collection & test evidence	Both	14 Apr 2026	18 Apr 2026
Report writing & documentation	Both	17 Apr 2026	24 Apr 2026
Final review & submission	Both	25 Apr 2026	27 Apr 2026

8. Implemented Components

This section provides a detailed breakdown of every module implemented in the NELLi platform, covering backend Spring Boot components and frontend React components for each.

8.1. Module A : Facilities & Assets Catalogue

Implementation Done by COORAY Y H

Module A provides the foundational resource registry for NELLi. It enables administrators to maintain a comprehensive catalogue of all bookable university assets lecture halls, computer laboratories, meeting rooms, and equipment items. Regular users can browse, search, and inspect resources before creating bookings.

8.1.1. Backend Components

Entity: Facility.java

- JPA entity annotated with @Entity, @Table(name = "facilities").
- Fields: id (UUID), name, type (FacilityType enum), capacity (Integer), location, description, status (FacilityStatus enum), availabilityWindows (JSONB via @Type), amenities (String[]), imageUrl, createdBy (User FK), timestamps.
- FacilityType enum: LECTURE_HALL, LAB, MEETING_ROOM, EQUIPMENT.
- FacilityStatus enum: ACTIVE, OUT_OF_SERVICE, MAINTENANCE.
- AvailabilityWindow embedded object: dayOfWeek (DayOfWeek enum), startTime (LocalTime), endTime (LocalTime).

Repository: FacilityRepository.java

- Extends JpaRepository<Facility, UUID>.
- Custom JPQL queries: findByTypeAndCapacityGreaterThanEqualAndLocation(...) for multi-criteria search.
- findByStatus(FacilityStatus status) for admin status filtering.
- findById with status ACTIVE check to exclude soft-deleted resources.

Service: FacilityService.java

- createFacility(FacilityRequest request, UUID adminId) - validates admin role, maps DTO to entity, persists, returns FacilityResponse.
- updateFacility(UUID id, FacilityRequest request) - full update with validation.
- patchFacility(UUID id, Map<String, Object> patches) - partial update using reflection-based patching.
- deleteFacility(UUID id) - soft delete by setting status to DELETED; returns 204 No Content.
- searchFacilities(String type, Integer minCapacity, String location) - dynamic filtering with JPA Specification or custom JPQL.
- getFacilityAvailability(UUID facilityId, LocalDate date) - queries BookingRepository to return available time slots for the requested date.

Controller: FacilityController.java

Endpoint	Description
`GET /api/v1/facilities`	List all active facilities; supports query params ?type=LAB&capacity=30&location=BuildingA
`GET /api/v1/facilities/{id}`	Get single facility by ID with full metadata and HATEOAS links
`POST /api/v1/facilities`	Create facility (ADMIN only) - returns 201 Created
`PUT /api/v1/facilities/{id}`	Full update of facility metadata (ADMIN only)
`PATCH /api/v1/facilities/{id}`	Partial update - e.g., status change only (ADMIN only)
`DELETE /api/v1/facilities/{id}`	Soft delete facility - sets status to DELETED (ADMIN only)

Endpoint	Description
`GET /api/v1/facilities/search`	Advanced search with multiple query parameters
`GET /api/v1/facilities/{id}/availability`	Return available time slots for a specific date

DTOs

- FacilityRequest: name (@NotBlank), type (@NotNull), capacity (@Positive), location (@NotBlank), availabilityWindows (List<AvailabilityWindowDTO>), amenities (List<String>).
- FacilityResponse: all entity fields plus _links (HATEOAS: self, bookings, availability, tickets, update, delete).

8.1.2. Frontend Components

Pages

- Facilities.jsx - Grid/list view of all facilities with real-time search bar and filter panel (type dropdown, capacity slider, location text). On load, calls facilityApi.getFacilities() and renders FacilityCard components.
- FacilityDetails.jsx - Detail view showing full metadata, availability calendar, and embedded BookingForm for authenticated users.
- AdminFacilities.jsx - Admin-only management page with add/edit/delete capabilities; uses FacilityForm modal.

Components

- FacilityCard.jsx - Card component displaying type badge, capacity, location, status chip, and Book Now CTA button.
- FacilityFilter.jsx - Filter panel with controlled inputs synchronized to URL query parameters.

- FacilityForm.jsx - Modal form for creating/editing facilities with availability window builder (add/remove day-time range pairs).

API Service: facilityApi.js

- getFacilities(params) - GET `/api/v1/facilities` with query string parameters.
- getFacilityById(id) - GET `/api/v1/facilities/{id}`.
- createFacility(data) - POST `/api/v1/facilities` with Authorization header.
- updateFacility(id, data) - PUT `/api/v1/facilities/{id}`.
- deleteFacility(id) - DELETE `/api/v1/facilities/{id}`.
- getFacilityAvailability(id, date) - GET `/api/v1/facilities/{id}/availability?date=...`

8.1.3. Non-Functional Requirements of Module A

- **Performance**:
  - Facility search results must load within **< 2 seconds** for up to 10,000 records
- **Scalability**:
  - Must support increasing number of facilities without performance degradation
- **Usability**:
  - Intuitive filtering (type, capacity, location) with responsive UI
- **Availability**:
  - System must ensure **99% uptime** for facility browsing
- **Data Integrity**:
  - Prevent invalid entries (e.g., negative capacity) using validation rules
- **Security**:
  - Only ADMIN users can modify facility data

8.2. Module B : Booking Management

Implementation Done by HIRUSHA D G A D

Module B implements the end-to-end resource booking workflow. Users submit booking requests specifying a facility, date/time range, and purpose. The system enforces conflict detection before persisting the request, then routes it through an admin approval workflow. Approved bookings trigger QR token generation and email notification.

8.2.1. Backend Components

Entity: Booking.java

- Fields: id (UUID), facility (ManyToOne @JoinColumn), user (ManyToOne), startDatetime (ZonedDateTime), endDatetime (ZonedDateTime), purpose, expectedAttendees, status (BookingStatus enum), adminNote, qrToken, contactEmail.
- BookingStatus enum: PENDING, APPROVED, REJECTED, CANCELLED.
- @PrePersist validation: endDatetime must be after startDatetime; endDatetime must be within facility availability windows.

Repository: BookingRepository.java

- findOverlappingBookings(UUID facilityId, ZonedDateTime start, ZonedDateTime end) - custom JPQL query checking for time range overlaps with APPROVED/PENDING status bookings.
- findByUserId(UUID userId, Pageable pageable) - paginated user booking history.
- findAllWithFilters(BookingStatus status, UUID facilityId, LocalDate date, Pageable pageable) - admin multi-filter query.

Service: BookingService.java

- createBooking(BookingRequest request, UUID userId) - Validates request; calls findOverlappingBookings to check for conflicts (throws BookingConflictException with HTTP 409 if conflict found); persists booking with PENDING status; triggers

BOOKING_REQUESTED notification to admin users; triggers confirmation email to user.

- approveBooking(UUID bookingId, UUID adminId) - Transitions status to APPROVED; generates unique QR token (UUID-based, stored in booking.qrToken); triggers BOOKING_APPROVED notification and email to user.

- rejectBooking(UUID bookingId, String reason, UUID adminId) - Transitions to REJECTED; stores reason in adminNote; triggers BOOKING_REJECTED notification and email with rejection reason.

- cancelBooking(UUID bookingId, UUID userId) - Only owner or ADMIN can cancel; only PENDING or APPROVED bookings may be cancelled; transitions to CANCELLED.

- getMyBookings(UUID userId, Pageable pageable) - Returns paginated user bookings sorted by createdAt DESC.

- getAllBookings(BookingStatus status, UUID facilityId, LocalDate date, Pageable pageable) - Admin endpoint with multi-parameter filtering.

Controller: BookingController.java

Endpoint	Description
`GET /api/v1/bookings`	List all bookings with filters (ADMIN) - ?status=PENDING&facilityId=...&date=...
`GET /api/v1/bookings/my-bookings`	Current authenticated user's bookings (paginated)
`GET /api/v1/bookings/{id}`	Get single booking details

Endpoint	Description
`POST` `/api/v1/bookings`	Submit booking request (USER) - returns 201 with PENDING status
`PUT` `/api/v1/bookings/{id}`	Update booking details before approval (USER, PENDING only)
`DELETE` `/api/v1/bookings/{id}`	Cancel booking (USER = own, ADMIN = any)
`PATCH` `/api/v1/bookings/{id}/approve`	Approve booking - transitions to APPROVED, generates QR (ADMIN)
`PATCH` `/api/v1/bookings/{id}/reject`	Reject with reason - transitions to REJECTED (ADMIN)

Conflict Detection Logic

Before persisting any new booking, BookingService executes the following JPQL query:

```
SELECT b FROM Booking b
WHERE b.facility.id = :facilityId
  AND b.status IN ('PENDING', 'APPROVED')
  AND b.startDatetime < :endDatetime
  AND b.endDatetime > :startDatetime
```

If any records are returned, a BookingConflictException is thrown and the API responds with HTTP 409 Conflict, including a message listing the conflicting booking IDs.

8.2.2 Frontend Components

Pages

- MyBookings.jsx - Tabbed view showing bookings by status (All / Pending / Approved / Rejected / Cancelled). Each booking card shows facility name, time range, status badge, and action buttons (Cancel for PENDING/APPROVED, View QR for APPROVED).
- CreateBooking.jsx - Multi-step booking form: Step 1 selects facility and date, Step 2 picks time slot from availability grid, Step 3 fills purpose and attendee count, Step 4 confirmation. Calls facilityApi.getFacilityAvailability() to populate available slots.
- AdminBookings.jsx - Admin view of all bookings with filter controls. Each row has Approve and Reject buttons with rejection reason modal.

Components

- BookingCalendar.jsx - Visual calendar component showing booked and available slots for a selected facility and date.
- BookingForm.jsx - Embedded booking request form used within FacilityDetails.jsx.
- BookingList.jsx - Reusable list component rendering booking cards with role-appropriate action buttons.

8.2.2. Non-Functional Requirements of Module B

- **Performance**:
  - Booking creation and validation (conflict detection) must complete within **< 1 second**
- **Consistency**:
  - No overlapping bookings allowed (strict conflict enforcement)
- **Reliability**:
  - Booking workflow states must always remain valid (PENDING → APPROVED/REJECTED)
- **Scalability**:
  - Must handle concurrent booking requests without race conditions

- **Security**:
  - Only authorized users can create or modify bookings
- **Availability**:
  - Booking system must be available during peak hours without downtime

8.3. Module C : Maintenance & Incident Ticketing

Implementation Done by COORAY Y H

Module C provides a structured workflow for reporting, tracking, and resolving facility maintenance issues and equipment incidents. Users can raise tickets with detailed descriptions, priority levels, and photographic evidence. Admins assign tickets to technicians, who then update status and add resolution notes.

8.3.1. Backend Components

Entities

- Ticket.java - Core entity with fields: id, ticketNumber (auto-generated), facility (FK), reportedBy (FK), assignedTo (FK, nullable), category (TicketCategory enum), priority (TicketPriority enum), status (TicketStatus enum), description, resolutionNotes, contactEmail, contactPhone, firstResponseAt, resolvedAt, timestamps.
- Comment.java - id, ticket (FK), user (FK), content (TEXT), isEdited (BOOL), createdAt, updatedAt. Business rule: only the comment author may edit/delete.
- Attachment.java - id, ticket (FK), fileName, fileUrl, fileSize (Long), mimeType, uploadedAt. Maximum 3 attachments per ticket enforced in service layer.

Enums

- TicketCategory - EQUIPMENT_FAILURE, STRUCTURAL_DAMAGE, ELECTRICAL, PLUMBING, SAFETY_HAZARD, OTHER.
- TicketPriority - LOW, MEDIUM, HIGH, CRITICAL.
- TicketStatus - OPEN, IN_PROGRESS, RESOLVED, CLOSED, REJECTED.

Service: TicketService.java

- createTicket(TicketRequest request, UUID userId) - Generates ticket number (TCK-YYYY-NNNNNN using AtomicLong sequence), persists ticket, triggers TICKET_CREATED notifications to admin users, sends confirmation email to reporter.

- updateTicketStatus(UUID ticketId, TicketStatus newStatus, String notes, UUID actorId) - Validates allowed transitions per actor role; records firstResponseAt when transitioning from OPEN to IN_PROGRESS; records resolvedAt on RESOLVED; triggers TICKET_STATUS_CHANGED notification to reporter.

- assignTicket(UUID ticketId, UUID technicianId, UUID adminId) - Admin assigns a TECHNICIAN-role user to ticket; triggers TICKET_ASSIGNED notification to technician.

- addComment(UUID ticketId, String content, UUID userId) - Persists comment; triggers NEW_COMMENT notification to ticket owner (if commenter is not the owner).

- editComment(UUID commentId, String content, UUID userId) - Validates that userId matches comment.user.id; updates content and sets isEdited = true.

- deleteComment(UUID commentId, UUID userId) - Validates ownership or ADMIN role; hard deletes comment.

- uploadAttachment(UUID ticketId, MultipartFile file, UUID userId) - Validates ticket attachment count (max 3); validates file type (JPEG, PNG, WEBP) and size (max 5MB); stores file; persists Attachment entity.

Controller: TicketController.java

Endpoint	Description
`GET /api/v1/tickets`	List all tickets with filters (ADMIN) - ?status=OPEN&priority=HIGH&facilityId=...
`GET /api/v1/tickets/my-tickets`	Current user's submitted tickets
`GET /api/v1/tickets/{id}`	Get single ticket with comments and attachments
`POST /api/v1/tickets`	Create incident ticket (USER)
`PUT /api/v1/tickets/{id}`	Update ticket details (owner, OPEN status only)
`DELETE /api/v1/tickets/{id}`	Delete ticket (owner, OPEN only)
`PATCH /api/v1/tickets/{id}/status`	Update ticket status with notes (ADMIN/TECHNICIAN)
`PATCH /api/v1/tickets/{id}/assign`	Assign technician to ticket (ADMIN)
`POST /api/v1/tickets/{id}/comments`	Add comment to ticket
`PUT /api/v1/tickets/{id}/comments/{commentId}`	Edit own comment
`DELETE /api/v1/tickets/{id}/comments/{commentId}`	Delete own comment (or ADMIN any)

Endpoint	Description
`POST` `/api/v1/tickets/{id}/attachments`	Upload image attachment (multipart/form-data, max 3)

8.3.2. Frontend Components

Pages

- ❖ MyTickets.jsx - List of user's tickets with priority colour-coded badges, status chips, and quick-filter tabs. Links to TicketDetails page.
- ❖ TicketDetails.jsx - Full ticket view: metadata panel, status timeline, image attachment gallery, comment thread (add/edit/delete with ownership check), and admin/technician action buttons (assign, status update with notes).
- ❖ CreateTicket.jsx - Multi-step form: facility selection, category and priority selection, description textarea with character counter (TextArea.jsx with counter hook), contact details, image upload with drag-and-drop (ImageUpload.jsx, max 3 files with preview).
- ❖ AdminTickets.jsx - Admin management view with bulk filter controls, assignee dropdown, and status update modal.

Components

- ❖ TicketCard.jsx - Summary card with priority indicator, status badge, facility name, ticket number, and created date.
- ❖ CommentSection.jsx - Comment thread component with add-comment form, edit/delete controls (shown conditionally based on ownership), and timestamps.

- ImageUpload.jsx - Drag-and-drop file upload with preview thumbnails, file validation (type and size), and maximum 3 file limit enforcement.

8.3.3. Non-Functional Requirements of Module C

- **Performance**:
  - Ticket creation and updates must respond within **< 2 seconds**
- **Reliability**:
  - Ticket lifecycle transitions must be strictly enforced
- **Usability**:
  - Easy-to-use interface for reporting issues with attachments
- **Scalability**:
  - Support large number of concurrent tickets and comments
- **Security**:
  - Only authorized users can edit/delete tickets and comments
- **Data Integrity**:
  - Maximum attachment limits and file validation enforced

8.4. Module D : Notifications & Email Service

Implementation Done by HIRUSHA D G A D

Module D provides a dual-channel notification system: in-app notifications accessible through the web UI notification panel, and outbound email notifications delivered via Gmail SMTP. Notifications are triggered automatically by system events in other modules (booking status changes, ticket updates, new comments).

8.4.1. Backend Components

Entity: Notification.java

- Fields: id (UUID), user (FK to recipient), type (NotificationType enum), title, message, relatedEntityType (BOOKING / TICKET / COMMENT), relatedEntityId (UUID), isRead (Boolean, DEFAULT false), createdAt.
- NotificationType enum: BOOKING_APPROVED, BOOKING_REJECTED, TICKET_STATUS_CHANGED, TICKET_ASSIGNED, NEW_COMMENT, SYSTEM_ALERT.

Service: NotificationService.java

- createNotification(UUID userId, NotificationType type, String title, String message, String entityType, UUID entityId) - Creates and persists Notification entity; called by BookingService and TicketService on key events.
- getUserNotifications(UUID userId, Pageable pageable) - Returns paginated notifications for the authenticated user, sorted by createdAt DESC.
- getUnreadCount(UUID userId) - Returns count of unread notifications for badge display.
- markAsRead(UUID notificationId, UUID userId) - Validates ownership; sets isRead = true.
- markAllAsRead(UUID userId) - Bulk update: sets isRead = true for all user's notifications.
- deleteNotification(UUID notificationId, UUID userId) - Validates ownership; hard deletes.
- getUserPreferences(UUID userId) / updatePreferences(UUID userId, NotificationPreferencesRequest req) - Manages per-user notification preference flags (e.g., emailOnBookingApproval, emailOnTicketUpdate).

Service: EmailService.java

- Implements JavaMailSender using Spring Boot Mail auto-configuration.
- sendBookingConfirmation(String toEmail, String userName, BookingResponse booking) - HTML email template with booking details.
- sendBookingApproved(String toEmail, String userName, BookingResponse booking) - Email with QR code download link.
- sendBookingRejected(String toEmail, String userName, String reason) - Email with rejection reason.
- sendTicketCreated(String toEmail, String userName, TicketResponse ticket) - Confirmation with ticket number.
- sendTicketStatusUpdate(String toEmail, String userName, TicketResponse ticket) - Status change alert.
- All email methods use MimeMessageHelper to support HTML content; error handling wraps sending in try-catch to prevent email failures from disrupting primary business operations.

Controller: NotificationController.java

Endpoint	Description
`GET /api/v1/notifications`	Get authenticated user's notifications (paginated)
`GET /api/v1/notifications/unread`	Get unread notification count for badge

Endpoint	Description
`PATCH /api/v1/notifications/{id}/read`	Mark single notification as read
`PATCH /api/v1/notifications/mark-all-read`	Mark all notifications as read
`DELETE /api/v1/notifications/{id}`	Delete single notification
`GET /api/v1/notifications/preferences`	Get notification preferences
`PATCH /api/v1/notifications/preferences`	Update notification preferences

8.4.2. Frontend Components

Components

- NotificationPanel.jsx - Dropdown panel accessible from Navbar bell icon. Shows unread count badge. Fetches notifications on open; renders NotificationItem list with mark-all-read button and link to full Notifications page.

- NotificationItem.jsx - Single notification row: type icon, title, truncated message, relative timestamp, read/unread visual state, click-to-navigate to related entity.

Pages

- Notifications.jsx - Full-page notification centre with preference management toggle controls (per notification category) and full notification history with pagination.

Context: NotificationContext.jsx

- ❖ Polls GET /api/v1/notifications/unread every 30 seconds for authenticated users.
- ❖ Provides unreadCount and triggerRefresh() to all consuming components.

8.4.3. Non-Functional Requirements of Module D

- ❖ **Performance**:
  - Notifications must be generated in **real-time (< 1 second delay)**
- ❖ **Reliability**:
  - Email failures must not affect core system operations
- ❖ **Availability**:
  - Notification system must operate continuously
- ❖ **Scalability**:
  - Must support high volume of notifications without delay
- ❖ **Usability**:
  - Clear, readable notification messages and UI indicators
- ❖ **Security**:
  - Notifications must only be accessible to intended users

8.5. Module E : Authentication & Authentication

OAuth2 + Security Configuration Done by HIRUSHA D G A D

JWT Authentication Done by COORAY Y H

Module E secures the entire platform through a two-mechanism authentication system: Google OAuth 2.0 for social login and traditional email/password credentials. Both flows issue JWT tokens consumed by all secured endpoints. Role-based access control is enforced at the Spring Security layer for backend endpoints and via React route guards on the frontend.

8.5.1. Backend Components

Security Configuration: SecurityConfig.java

- ❖ Extends WebSecurityConfigurerAdapter (Spring Security 5) / SecurityFilterChain bean (Spring Security 6).
- ❖ Configures CSRF disabled (stateless JWT API), CORS settings (CorsConfig.java: allows http://localhost:5173 in development), and session management as STATELESS.
- ❖ Adds JwtAuthenticationFilter before UsernamePasswordAuthenticationFilter in the filter chain.
- ❖ Defines permit-all paths: /api/v1/auth/**, /swagger-ui/**, /v3/api-docs/**, /api/v1/facilities (GET only, for public browsing).
- ❖ Configures OAuth2 login with success handler: .oauth2Login().successHandler(oAuth2SuccessHandler).

JWT Components

- ❖ JwtTokenProvider.java - Uses JJWT library. generateToken(UserDetails user) creates a signed JWT with subject (userId), roles claim, and configurable expiration (default 24 hours). validateToken(String token) and extractClaims(String token) support filter validation.
- ❖ JwtAuthenticationFilter.java - OncePerRequestFilter that extracts Bearer token from Authorization header, validates it via JwtTokenProvider, loads UserDetails from CustomUserDetailsService, and sets the Authentication in SecurityContextHolder.
- ❖ JwtConfig.java - Reads jwt.secret and jwt.expiration from application.properties (injected via @Value). Secret is stored as environment variable (JWT_SECRET).

OAuth2 Components

- ❖ OAuth2SuccessHandler.java - Implements AuthenticationSuccessHandler. On successful Google sign-in: extracts Google subject ID and email from OAuth2User attributes; checks if user exists by email or providerId; creates new user account if first login (role = USER); generates JWT access token; redirects to frontend callback URL with token as query parameter: /auth/callback?token=...

- ❖ CustomUserDetailsService.java - Implements UserDetailsService; loads user by email from UserRepository; wraps in Spring Security UserDetails with granted authorities derived from the user's Role.

Auth Controller: AuthController.java

Endpoint	Description
`POST /api/v1/auth/register`	Register with email, password, name - returns JWT token pair
`POST /api/v1/auth/login`	Login with credentials - returns access + refresh tokens
`POST /api/v1/auth/google`	Exchange Google ID token for NELLi JWT - used by frontend OAuth callback
`POST /api/v1/auth/refresh-token`	Exchange refresh token for new access token
`POST /api/v1/auth/logout`	Invalidate token (adds to server-side denylist if implemented)
`GET /api/v1/auth/me`	Return current authenticated user's profile

Role-Based Access Summary

Role	Capabilities	Example Endpoints
USER	Browse facilities, create bookings, submit tickets, manage own comments, view own notifications	GET /facilities, POST /bookings, POST /tickets
ADMIN	All USER capabilities + approve/reject bookings, manage facilities, assign tickets, change statuses, view analytics, manage user roles	PATCH /bookings/{id}/approve, POST /facilities, GET /analytics
TECHNICIAN	View assigned tickets, update ticket status, add resolution notes, add comments	PATCH /tickets/{id}/status, POST /tickets/{id}/comments

8.5.2. Frontend Components

- Login.jsx - Dual-mode login page: email/password form and Google Sign-In button. Calls authApi.login() or redirects to backend OAuth2 initiation URL.
- Register.jsx - Multi-step registration form with email validation and password strength indicator.
- OAuthCallback.jsx - Intercepts /auth/callback?token=... redirect from OAuth2SuccessHandler; extracts token from URL; stores in AuthContext; redirects to dashboard.
- AuthContext.jsx - Stores user object (decoded from JWT), access token, and role. Provides login(), logout(), isAuthenticated, and hasRole() methods to all consuming components.
- PrivateRoute.jsx - HOC that checks isAuthenticated; redirects to /login if not authenticated.
- AdminRoute.jsx - Extends PrivateRoute; additionally checks hasRole('ADMIN'); redirects to home if insufficient role.

8.5.3. Non-Functional Requirements of Module E

- **Security (Critical)**:
  - JWT-based authentication with encrypted tokens
  - OAuth 2.0 integration for secure login
- **Performance**:
  - Authentication process must complete within **< 1 second**
- **Reliability**:
  - Token validation must be consistent and fail-safe
- **Availability**:
  - Authentication services must be highly available
- **Scalability**:
  - Must handle multiple concurrent login requests
- **Compliance**:
  - Follow secure password handling and token management practices

8.6. Module F - QR Code Check-In for Approved Bookings (Innovative Feature)

Implementation Done by HIRUSHA D G A D

Module F implements a lightweight physical verification system for approved facility bookings. When a booking is approved, a unique QR code is generated linking to a verification URL. Users can present this QR code at a facility entrance kiosk or to a staff member, who scans it to instantly confirm booking validity without system login.

8.6.1. Backend Implementation

- QR Token Generation - BookingService.approveBooking() generates a cryptographically random UUID-based token stored in booking.qrToken. The token encodes the bookingId, facilityId, and an expiry timestamp.

- QR Verification Endpoint - GET /api/v1/bookings/verify/{qrToken} is a PUBLIC endpoint (no authentication required). It accepts the QR token, looks up the booking, validates that (a) the booking is APPROVED, (b) the current datetime falls within the booking's startDatetime - endDatetime window, and returns a verification response containing facility name, user name, time slot, and purpose.
- The QR token is invalidated (nulled) after first use to prevent re-entry fraud, with a configurable grace period.

- QR code generation uses the ZXing library on the backend to generate a QR code image (PNG) accessible at GET /api/v1/bookings/{id}/qr-code - returns the image as application/octet-stream.

8.6.2. Frontend Implementation

- QR Code Display - In MyBookings.jsx, APPROVED bookings show a View QR button. Clicking opens a modal rendering the QR code using the qrcode.react library, pointed at the public verification URL.

- Download option - Users can download the QR code PNG for printing or digital storage.

- Verification Screen - The route /verify/:token renders a public page (no auth required) that calls the verification endpoint and displays either a green VALID badge (with booking details) or a red INVALID/EXPIRED badge. This is the screen shown at facility kiosks.

- The verification page is mobile-responsive, designed for large-screen display at entry points and quick visual confirmation.

8.6.3. Non-Functional Requirements of Module F

- **Performance**:
    - QR verification must respond within **< 1 second**
- **Security**:

- QR tokens must be unique, time-bound, and non-reusable

❖ **Reliability**:

- Verification must accurately validate booking status and time window

❖ **Usability**:

- QR codes must be easily scannable and accessible

❖ **Availability**:

- Public verification endpoint must be always accessible

❖ **Scalability**:

- Must support multiple simultaneous QR scans

8.7. Module G - Admin Analytics Dashboard (Innovative Feature)

Implementation Done by HIRUSHA D G A D and COORAY Y H

Module G delivers an executive-level analytics dashboard for ADMIN users, providing real-time visibility into facility utilisation patterns, booking trends, and maintenance ticket KPIs. All analytics are computed server-side using aggregation queries to ensure consistent, accurate metrics.

8.7.1. Backend Components

Analytics DTOs

❖ DashboardSummaryDTO - totalFacilities, activeFacilities, totalBookings, pendingBookings, totalTickets, openTickets, resolvedTickets, totalUsers.

❖ FacilityUsageDTO - facilityId, facilityName, totalBookings, totalHoursBooked, utilizationRate (percentage of available hours booked).

- ❖ BookingTrendDTO - date (LocalDate), bookingCount - supports time-series charting.
- ❖ PeakHoursDTO - hour (0-23), averageBookingsPerDay - identifies peak demand periods.
- ❖ TicketMetricsDTO - avgTimeToFirstResponse (hours), avgTimeToResolution (hours), ticketsByCategory (Map<TicketCategory, Long>), ticketsByPriority (Map<TicketPriority, Long>).

Analytics Endpoints

Endpoint	Description
`GET /api/v1/analytics/dashboard`	Overall summary counts (ADMIN)
`GET /api/v1/analytics/facilities/usage`	Top 10 most-booked facilities with utilization rates
`GET /api/v1/analytics/bookings/trends?period=30d`	Booking count per day for specified period
`GET /api/v1/analytics/bookings/peak-hours`	Average bookings by hour of day
`GET /api/v1/analytics/tickets/metrics`	Ticket resolution KPIs and distribution
`GET /api/v1/analytics/users/growth?period=90d`	New user registrations over time

Implementation Notes

- ❖ All analytics queries use JPQL aggregation functions: COUNT, SUM, AVG, EXTRACT(HOUR FROM ...).
- ❖ Results are cached using Spring Cache (@Cacheable) with a 5-minute TTL to reduce database load for frequently accessed dashboard data.

Date range parameters use ISO 8601 duration format (e.g., 30d, 90d) parsed in a utility method.

8.7.2. Frontend Components

- ❖ AdminDashboard.jsx - Summary card row (KPI widgets): total bookings, pending approvals, open tickets, active facilities. Quick-action buttons for common admin tasks.
- ❖ AdminAnalytics.jsx - Full analytics page with four chart panels:
  - Booking Trends: Recharts LineChart plotting booking count per day over the selected time period.
  - Peak Hours: Recharts BarChart showing average bookings by hour of day, highlighting 9-11 AM and 2-4 PM as typical peak periods.
  - Top Facilities: Recharts HorizontalBarChart ranking facilities by total bookings and utilization rate.
  - Ticket KPIs: Recharts PieChart showing ticket distribution by category and priority; KPI cards for average response and resolution times.
- ❖ Period selector (7d / 30d / 90d) controls all charts simultaneously.

analyticsApi.js - API service module with dedicated functions for each analytics endpoint.

8.7.3. Non-Functional Requirements of Module G

- **Performance**:
  - Dashboard data must load within **< 3 seconds**
- **Scalability**:
  - Must handle large datasets efficiently using aggregation queries
- **Accuracy**:
  - Analytics must reflect real-time or near real-time data
- **Usability**:
  - Clear visualization (charts, graphs, KPIs) for decision-making
- **Availability**:
  - Dashboard must be accessible to ADMIN users at all times
- **Reliability**:
  - Cached data must refresh periodically to ensure consistency

9. REST API Architecture

9.1. API Design Principle

- Base URL: http://localhost:8080/api/v1 - All endpoints are versioned under /v1 to enable non-breaking future updates.

- Resource naming: Plural nouns for collections (/facilities, /bookings, /tickets), kebab-case for multi-word paths (/my-bookings, /mark-all-read).

- HTTP methods: Strict semantic use - GET (read), POST (create), PUT (full update), PATCH (partial update / state change), DELETE (remove or cancel).

- HTTP status codes: 200 OK (read/update), 201 Created (POST), 204 No Content (DELETE), 400 Bad Request (validation), 401 Unauthorized, 403 Forbidden (role), 404 Not Found, 409 Conflict (booking overlap), 422 Unprocessable Entity (business rule violation), 500 Internal Server Error.

- HATEOAS: All response bodies include a _links object with rel-href-method tuples for discoverability (self, related resources, available actions based on current state and role).

- Pagination: All list endpoints support ?page=0&size=20&sort=createdAt,desc parameters via Spring Data Pageable.

- Consistent error response format: { timestamp, status, error, message, path, validationErrors? }.

9.2. Complete Endpoint Inventory

Module	Endpoints	Methods Used
Authentication	6 endpoints	GET, POST

Module	Endpoints	Methods Used
Module A - Facilities	8 endpoints	GET, POST, PUT, PATCH, DELETE
Module B - Bookings	8 endpoints	GET, POST, PUT, PATCH, DELETE
Module C - Tickets	12 endpoints	GET, POST, PUT, PATCH, DELETE
Module D - Notifications	7 endpoints	GET, PATCH, DELETE
User Management	4 endpoints	GET, PUT, PATCH
Module F - QR Verification	2 endpoints	GET
Module G - Analytics	6 endpoints	GET
TOTAL	53 endpoints	All HTTP methods represented

9.3. Standardized Error Response Format

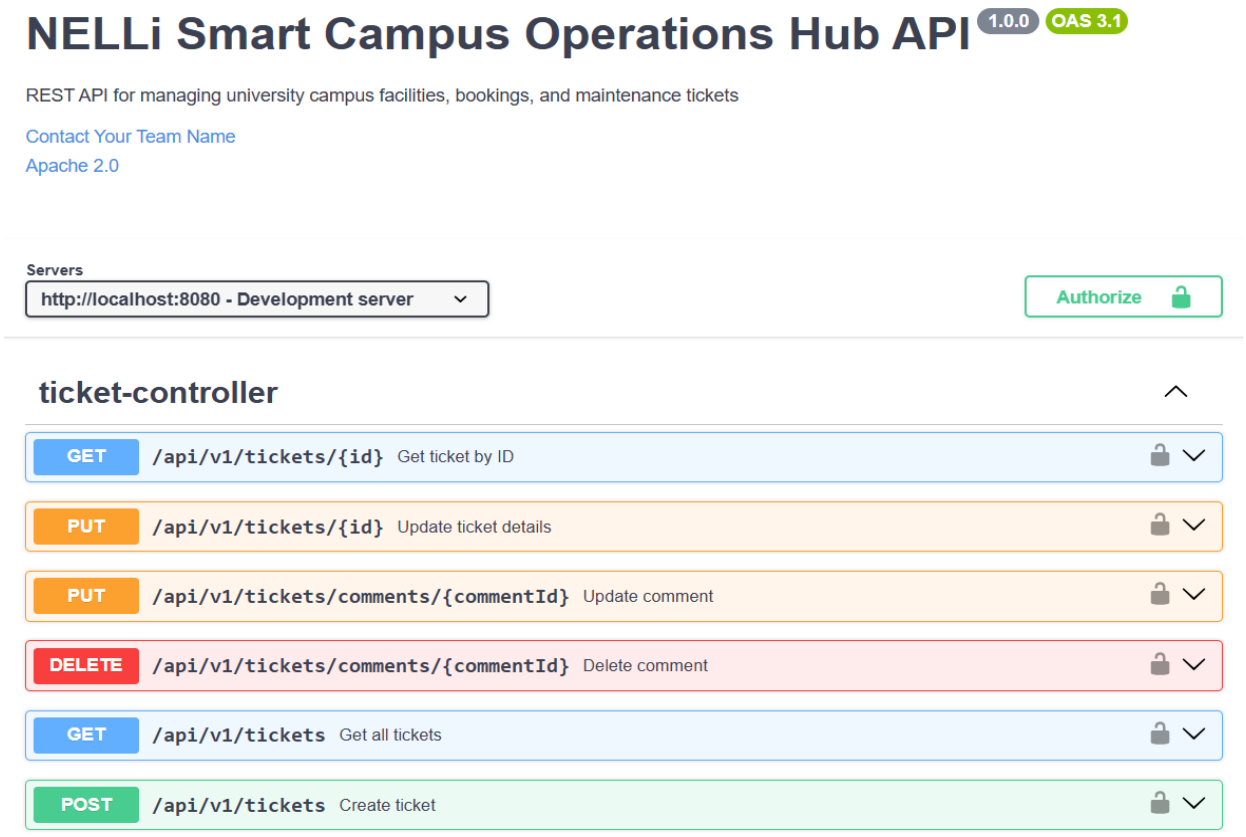
```
// Example: POST /api/v1/bookings - Validation Error (400)
{
  "timestamp": "2026-04-15T09:30:00Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed for 1 field(s)",
  "path": "/api/v1/bookings",
  "validationErrors": {
    "endDateTime": "End date must be after start date"
  }
}

// Example: POST /api/v1/bookings - Conflict (409)
```

```
{
  "timestamp": "2026-04-15T09:30:00Z",
  "status": 409,
  "error": "Booking Conflict",
  "message": "The requested time slot overlaps with an existing booking",
  "path": "/api/v1/bookings",
  "conflictingBookingId": "booking-456"
}
```

9.4. API Documentation

The backend exposes a Swagger UI at http://localhost:8080/swagger-ui.html, auto-generated by SpringDoc OpenAPI. All endpoints are documented with request/response schemas, example payloads, authentication requirements, and possible error codes. The OpenAPI specification is available in JSON format at /v3/api-docs.



Servers

http://localhost:8080 - Development server

Authorize

GET	/api/v1/tickets/{id}/comments	Get ticket comments
POST	/api/v1/tickets/{id}/comments	Add comment
GET	/api/v1/tickets/{id}/attachments	Get ticket attachments
POST	/api/v1/tickets/{id}/attachments	Upload attachment
PATCH	/api/v1/tickets/{id}/status	Update ticket status
PATCH	/api/v1/tickets/{id}/assign	Assign ticket to technician
GET	/api/v1/tickets/my-tickets	Get my tickets
GET	/api/v1/tickets/assigned	Get assigned tickets
DELETE	/api/v1/tickets/attachments/{attachmentId}	Delete attachment

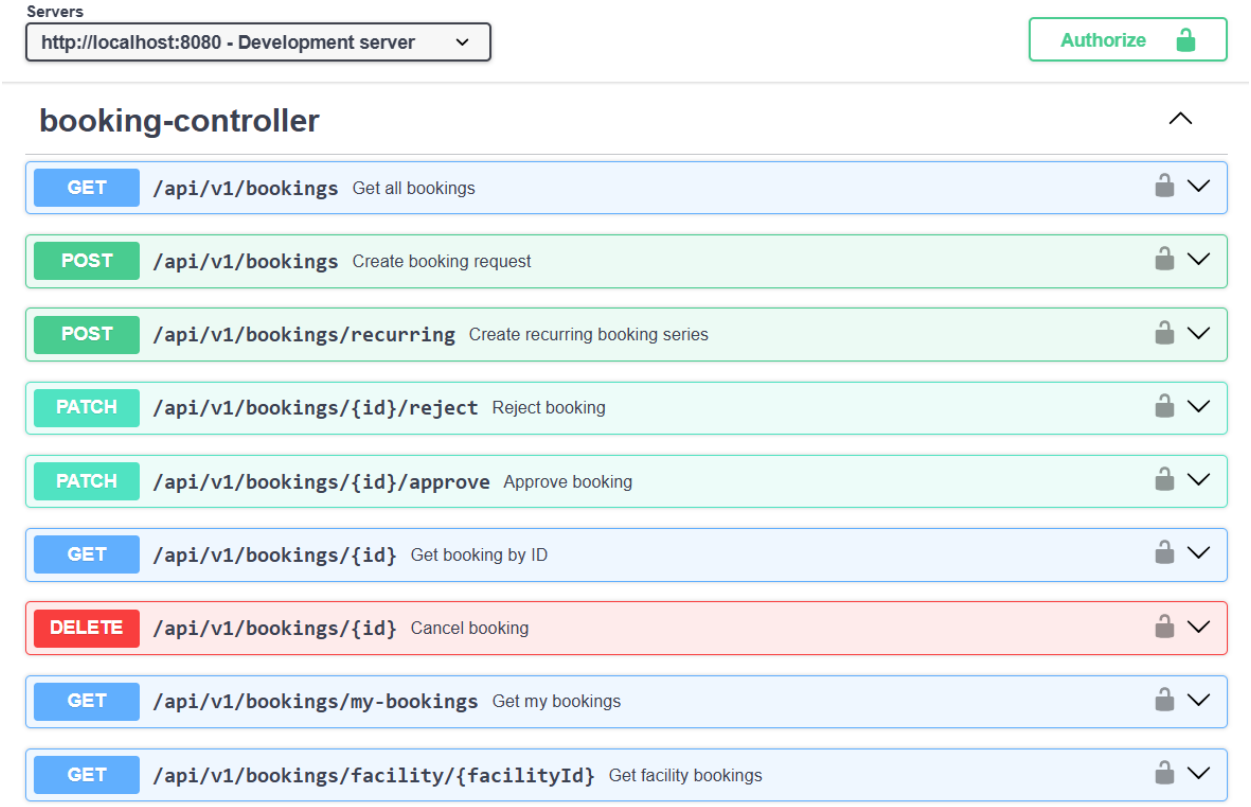
Servers

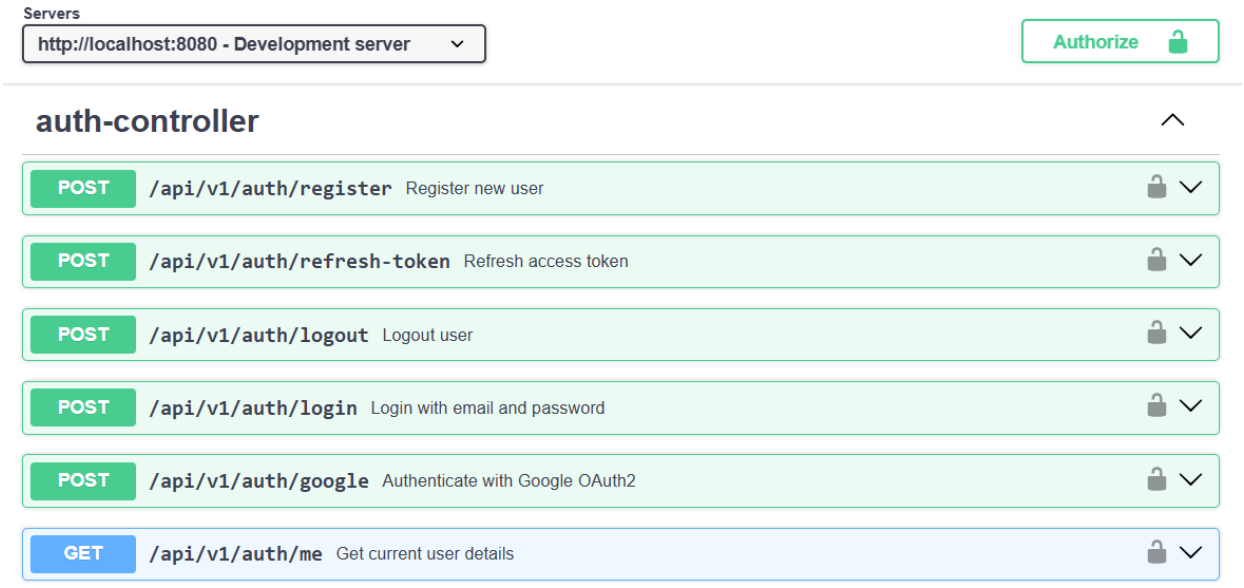
http://localhost:8080 - Development server

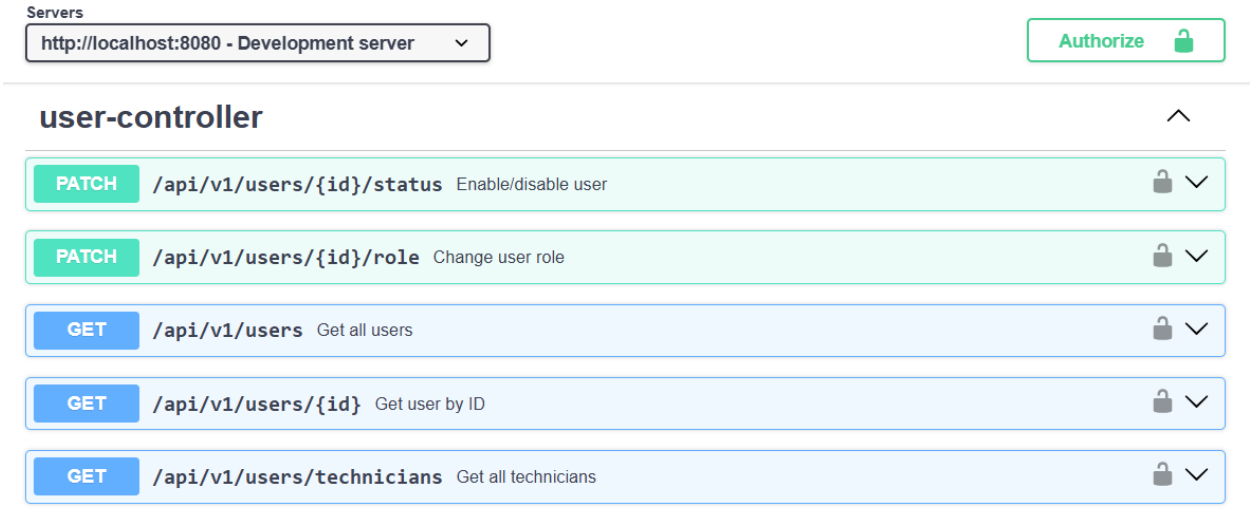
Authorize

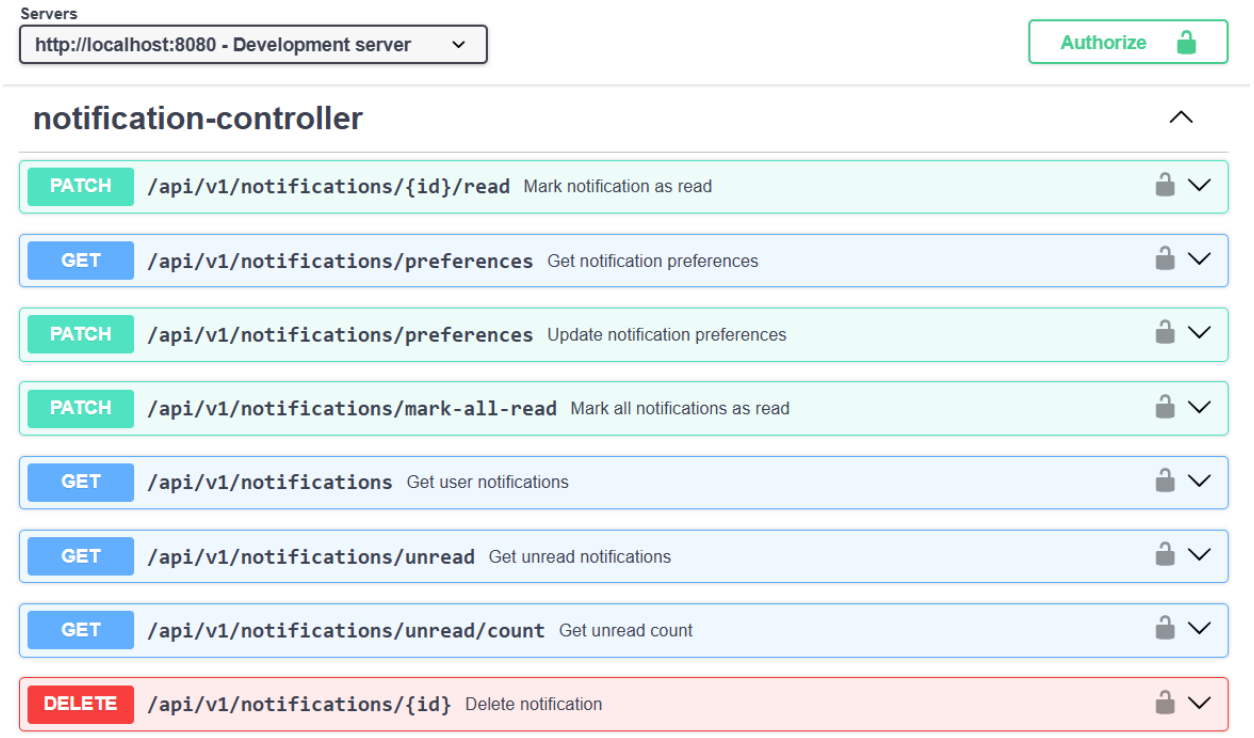
facility-controller

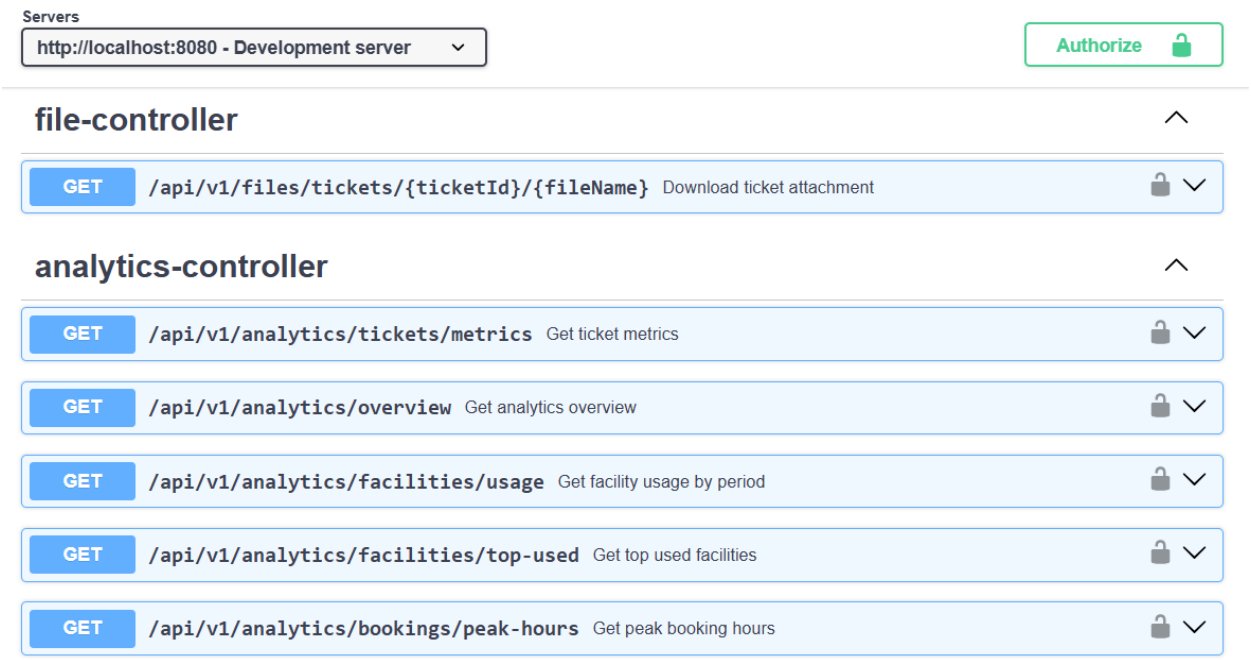
GET	/api/v1/facilities/{id}	Get facility by ID
PUT	/api/v1/facilities/{id}	Update facility
DELETE	/api/v1/facilities/{id}	Delete facility
GET	/api/v1/facilities	Get all facilities
POST	/api/v1/facilities	Create new facility
GET	/api/v1/facilities/{id}/bookings	Get facility bookings
GET	/api/v1/facilities/{id}/availability	Get approved facility availability slots
GET	/api/v1/facilities/search	Search facilities


Servers
http://localhost:8080 - Development server
Authorize
booking-controller
GET /api/v1/bookings Get all bookings
POST /api/v1/bookings Create booking request
POST /api/v1/bookings/recurring Create recurring booking series
PATCH /api/v1/bookings/{id}/reject Reject booking
PATCH /api/v1/bookings/{id}/approve Approve booking
GET /api/v1/bookings/{id} Get booking by ID
DELETE /api/v1/bookings/{id} Cancel booking
GET /api/v1/bookings/my-bookings Get my bookings
GET /api/v1/bookings/facility/{facilityId} Get facility bookings


Servers
http://localhost:8080 - Development server
Authorize
auth-controller
POST /api/v1/auth/register Register new user
POST /api/v1/auth/refresh-token Refresh access token
POST /api/v1/auth/logout Logout user
POST /api/v1/auth/login Login with email and password
POST /api/v1/auth/google Authenticate with Google OAuth2
GET /api/v1/auth/me Get current user details


Servers
http://localhost:8080 - Development server
Authorize
user-controller
PATCH /api/v1/users/{id}/status Enable/disable user
PATCH /api/v1/users/{id}/role Change user role
GET /api/v1/users Get all users
GET /api/v1/users/{id} Get user by ID
GET /api/v1/users/technicians Get all technicians


Servers
http://localhost:8080 - Development server
Authorize
notification-controller
PATCH /api/v1/notifications/{id}/read Mark notification as read
GET /api/v1/notifications/preferences Get notification preferences
PATCH /api/v1/notifications/preferences Update notification preferences
PATCH /api/v1/notifications/mark-all-read Mark all notifications as read
GET /api/v1/notifications Get user notifications
GET /api/v1/notifications/unread Get unread notifications
GET /api/v1/notifications/unread/count Get unread count
DELETE /api/v1/notifications/{id} Delete notification


Servers
http://localhost:8080 - Development server
Authorize
file-controller
GET /api/v1/files/tickets/{ticketId}/{fileName} Download ticket attachment
analytics-controller
GET /api/v1/analytics/tickets/metrics Get ticket metrics
GET /api/v1/analytics/overview Get analytics overview
GET /api/v1/analytics/facilities/usage Get facility usage by period
GET /api/v1/analytics/facilities/top-used Get top used facilities
GET /api/v1/analytics/bookings/peak-hours Get peak booking hours

10. Testing & Quality Management w/ Postman

10.1. Testing Strategy

NELLi's quality management approach covers three distinct testing layers: automated unit and integration tests in the Spring Boot backend, manual API testing via Postman collections, and frontend component validation through manual workflow testing.

10.2. Backend Unit & Integration Tests

10.2.1. Service Layer Unit Tests

- BookingServiceTest.java - Tests conflict detection logic with mocked BookingRepository. Scenarios: no conflict (creates booking), same facility different time (no conflict), overlapping APPROVED booking (throws BookingConflictException), overlapping PENDING booking (throws exception), REJECTED booking in slot (no conflict - should succeed).
- FacilityServiceTest.java - Tests CRUD operations, search filtering, and availability calculation. Uses Mockito for repository mocking.

- TicketServiceTest.java - Tests ticket creation with number generation, status transition validation, attachment count enforcement (rejects 4th attachment), and comment ownership rules.

- NotificationServiceTest.java - Tests notification creation triggers and unread count queries.

10.2.2. Controller Integration Tests

- BookingControllerIntegrationTest.java - Uses @SpringBootTest and @AutoConfigureMockMvc. Tests full request/response cycle for booking creation (with JWT token), approval workflow, and conflict response (expecting 409).

- FacilityControllerIntegrationTest.java - Tests CRUD endpoints, search filtering, and role-based access (expects 403 for USER attempting to POST /facilities).

- AuthControllerIntegrationTest.java - Tests registration, login, and token refresh flows.

10.3. Postman Testing

10.3.1. Collection Structure

A comprehensive Postman collection is organised as follows:

- NELLi API v1 - Root collection with environment variables: baseUrl, adminToken, userToken, techToken.

- Auth - All authentication endpoint tests (register, login, OAuth exchange, token refresh, /me).

- Facilities - CRUD tests including search with various filter combinations, availability check.
- Bookings - Full booking lifecycle: create → admin approve → user view QR → cancel. Conflict scenario tests.

- Tickets - Create ticket, upload attachment, add comment, update status, assign technician, resolve.

- Notifications - Fetch, mark read, bulk mark read, preference update.

- Analytics - All dashboard and analytics endpoints (admin token required).

- QR Verification - Verify valid token, verify expired token, verify invalid token.

10.3.2. Test Scripts

Each Postman request includes Tests tab scripts for automated responses validation:

```
// Example: POST /api/v1/bookings test script
pm.test("Status is 201 Created", () => {
    pm.response.to.have.status(201);
});
pm.test("Response has booking ID", () => {
    const body = pm.response.json();
    pm.expect(body).to.have.property("id");
    pm.environment.set("bookingId", body.id);
});
pm.test("Status is PENDING", () => {
    pm.expect(pm.response.json().status).to.equal("PENDING");
});
pm.test("HATEOAS links present", () => {
    pm.expect(pm.response.json()._links).to.have.property("self");
    pm.expect(pm.response.json()._links).to.have.property("approve");
});
```

10.3.3. Test Coverage Summary

Module	# Requests	# Test Assertions	Edge Cases Covered
Authentication	8	24	Invalid credentials, expired token, duplicate email
Facilities	10	35	Search filter combos, soft delete, role enforcement
Bookings	12	48	Conflict detection, workflow transitions, role checks
Tickets	15	52	Attachment limit, comment ownership, status transitions

Module	# Requests	# Test Assertions	Edge Cases Covered
Notifications	8	20	Unread count accuracy, bulk read, preference persistence
Analytics	6	18	Admin-only access, date range validation
QR / Verify	4	12	Valid scan, expired, already used, invalid token
TOTAL	63	209	Full happy-path + error scenarios

9.4. Input Validation & Error Handling

- All request DTOs use Jakarta Bean Validation annotations. @NotBlank, @NotNull, @Email, @Size, @Min, @Max, @Future (for booking dates) are applied at the DTO level.

- @Valid annotation on all @RequestBody parameters activates validation; MethodArgumentNotValidException is caught by GlobalExceptionHandler and returns structured 400 responses.

- Custom validators: @ValidTimeRange checks that endDateTime > startDateTime; @ValidBookingWindow checks that the requested time fits within facility availability windows.

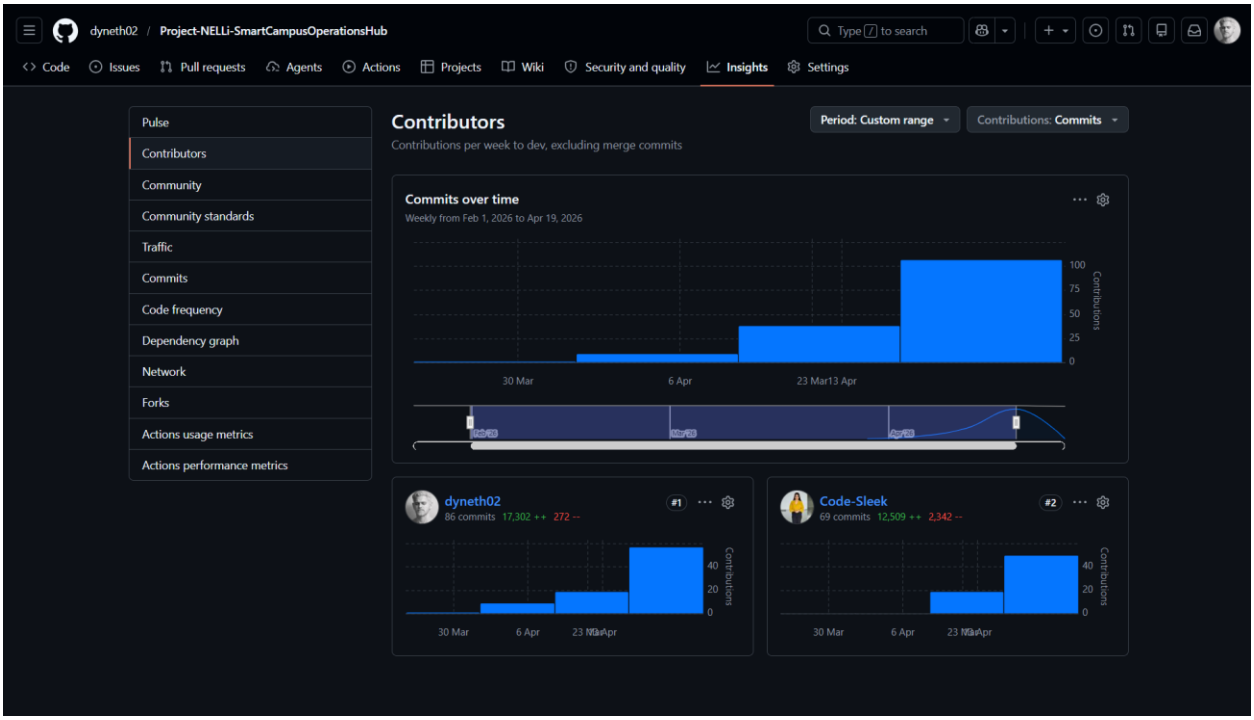
- Business rule violations (e.g., booking a resource with OUT_OF_SERVICE status) throw custom exception classes (ResourceUnavailableException) with meaningful HTTP status codes.

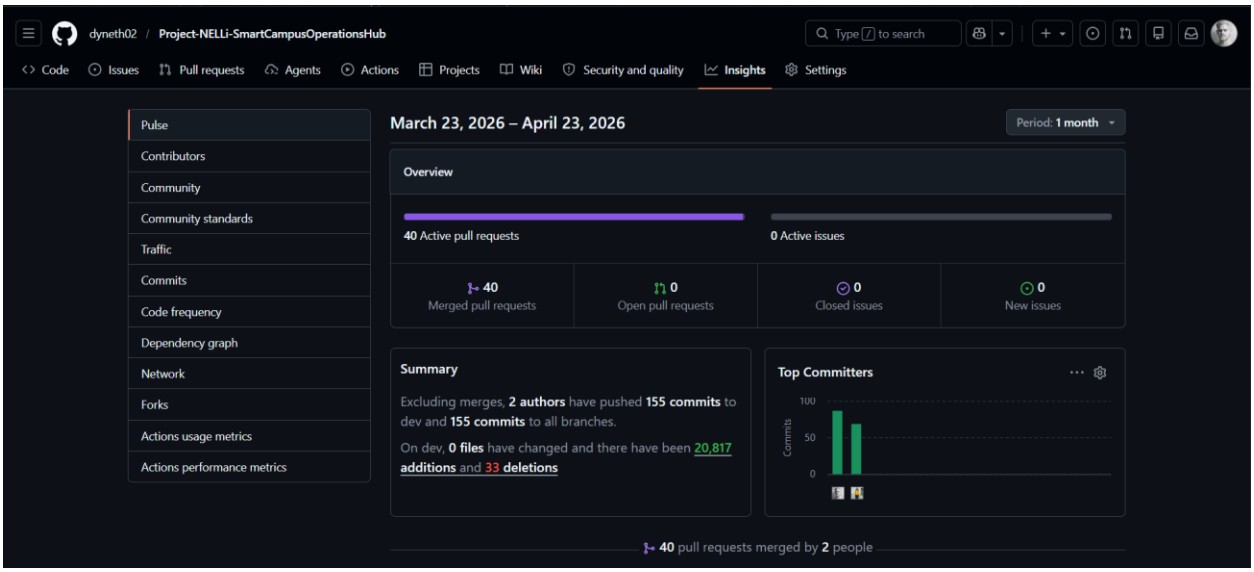
Safe file handling: attachment uploads validate MIME type from file header bytes (not just extension), enforce size limits (5MB), and sanitize file names before storage.

11. Version Control w/ Github

11.1. Repository Structure (Source Code Links)

Detail	Value
Repository Name	Project-NELLi-SmartCampusOperationsHub
Source Code Link (One Drive)	Nelli-Project-Source-Code-in_OneDrive
Primary Branch	main (production-ready releases)
Development Branch	dev (integration branch for feature merges)
Feature Branches	28 feature branches (numbered 001-028)
Total Merged Pull Requests	40 Merged Pull Requests
Branch Naming Convention	feature/{number}/{member}/{feature-name}
Total Commits	~150+ commits across all branches





11.2. Branching Strategy

The project adopts a structured GitFlow-inspired branching model adapted for a two-member team:

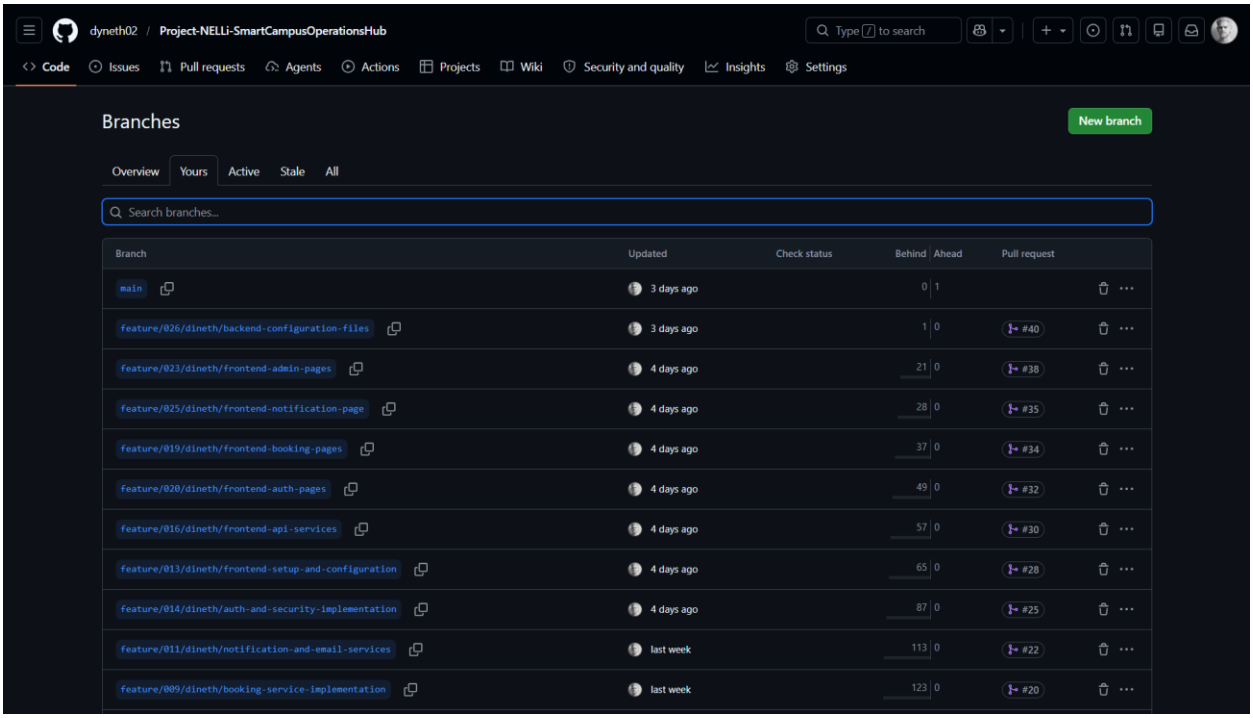
- ❖ main - Contains only release-ready, tested code. Merged from develop after milestone completion.
- ❖ develop - Integration branch. All feature branches are merged here via Pull Request after self-review.
- ❖ feature/* - Individual feature branches, one per task unit, named with developer identity and sequential numbering for traceability.

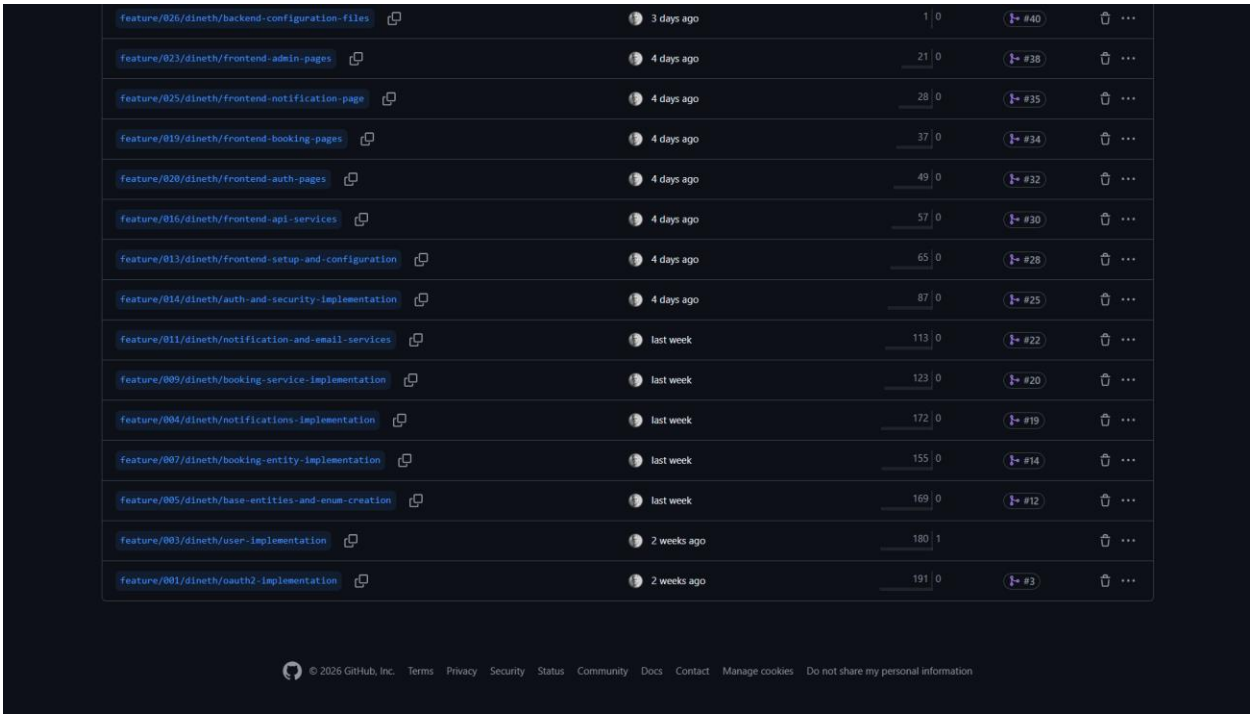
11.3. Complete Branch Registery

Branch ID	Branch Name	Owner	Scope
001	feature/001/dineth/oauth2-implementation	Dineth	Google OAuth2 + Spring Security setup
002	feature/002/yashodha/jwt-authentication-implementation	Yashodha	JWT provider, filter, config classes
003	feature/003/dineth/user-implementation	Dineth	User entity, repository, user service

Branch ID	Branch Name	Owner	Scope
004	feature/004/dineth/notifications-implementation	Dineth	Notification entity & early triggers
005	feature/005/dineth/base-entities-and-enums	Dineth	All enums and base entity classes
006	feature/006/yashodha/facility-entity-implementation	Yashodha	Facility entity, repo, DTOs
007	feature/007/dineth/booking-entity-implementation	Dineth	Booking entity, repository, DTOs
008	feature/008/yashodha/ticket-entity-implementation	Yashodha	Ticket, Comment, Attachment entities
009	feature/009/dineth/notification-entity-implementation	Dineth	Notification entity refinement
010	feature/010/yashodha/facility-service-implementation	Yashodha	FacilityService + FacilityController
011	feature/011/dineth/booking-service-implementation	Dineth	BookingService with conflict detection
012	feature/012/yashodha/ticket-service-implementation	Yashodha	TicketService, CommentService, AttachmentService
013	feature/013/dineth/notification-and-email-services	Dineth	NotificationService + EmailService (SMTP)
014	feature/014/dineth/auth-and-security-implementation	Dineth	AuthController, SecurityConfig, CORS
015	feature/015/yashodha/analytics-and-common-dtos	Yashodha	AnalyticsService, DTOs, QR endpoints
016	feature/016/yashodha/frontend-setup-and-configuration	Yashodha	Vite + React project init, Tailwind, Axios config
017	feature/017/dineth/frontend-common-components	Dineth	Button, Input, Modal, Card, Badge, Spinner
018	feature/018/yashodha/frontend-layout-components	Yashodha	Navbar, Sidebar, Footer, MainLayout, AdminLayout
019	feature/019/dineth/frontend-api-services	Dineth	All API service modules (facilityApi, bookingApi, etc.)
020	feature/020/dineth/frontend-auth-pages	Dineth	Login, Register, OAuthCallback, NotFound pages
021	feature/021/dineth/frontend-home-and-dashboard	Dineth	Home, Dashboard, Profile pages

Branch ID	Branch Name	Owner	Scope
022	feature/022/yashodha/frontend-facility-pages	Yashodha	Facilities, FacilityDetails pages
023	feature/023/dineth/frontend-booking-pages	Dineth	MyBookings, CreateBooking pages
024	feature/024/yashodha/frontend-ticket-pages	Yashodha	MyTickets, TicketDetails, CreateTicket pages
025	feature/025/dineth/frontend-notification-page	Dineth	Notifications page + preference management
026	feature/026/Dineth/qr-pass-implementation	Dineth	BookingVerify.jsx, bookingApi.js, QrCodeService.java, QrVerificationController.java
027	feature/027/yashodha/frontend-admin-pages	Yashodha	AdminDashboard, AdminFacilities, AdminBookings, AdminTickets, AdminUsers, AdminAnalytics
028	feature/028/dineth/backend-configuration-files	Dineth	pom.xml, application.properties, README.md





11.4. Commit Message Convention

All commits follow a standardized convention to ensure traceability and clear history:

```
Format: prefix(type): Descriptive message

Prefixes:
  impl     – New feature implementation
  upd      – Update to existing feature
  fix      – Bug fix
  refactor – Code restructure without behavior change
  config   – Configuration changes
  docs     – Documentation updates
  test     – Test additions or changes
  style    – Formatting / style only
```

```
Types (in parentheses):
  (model)      – JPA entity
  (service)    – Service layer
  (controller) – REST controller
  (dto)        – Data Transfer Object
  (security)   – Security component
  (config)     – Configuration class
  (component)  – React component
  (page)       – React page component
  (api)        – API service module
  (hooks)      – Custom React hooks

Examples:
```

- `impl(model): Created Booking entity with status enum and FK relationships`
- `impl(service): Added conflict detection logic in BookingService`
- `impl(controller): Created BookingController with PATCH approve endpoint`
- `config(backend): Created application.properties with Neon DB and SMTP settings`
- `impl(page): Created MyBookings page with status filter tabs`

11.5. GitFlow WorkFlow

```
# 1. Create feature branch from develop
   git checkout develop
   git pull origin develop
   git checkout -b feature/011/dineth/booking-service-implementation

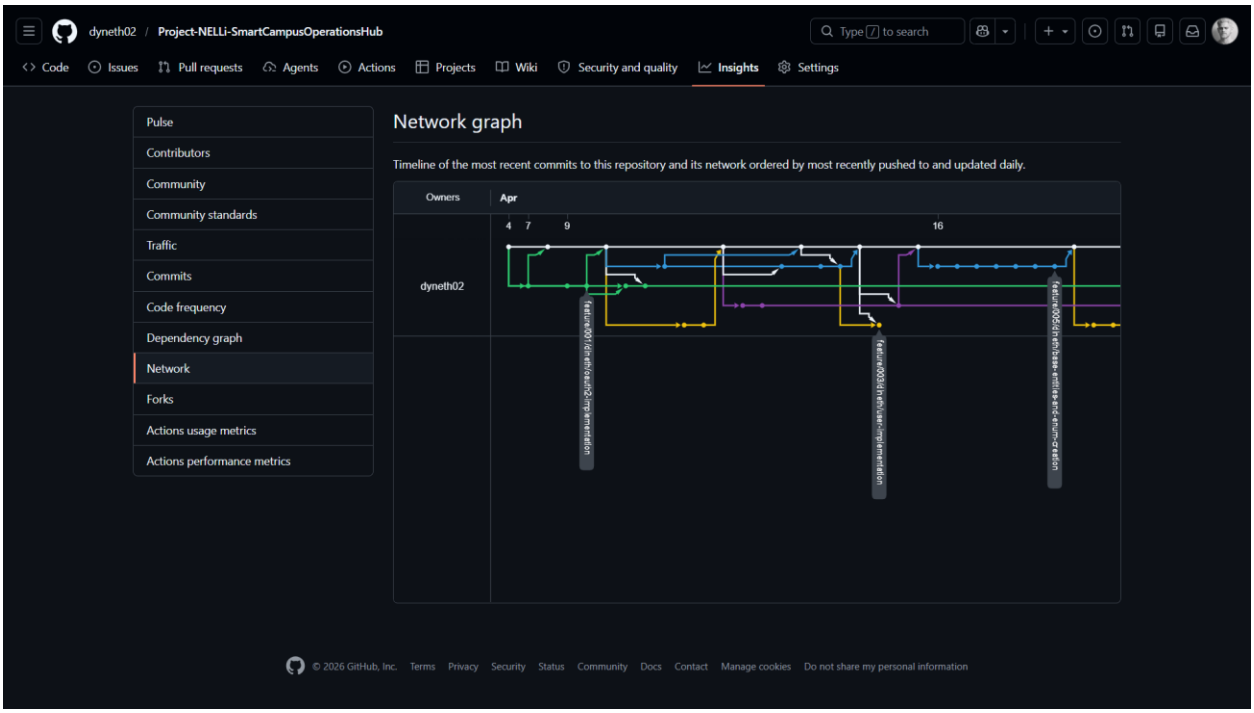
# 2. Implement feature with incremental commits
   git add src/main/java/com/smartcampus/service/BookingService.java
   git commit -m "impl(service): Added createBooking with conflict
   detection"
   git add src/main/java/com/smartcampus/service/BookingService.java
   git commit -m "impl(service): Added approveBooking with QR token
   generation"

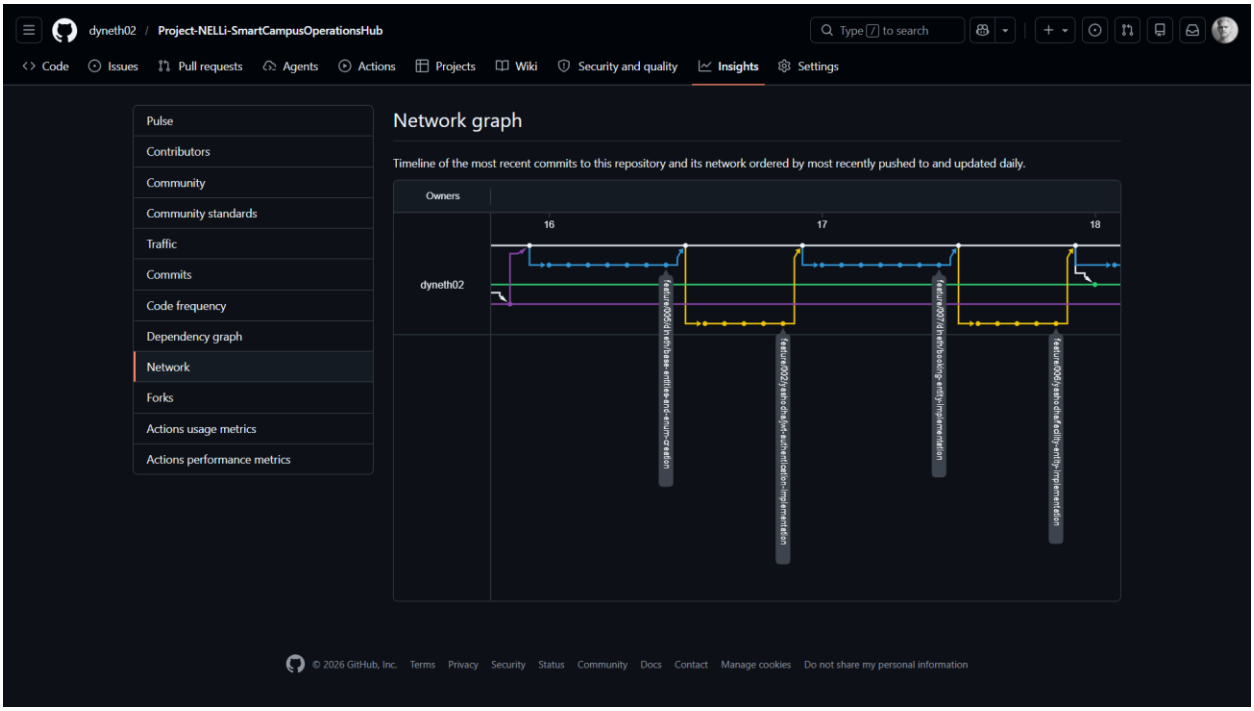
# 3. Push branch to remote
   git push origin feature/011/dineth/booking-service-implementation

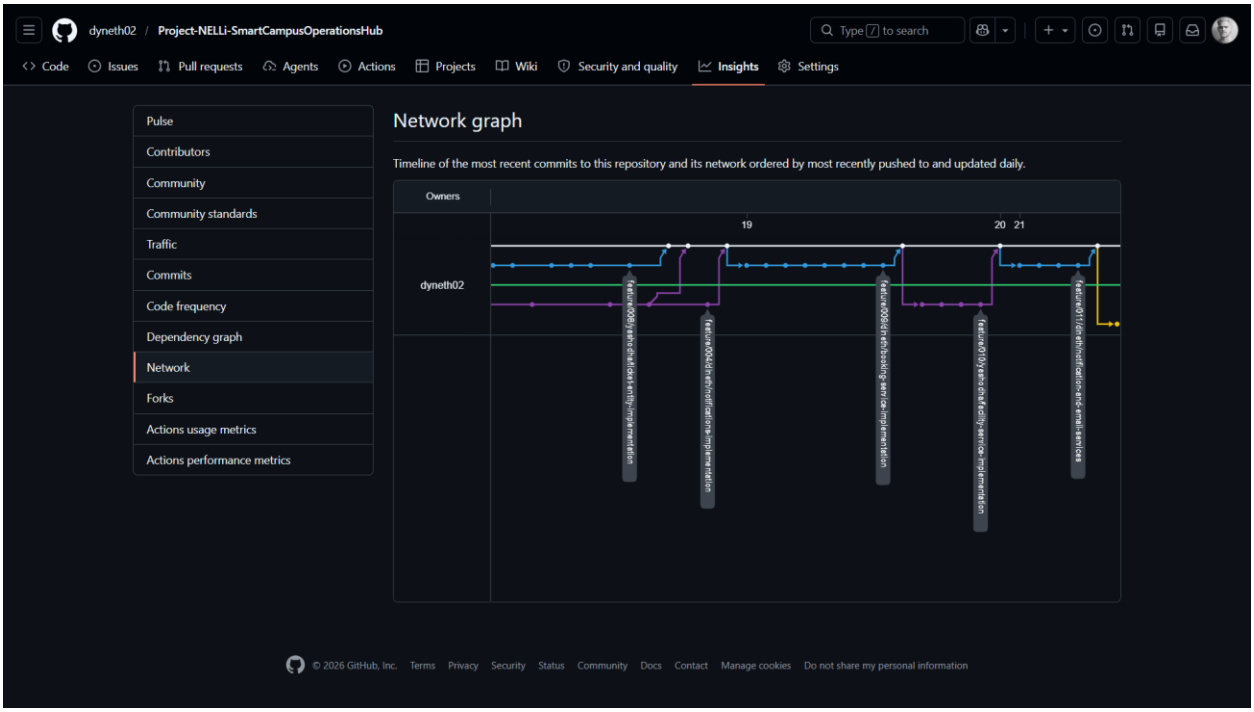
# 4. Create Pull Request to develop on GitHub
```

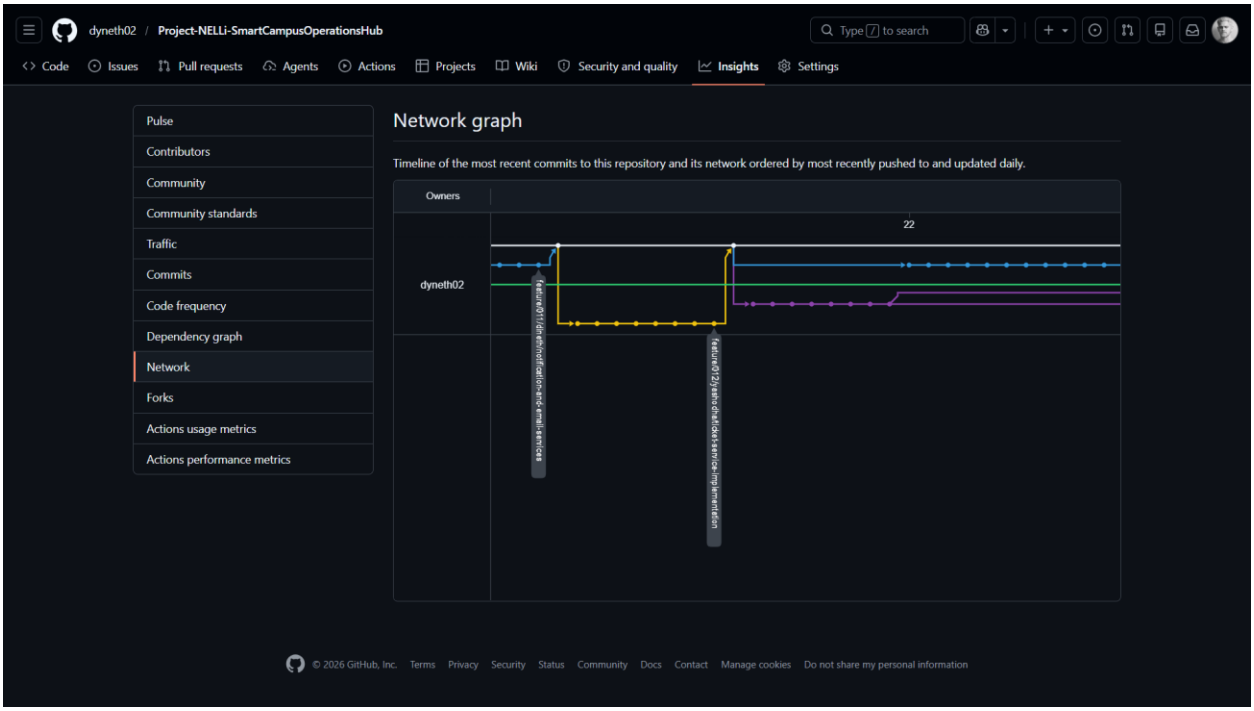
```
# 5. After review, merge to develop
   git checkout develop
   git merge feature/011/dineth/booking-service-implementation
   git push origin develop

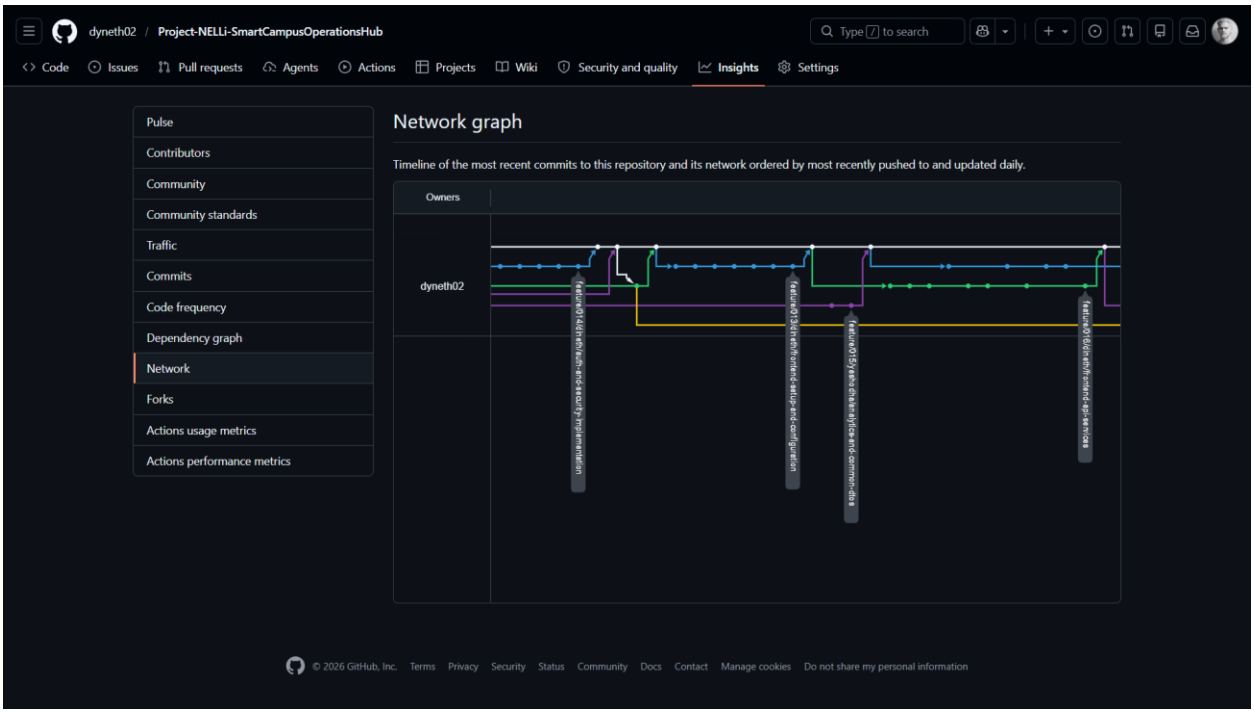
# 6. Release merge: develop → main
   git checkout main
   git merge develop
   git tag -a v1.0.0 -m "Release: Feature-complete NELLi v1.0.0"
   git push origin main --tags
```

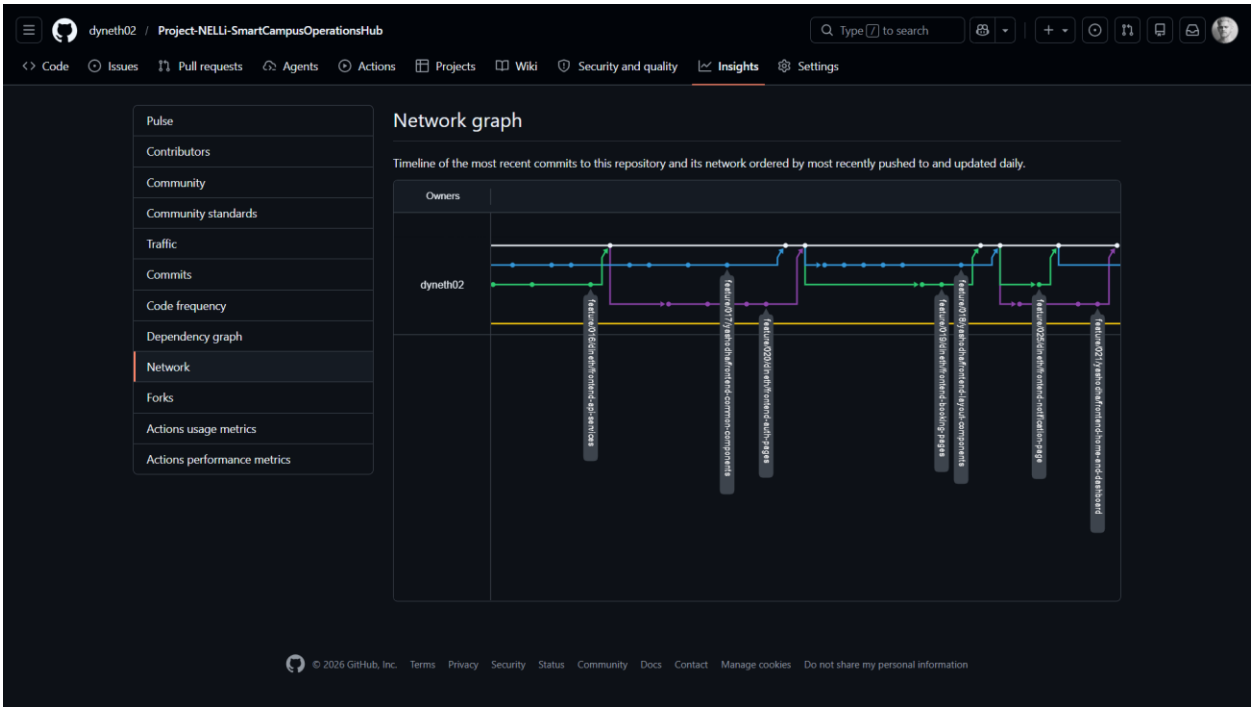


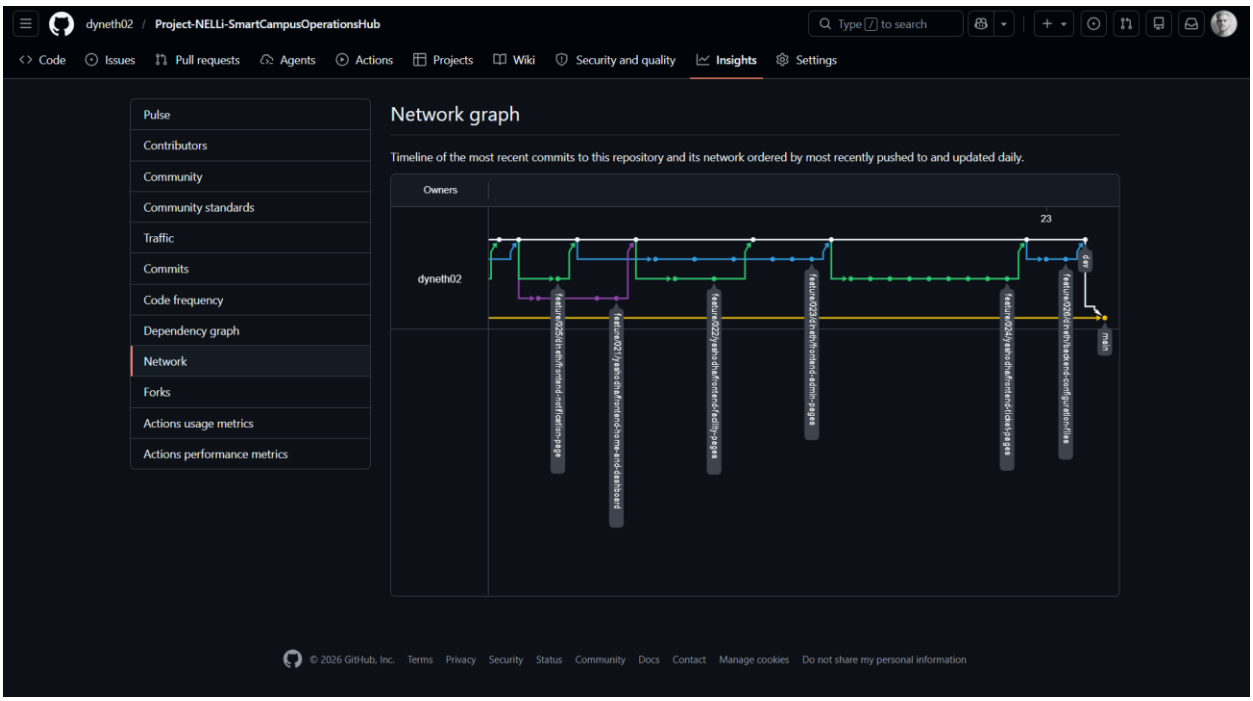

dyneth02 / Project-NELLi-SmartCampusOperationsHub
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Pulse
Contributors
Community
Community standards
Traffic
Commits
Code frequency
Dependency graph
Network
Forks
Actions usage metrics
Actions performance metrics
Network graph
Timeline of the most recent commits to this repository and its network ordered by most recently pushed to and updated daily.
Owners
dyneth02
16
17
18
feature/005-dh-eth/base-entities-and-enum-creation
feature/002/yasiru-chathuranga-authentication-implementation
feature/007-dh-eth/booking-entity-implementation
feature/008/yasiru-chathuranga-facility-entity-implementation
© 2026 GitHub, Inc.
Terms
Privacy
Security
Status
Community
Docs
Contact
Manage cookies
Do not share my personal information


dyneth02 / Project-NELLi-SmartCampusOperationsHub
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Pulse
Contributors
Community
Community standards
Traffic
Commits
Code frequency
Dependency graph
Network
Forks
Actions usage metrics
Actions performance metrics
Network graph
Timeline of the most recent commits to this repository and its network ordered by most recently pushed to and updated daily.
Owners
dyneth02
19
20
21
feature/008/yasiru-chathuranga-facility-entity-implementation
feature/004-dh-eth/notifications-implementation
feature/008/dyneth-booking-service-implementation
feature/010/yasiru-chathuranga-facility-service-implementation
feature/011/dyneth/notification-and-email-services
© 2026 GitHub, Inc.
Terms
Privacy
Security
Status
Community
Docs
Contact
Manage cookies
Do not share my personal information


dyneth02 / Project-NELLi-SmartCampusOperationsHub
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Pulse
Contributors
Community
Community standards
Traffic
Commits
Code frequency
Dependency graph
Network
Forks
Actions usage metrics
Actions performance metrics
Network graph
Timeline of the most recent commits to this repository and its network ordered by most recently pushed to and updated daily.
Owners
dyneth02
22
feature/01/dhenn/notification-and-email-services
feature/02/sanu/shelf/clock-session-implementation
© 2026 GitHub, Inc.
Terms
Privacy
Security
Status
Community
Docs
Contact
Manage cookies
Do not share my personal information


dyneth02 / Project-NELLi-SmartCampusOperationsHub
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Pulse
Contributors
Community
Community standards
Traffic
Commits
Code frequency
Dependency graph
Network
Forks
Actions usage metrics
Actions performance metrics
Network graph
Timeline of the most recent commits to this repository and its network ordered by most recently pushed to and updated daily.
Owners
dyneth02
feature/01/42dyneth/auth-service-security-implementation
feature/03/dyneth/frontend-setup-and-configuration
feature/05/yasmi/database-entity-base-and-common-dtos
feature/06/dyneth/microservices
© 2026 GitHub, Inc.
Terms
Privacy
Security
Status
Community
Docs
Contact
Manage cookies
Do not share my personal information


dyneth02 / Project-NELLi-SmartCampusOperationsHub
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Pulse
Contributors
Community
Community standards
Traffic
Commits
Code frequency
Dependency graph
Network
Forks
Actions usage metrics
Actions performance metrics
Network graph
Timeline of the most recent commits to this repository and its network ordered by most recently pushed to and updated daily.
Owners
dyneth02
feature/016/dyneth/frontend-api-services
feature/017/yaeni-dha/frontend-common-components
feature/020/dyneth/frontend-auth-pages
feature/015/dyneth/frontend-booking-pages
feature/019/yaeni-dha/frontend-layout-components
feature/025/dyneth/frontend-notification-page
feature/021/yaeni-dha/frontend-home-and-dashboard
© 2026 GitHub, Inc.
Terms
Privacy
Security
Status
Community
Docs
Contact
Manage cookies
Do not share my personal information


dyneth02 / Project-NELLi-SmartCampusOperationsHub
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Pulse
Contributors
Community
Community standards
Traffic
Commits
Code frequency
Dependency graph
Network
Forks
Actions usage metrics
Actions performance metrics
Network graph
Timeline of the most recent commits to this repository and its network ordered by most recently pushed to and updated daily.
Owners
dyneth02
23
dev
main
feature/025/dyneth/frontend-notification-page
feature/021/yaeni-dha/frontend-home-and-dashboard
feature/022/yaeni-dha/frontend-facility-pages
feature/023/dyneth/frontend-admin-pages
feature/024/yaeni-dha/frontend-ui-kit-pages
feature/026/dyneth/backend-configuration-files
© 2026 GitHub, Inc.
Terms
Privacy
Security
Status
Community
Docs
Contact
Manage cookies
Do not share my personal information

11.6. Continuous Integration Pipeline

11.6.1. Github Actions Workflow

A GitHub Actions CI workflow (ci-cd.yml) is configured in .github/workflows/ to automatically build and test both the backend and frontend on every push to any branch and on every Pull Request targeting develop or main.

11.6.2. Workflow Configuration

```
name: NELLi CI Pipeline

on:
  push:
    branches: [ '**' ]
  pull_request:
    branches: [ develop, main ]

jobs:
  backend-build-test:
    name: Backend – Build & Test
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_USER: testuser
          POSTGRES_PASSWORD: testpass
          POSTGRES_DB: nelli_test
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
      - name: Checkout code
        uses: actions/checkout@v4
```

```
      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: maven

      - name: Build and test with Maven
        working-directory: ./backend
        env:
          SPRING_DATASOURCE_URL:
jdbc:postgresql://localhost:5432/nelli_test
          SPRING_DATASOURCE_USERNAME: testuser
          SPRING_DATASOURCE_PASSWORD: testpass
          JWT_SECRET: $[[ secrets.JWT_SECRET_TEST ]]
          GOOGLE_CLIENT_ID: $[[ secrets.GOOGLE_CLIENT_ID ]]
          GOOGLE_CLIENT_SECRET: $[[ secrets.GOOGLE_CLIENT_SECRET ]]
        run: mvn clean verify --no-transfer-progress

      - name: Publish test results
        uses: EnricoMi/publish-unit-test-result-action@v2
        if: always()
        with:
          files: backend/target/surefire-reports/*.xml

  frontend-build:
    name: Frontend – Build & Lint
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Node.js 20
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
          cache-dependency-path: frontend/package-lock.json

      - name: Install dependencies
        working-directory: ./frontend
        run: npm ci
```

```
- name: Run ESLint
  working-directory: ./frontend
  run: npm run lint

- name: Build production bundle
  working-directory: ./frontend
  env:
    VITE_API_BASE_URL: http://localhost:8080/api/v1
  run: npm run build

- name: Upload build artifacts
  uses: actions/upload-artifact@v4
  with:
    name: frontend-dist
    path: frontend/dist/
```

11.3. CI Pipeline Stages

Stage	Description
Trigger	On every push (all branches) and Pull Request to develop/main
Backend: Checkout	Clone repository to ubuntu-latest runner
Backend: JDK Setup	Java 21 (Temurin) with Maven dependency cache
Backend: PostgreSQL Service	Ephemeral PostgreSQL 15 container for integration tests
Backend: mvn clean verify	Compile, run unit tests (Surefire), run integration tests (Failsafe), package JAR
Backend: Test Reports	Publishes JUnit XML reports as PR check annotations
Frontend: Node Setup	Node.js 22.19.0 with npm cache
Frontend: npm ci	Clean install from package-lock.json (deterministic)

Stage	Description
Frontend: ESLint	Static analysis - fails build on lint errors
Frontend: npm run build	Vite production build - fails on TypeScript/JSX errors
Frontend: Artifact Upload	dist/ folder uploaded as workflow artifact for review

11.4. Secrets Management

- Sensitive values (JWT_SECRET, GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET) are stored as GitHub Repository Secrets, never committed to the codebase.

- Test-specific variants of secrets use _TEST suffix (JWT_SECRET_TEST) to prevent accidental production credential use in CI.

- The Neon production database is NOT used in CI; an ephemeral PostgreSQL service container is spun up for each CI run and torn down after.

- application.properties uses ${ENV_VAR} placeholders; actual values are injected via environment variables in CI and via .env file locally.

12. Individual Contribution

NELLi was developed by a two-member team (HIRUSHA D G A D and COORAY Y H) delivering the full scope of a four-member group assignment. This section provides a detailed, transparent account of each member's technical contributions, mapped to specific GitHub feature branches, files, and module responsibilities.

12.1. HIRUSHA D G A D - Contribution Summary

Dineth took primary ownership of the application's security architecture, authentication flows, booking business logic (including the conflict detection engine), notification and email services, and several key frontend modules.

12.1.1. Backend Contributions

Branch	Feature	Key Files Created	LOC Est.
feature/001	OAuth2 Implementation	SecurityConfig.java, OAuth2SuccessHandler.java, CorsConfig.java	~320
feature/003	User Implementation	User.java, UserRepository.java, UserService.java, UserController.java	~280
feature/004	Notification Entity (initial)	Notification.java, NotificationRepository.java	~120
feature/005	Base Entities & Enums	All enum classes (FacilityType, BookingStatus, TicketStatus, etc.), BaseEntity.java	~240
feature/007	Booking Entity	Booking.java, BookingRepository.java, BookingRequest.java, BookingResponse.java	~350
feature/009	Notification Entity (refined)	Notification.java (updated), NotificationPreferences.java	~160
feature/011	Booking Service	BookingService.java (conflict detection, approval, QR trigger), BookingController.java, AvailabilityWindowDTO.java	~520

Branch	Feature	Key Files Created	LOC Est.
feature/013	Notification & Email Services	NotificationService.java, EmailService.java, email HTML templates	~480
feature/014	Auth & Security	AuthController.java, AuthService.java, CustomUserDetailsService.java	~460
feature/026	QR Validation	QrCodeService.java, QrVerificationController.java	~220
feature/028	Backend Config Files	pom.xml, application.properties, application-dev.properties, .gitignore, backend README	~200

12.1.2. Frontend Contributions

Branch	Feature	Key Files Created	LOC Est.
feature/017	Common UI Components	Button.jsx, Input.jsx, TextArea.jsx, Card.jsx, Badge.jsx, LoadingSpinner.jsx, Modal.jsx, Dropdown.jsx, Select.jsx, useAnimations.js	~680
feature/019	API Service Modules	authApi.js, facilityApi.js, bookingApi.js, ticketApi.js, notificationApi.js, analyticsApi.js	~380
feature/020	Auth Pages	Login.jsx, Register.jsx, OAuthCallback.jsx, NotFound.jsx	~520
feature/021	Home & Dashboard	Home.jsx, Dashboard.jsx, Profile.jsx	~420
feature/023	Booking Pages	MyBookings.jsx, CreateBooking.jsx	~440
feature/025	Notification Page	Notifications.jsx (full page with preferences)	~280
Feature/026	QR Code Validation Page	BookingVerify.jsx	~220

12.1.3. Key Technical Achievements

- Implemented the booking conflict detection engine using JPQL overlap query and custom BookingConflictException, correctly handling PENDING and APPROVED bookings as blocking states while ignoring REJECTED and CANCELLED records.

- ❖ Built the full OAuth2 → JWT pipeline: Google OAuth2 callback → OAuth2SuccessHandler account linking → JWT issuance → frontend token storage → protected API access.

- ❖ Developed the Email Service with HTML email templates for all six notification types, ensuring email failures do not propagate as HTTP errors through the service chain.

- ❖ Created the common UI component library (Button, Modal, Card, Input, etc.) with Tailwind CSS styling, loading states, and GSAP animation integration, establishing the design system used throughout the entire frontend.

12.2. COORAY Y H - Contribution Summary

Yashodha was the primary contributor to the frontend architecture, having initiated the React/Vite project and established the project-wide design system. On the backend, Yashodha owned the Facility and Ticket domains, the Analytics module with supporting HIRUSHA D G A D. Yashodha also implemented the largest volume of frontend pages.

12.2.1. Backend Contributions

Branch	Feature	Key Files Created	LOC Est.
feature/002	JWT Authentication	JwtTokenProvider.java, JwtConfig.java, JwtAuthenticationFilter.java (initial), TokenRefreshService.java	~340
feature/006	Facility Entity	Facility.java, FacilityRepository.java, FacilityRequest.java, FacilityResponse.java	~380
feature/008	Ticket Entity	Ticket.java, Comment.java, Attachment.java, TicketRepository.java, CommentRepository.java, AttachmentRepository.java, all Ticket DTOs	~560
feature/010	Facility Service	FacilityService.java, FacilityController.java (all 8 endpoints)	~480
feature/012	Ticket Service	TicketService.java, CommentService.java, AttachmentService.java, TicketController.java (all 12 endpoints)	~640

Branch	Feature	Key Files Created	LOC Est.
feature/015	Analytics	AnalyticsService.java, AnalyticsController.java, all Analytics DTOs	~520

12.2.2. Frontend Contributions

Branch	Feature	Key Files Created	LOC Est.
feature/016	Frontend Setup	package.json, vite.config.js, tailwind.config.js, index.html, App.jsx, main.jsx, index.css, ThemeContext.jsx, AuthContext.jsx, axiosConfig.js, animations.js	~480
feature/018	Layout Components	Navbar.jsx, Sidebar.jsx, Footer.jsx, MainLayout.jsx, AdminLayout.jsx, AppRoutes.jsx	~560
feature/022	Facility Pages	Facilities.jsx, FacilityDetails.jsx	~380
feature/024	Ticket Pages	MyTickets.jsx, TicketDetails.jsx, CreateTicket.jsx	~560
feature/026	Admin Pages	AdminDashboard.jsx, AdminFacilities.jsx, AdminBookings.jsx, AdminTickets.jsx, AdminUsers.jsx, AdminAnalytics.jsx	~840

12.2.3. Key Technical Achievements

- ❖ Designed and implemented the complete Ticket domain including three related entities (Ticket, Comment, Attachment) with careful cascade rules and referential integrity, supporting all business workflows (lifecycle transitions, ownership enforcement, SLA tracking fields).

- ❖ Built the full analytics backend with aggregation JPQL queries for facility utilization, peak booking hours, ticket KPIs, and user growth all results cached with Spring Cache to reduce database load.

- Established the entire React/Vite frontend project from scratch including Tailwind CSS design system with custom dark/light theme tokens, GSAP animation utilities, and Axios instance with automatic JWT refresh interceptors.

- Implemented the Admin section (6 admin pages) covering the complete administrative workflow from facility management through booking approvals, ticket assignment, user role management, and the analytics dashboard with Recharts visualizations.

13. Conclusion

The NELLi Smart Campus Operations Hub successfully delivers a production-quality, full-stack web system that fulfils and significantly exceeds the minimum requirements of the IT3030 PAF Assignment. All five core modules are fully implemented Facilities Catalogue, Booking Management, Incident Ticketing, Notifications & Email, and Authentication & Authorization alongside both optional innovation features: the QR Code Check-In system and the Admin Analytics Dashboard.

Technically, the system demonstrates mastery of the required stack (Spring Boot REST API + React) and applies industry best practices throughout: layered backend architecture with clean separation of concerns, RESTful API design with HATEOAS links, JWT + OAuth 2.0 security, server-side conflict detection, role-based access control at both the API and frontend routing layers, structured Git branching with 27 feature branches, and automated CI via GitHub Actions.

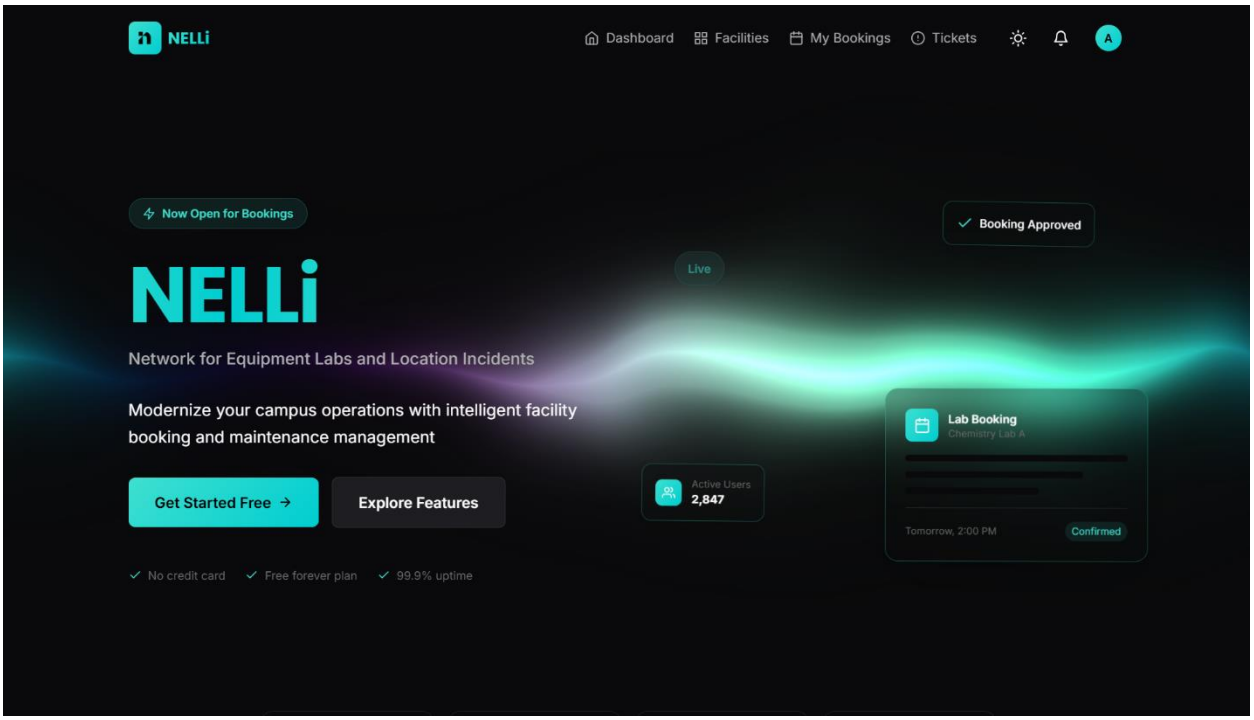
The two-member team delivered the full scope planned for a four-member group, with clearly delineated and individually traceable contributions evident in the Git commit history. Dineth owned the security, authentication, booking logic, and notification pipelines, while Yashodha owned the facility and ticket domains, analytics, and the majority of the frontend page implementations. Both members contributed to shared architecture decisions, the CI pipeline, integration testing, and documentation.

The 27-branch repository structure, standardized commit message conventions, and CI pipeline ensure full auditability of individual contributions and demonstrate professional version control practices. The comprehensive Postman test collection with 63 requests and 209 assertions validates both the happy-path workflows and critical edge cases across all seven modules.

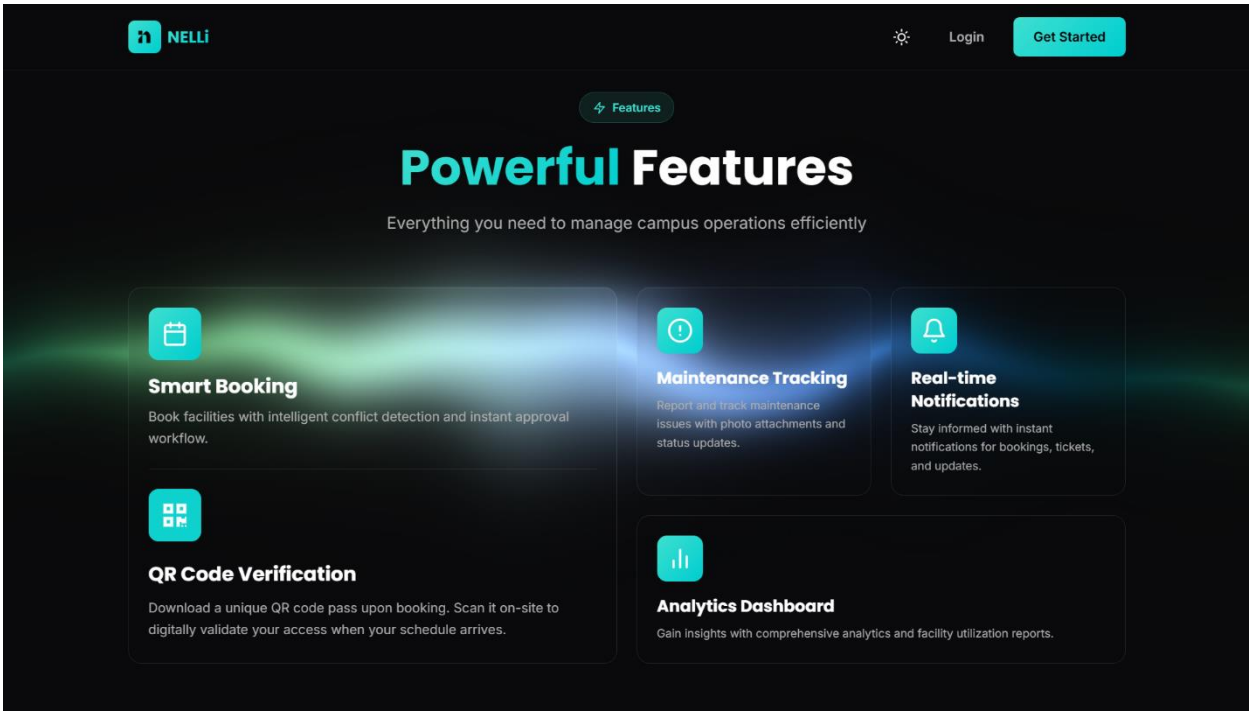
NELLi demonstrates that with disciplined project planning, clear technical architecture, and effective collaboration, a two-member team can deliver a feature-complete, production-inspired campus management system within a four-week sprint.

14. Appendix

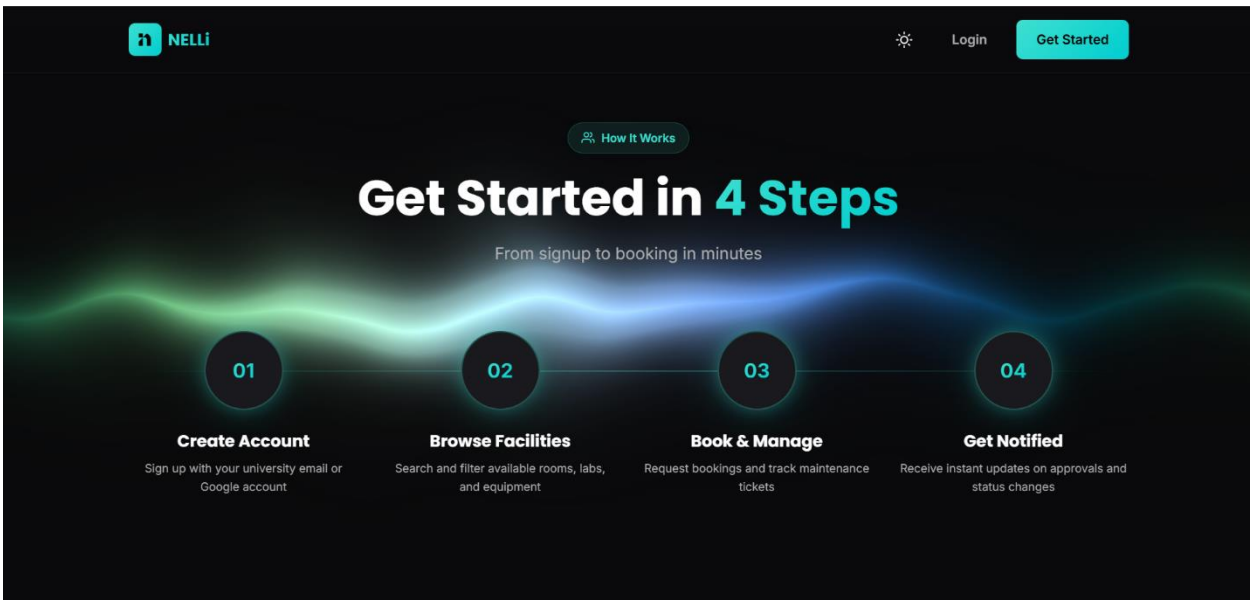
14.1. The Landing Page



[Hero Section]

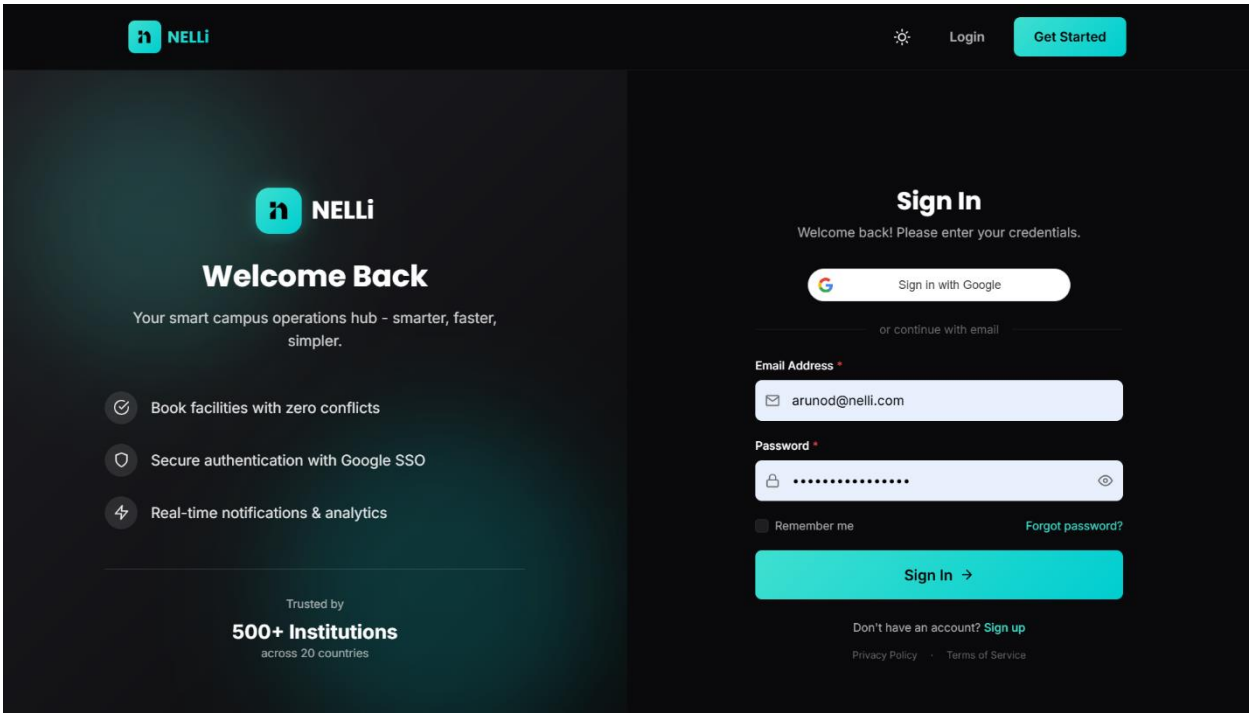


[Features Section]

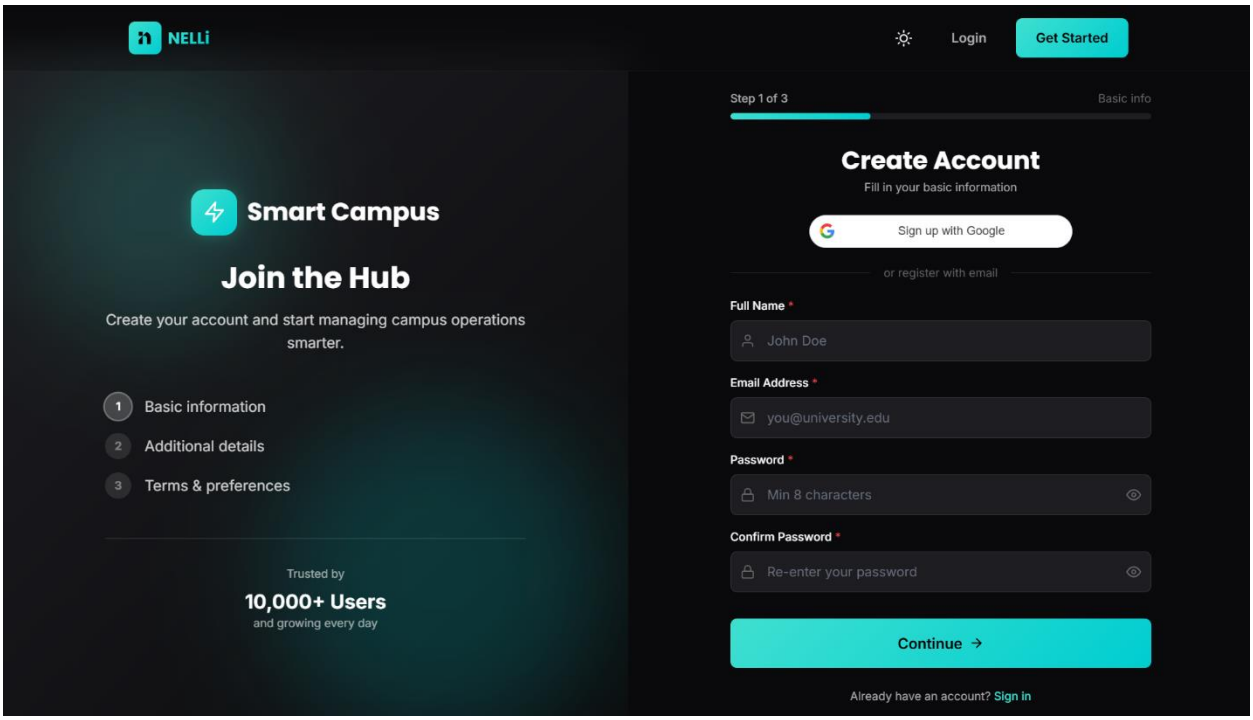


[Site-flow Showcase]

14.2. Login & Register Pages

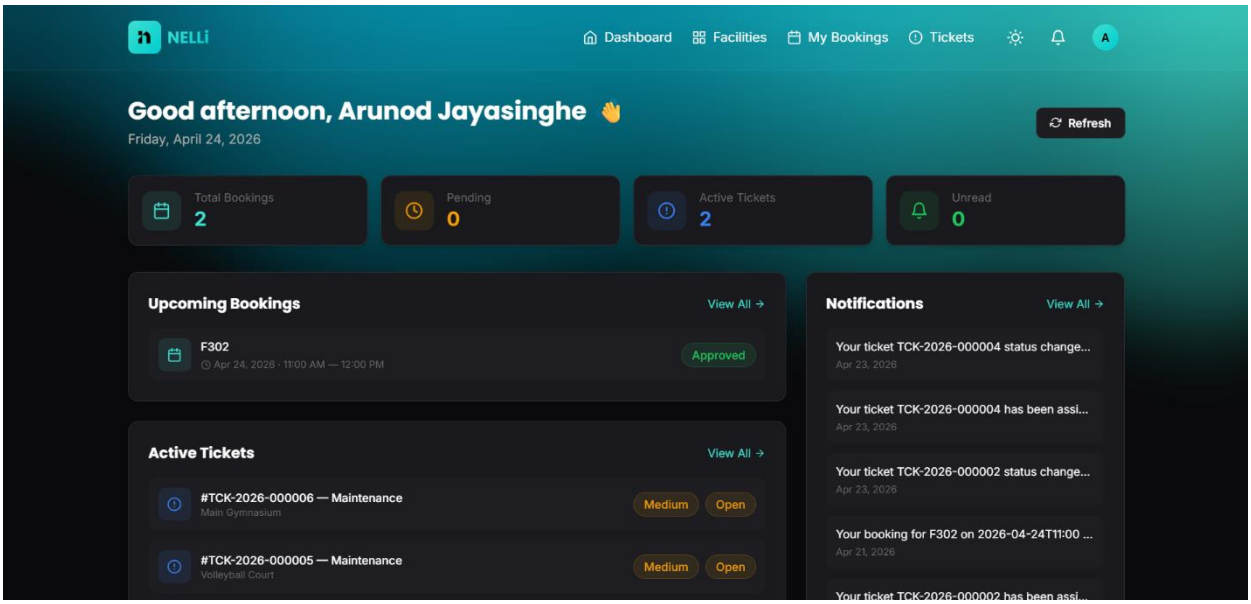


[Login Page]

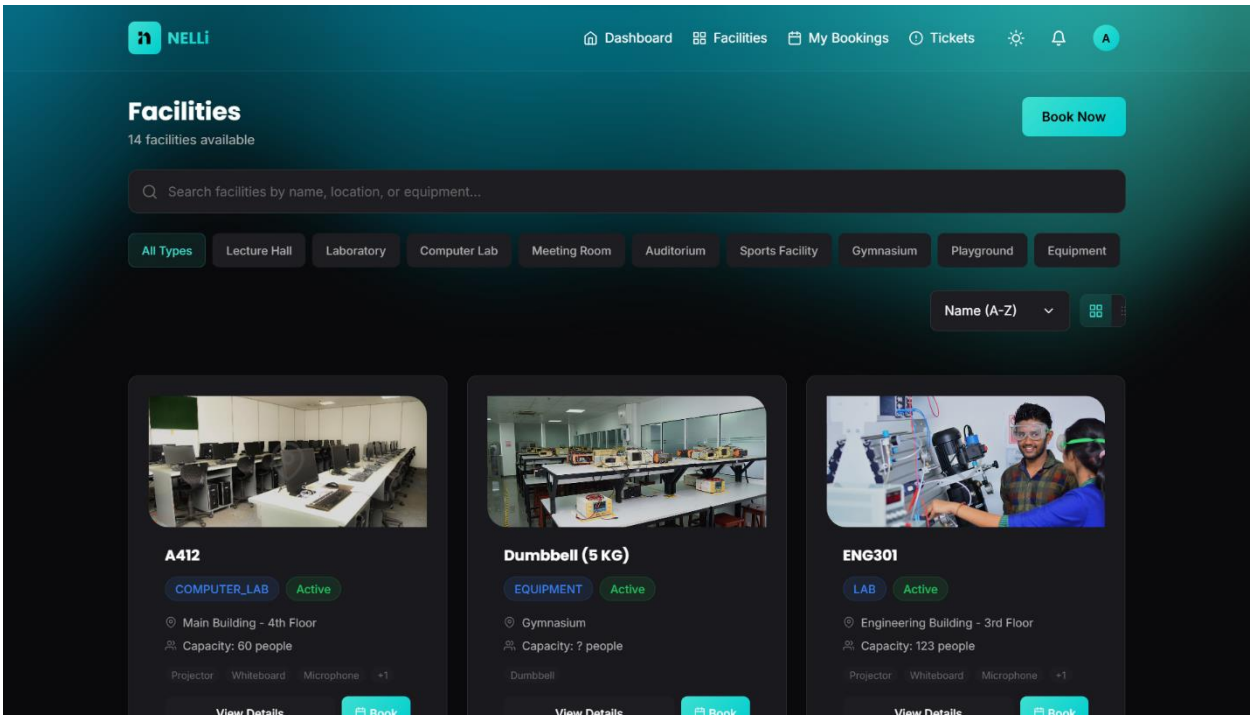


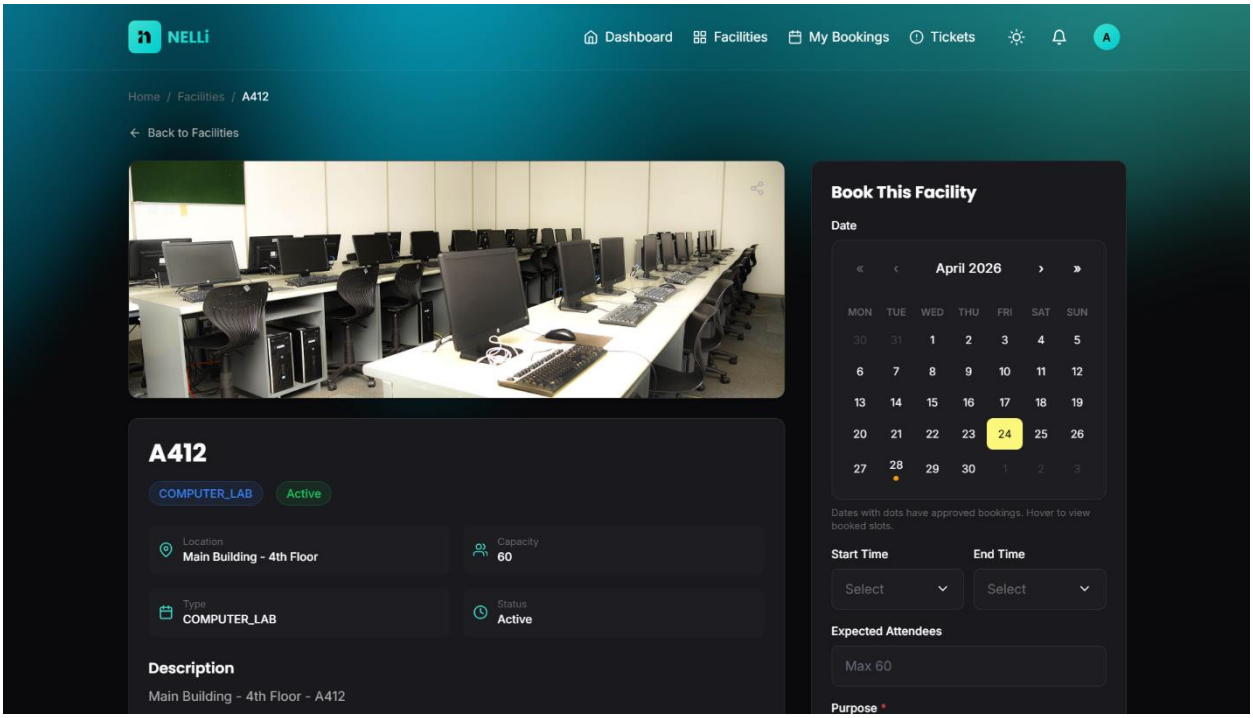
[Register Page]

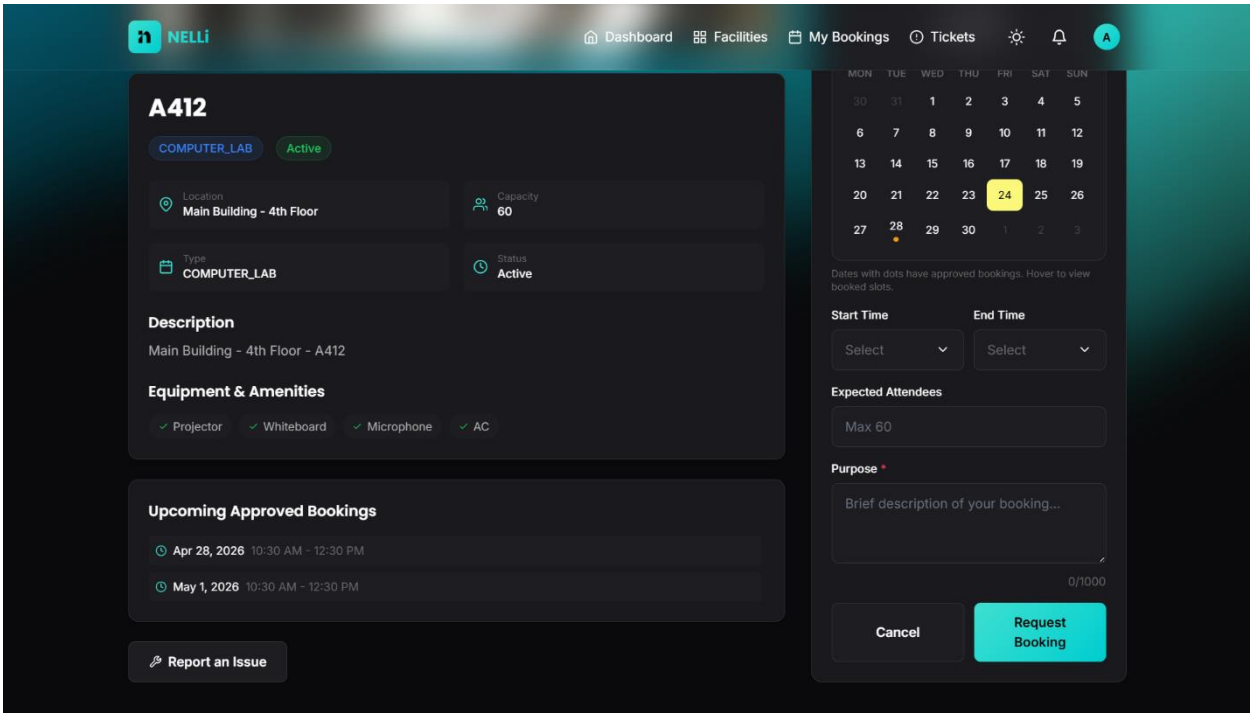
14.3. User Dashboard



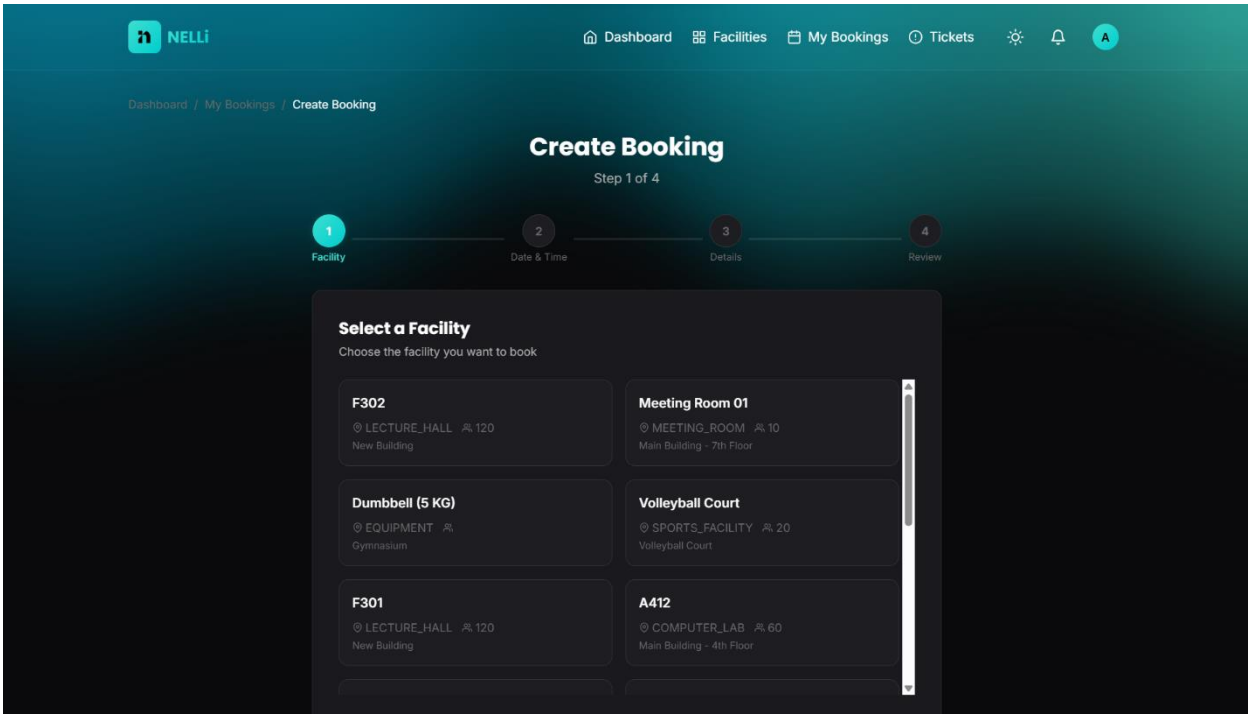
14.4. Facilities Component

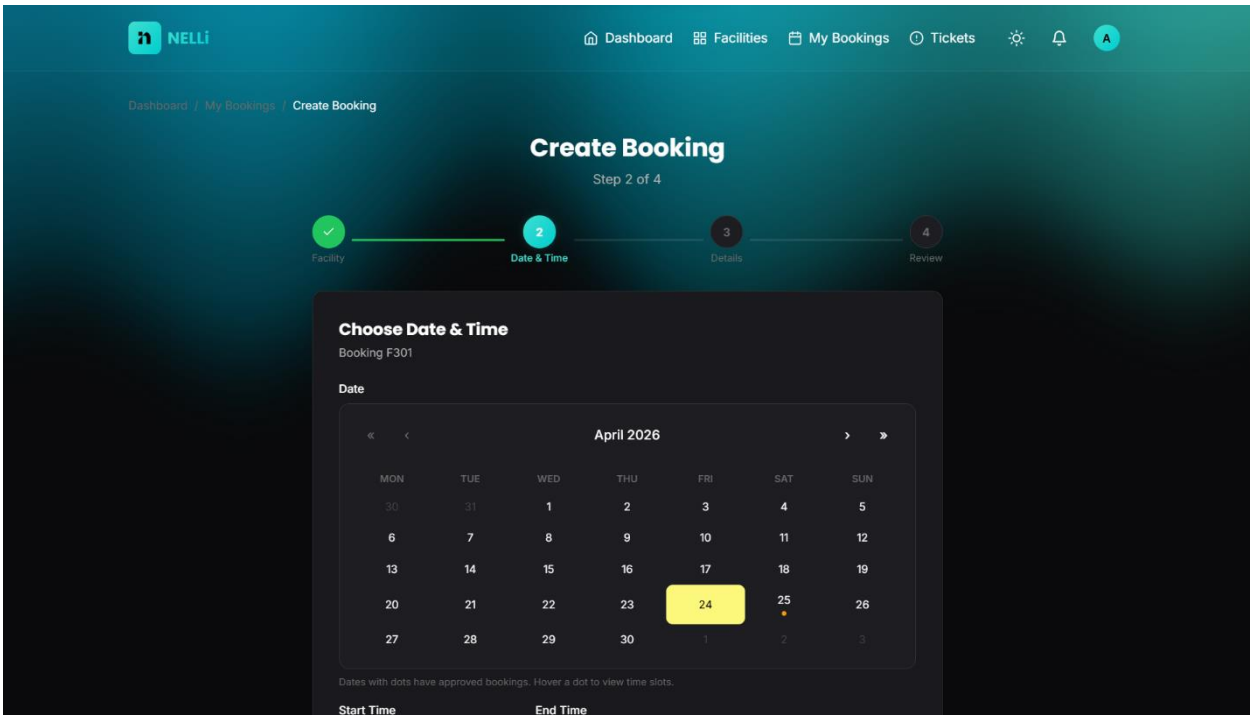


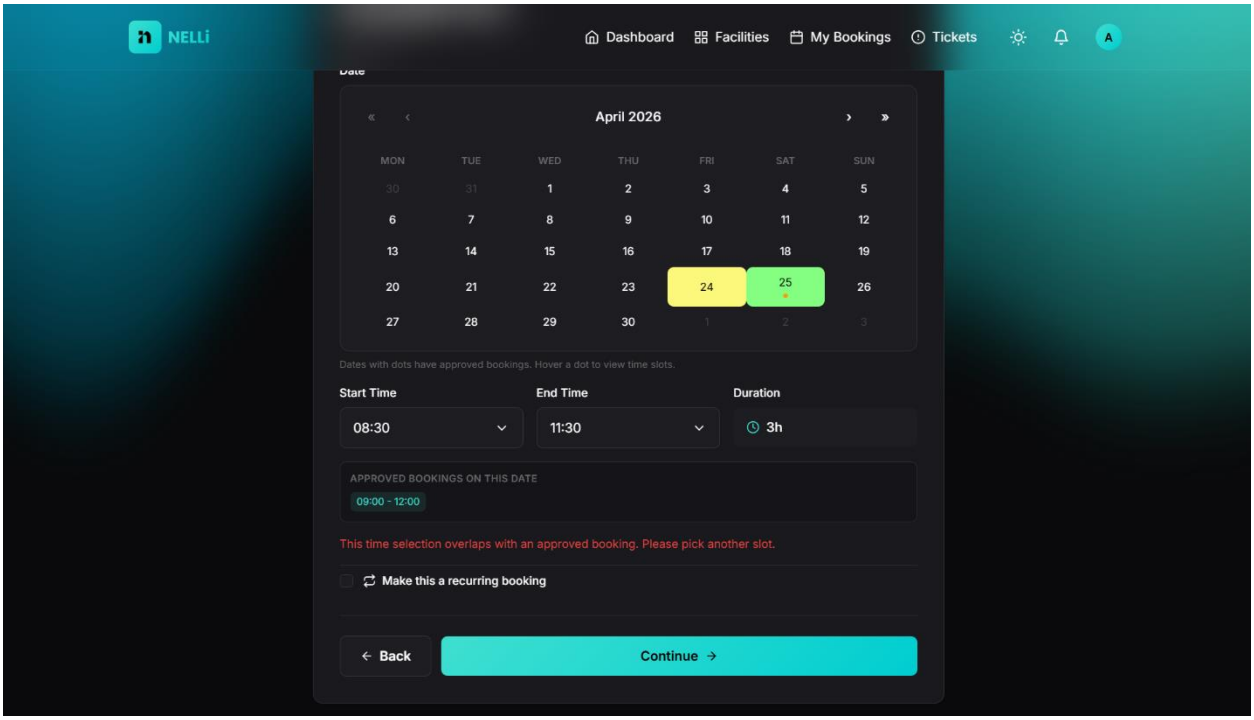

NELLi
Dashboard
Facilities
My Bookings
Tickets
A
Home / Facilities / A412
Back to Facilities
A412
COMPUTER_LAB
Active
Location
Main Building - 4th Floor
Capacity
60
Type
COMPUTER_LAB
Status
Active
Description
Main Building - 4th Floor - A412
Book This Facility
Date
April 2026
MON TUE WED THU FRI SAT SUN
Dates with dots have approved bookings. Hover to view booked slots.
Start Time
End Time
Select
Select
Expected Attendees
Max 60
Purpose *


NELLi
Dashboard
Facilities
My Bookings
Tickets
A
A412
COMPUTER_LAB
Active
Location
Main Building - 4th Floor
Capacity
60
Type
COMPUTER_LAB
Status
Active
Description
Main Building - 4th Floor - A412
Equipment & Amenities
Projector
Whiteboard
Microphone
AC
Upcoming Approved Bookings
Apr 28, 2026 10:30 AM - 12:30 PM
May 1, 2026 10:30 AM - 12:30 PM
Report an Issue
MON TUE WED THU FRI SAT SUN
Dates with dots have approved bookings. Hover to view booked slots.
Start Time
End Time
Select
Select
Expected Attendees
Max 60
Purpose *
Brief description of your booking...
0/1000
Cancel
Request Booking

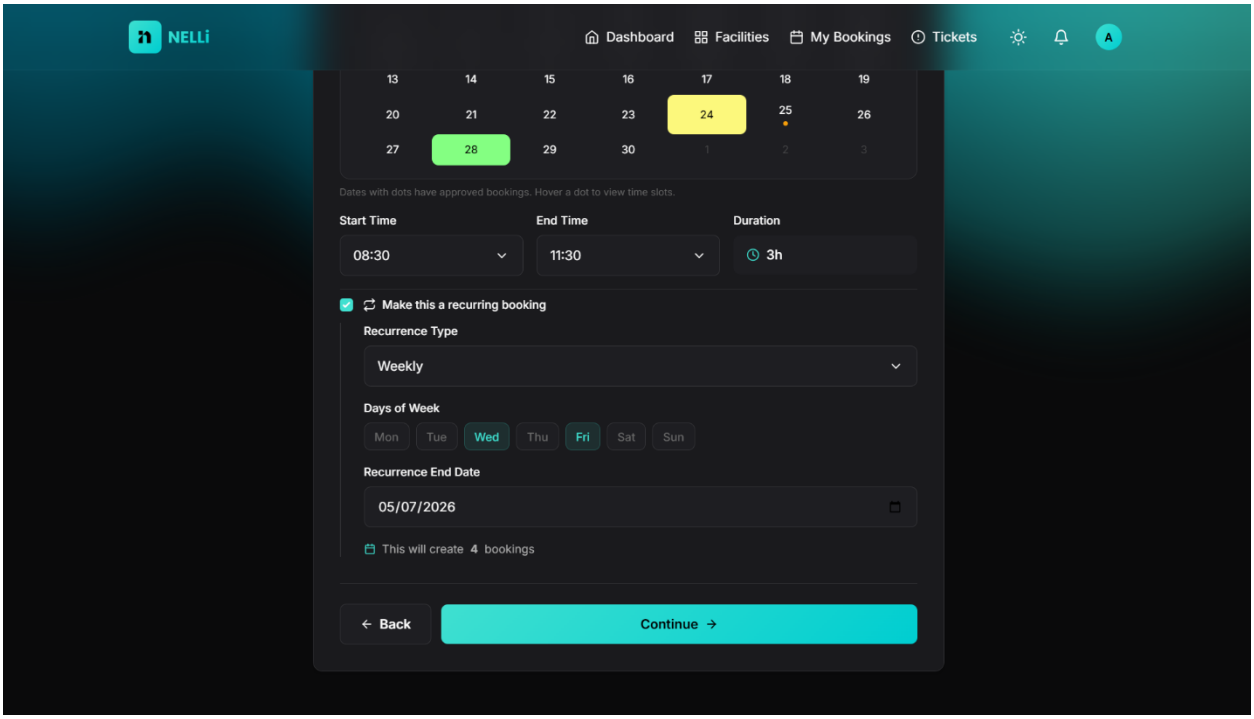
14.5. Booking Component



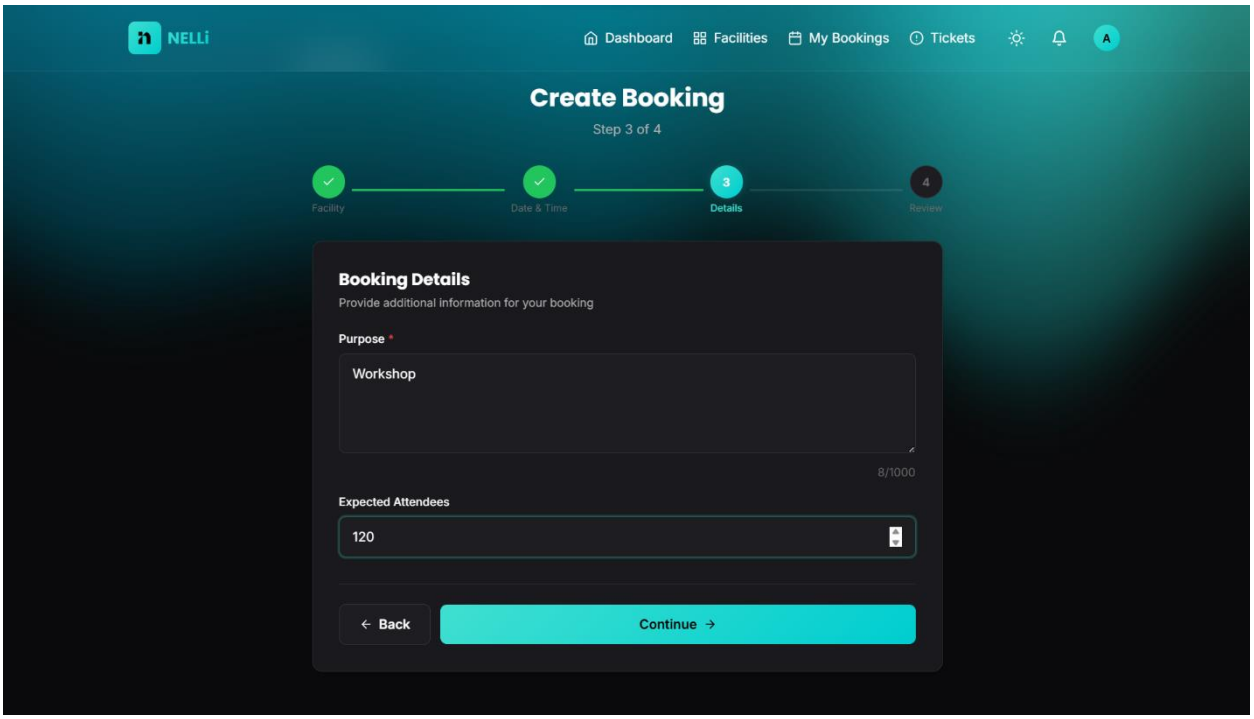


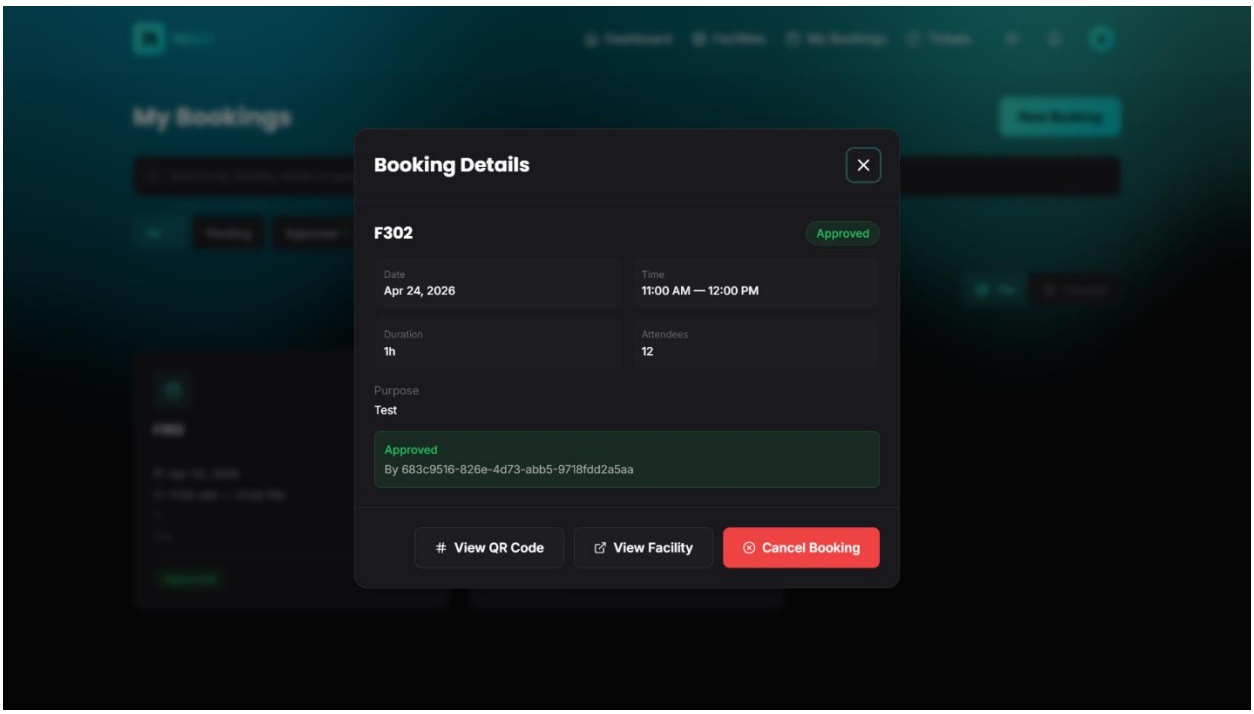


[Resolved Booking Conflicts]

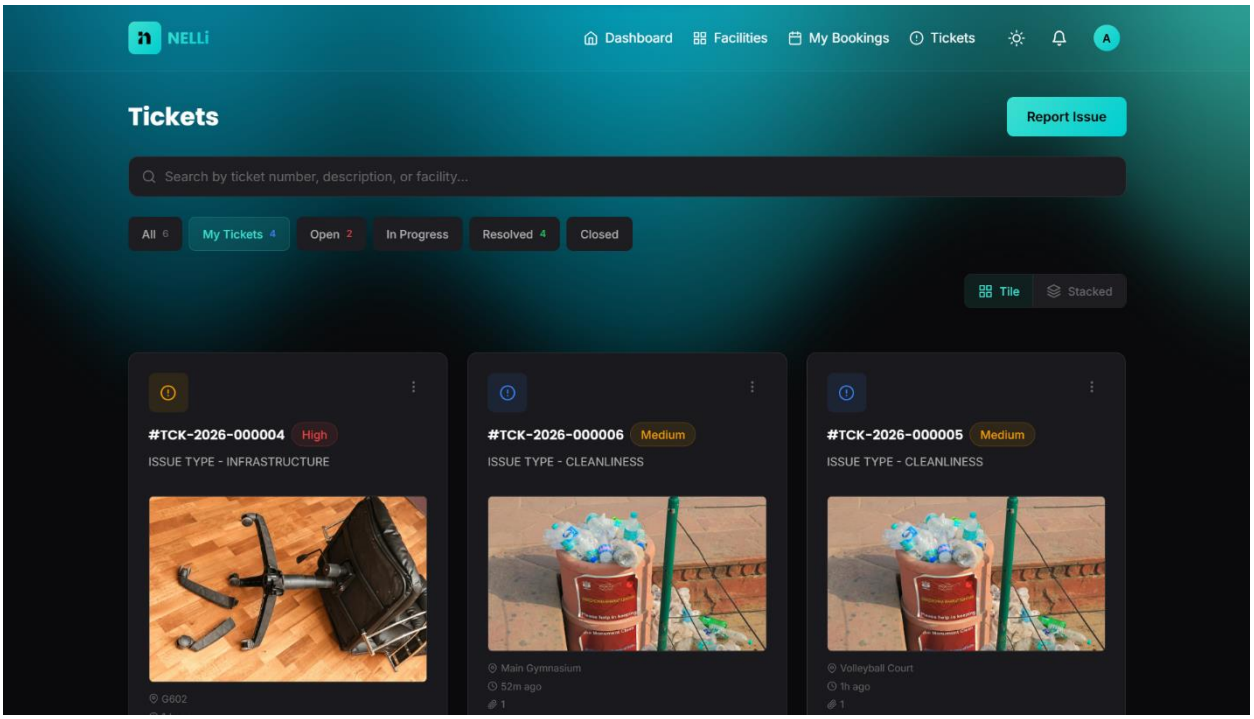


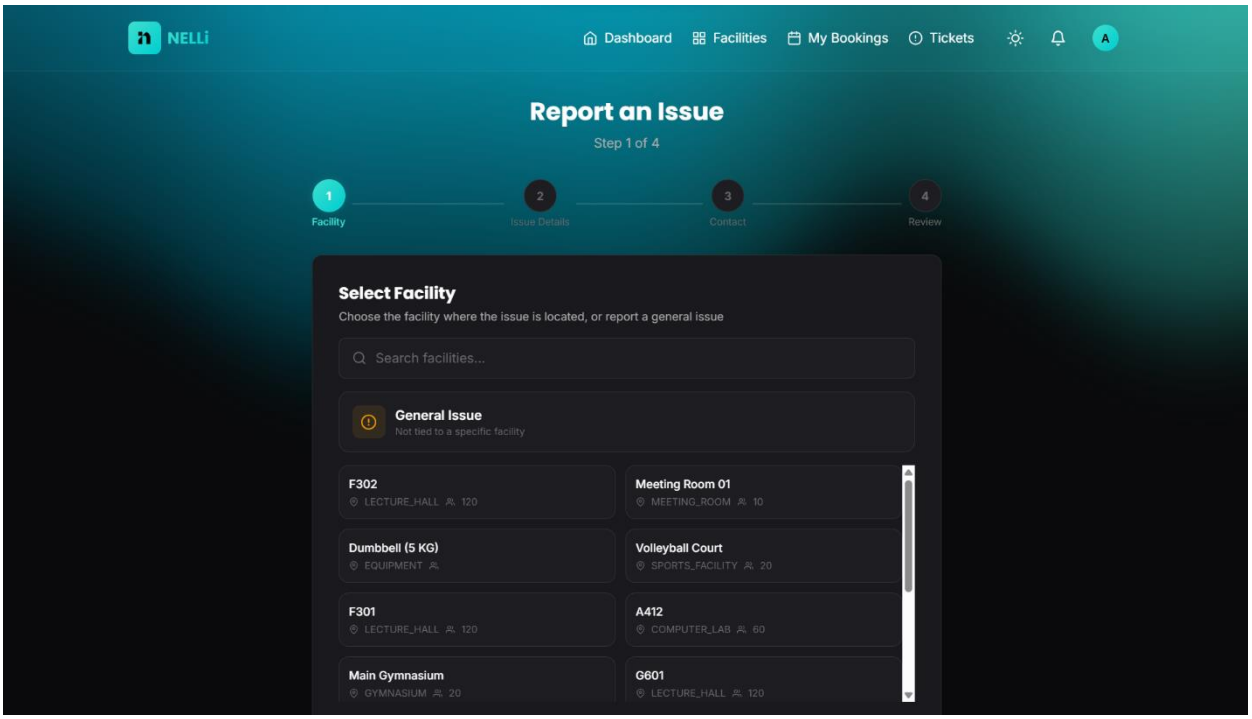
[Recurring Bookings]

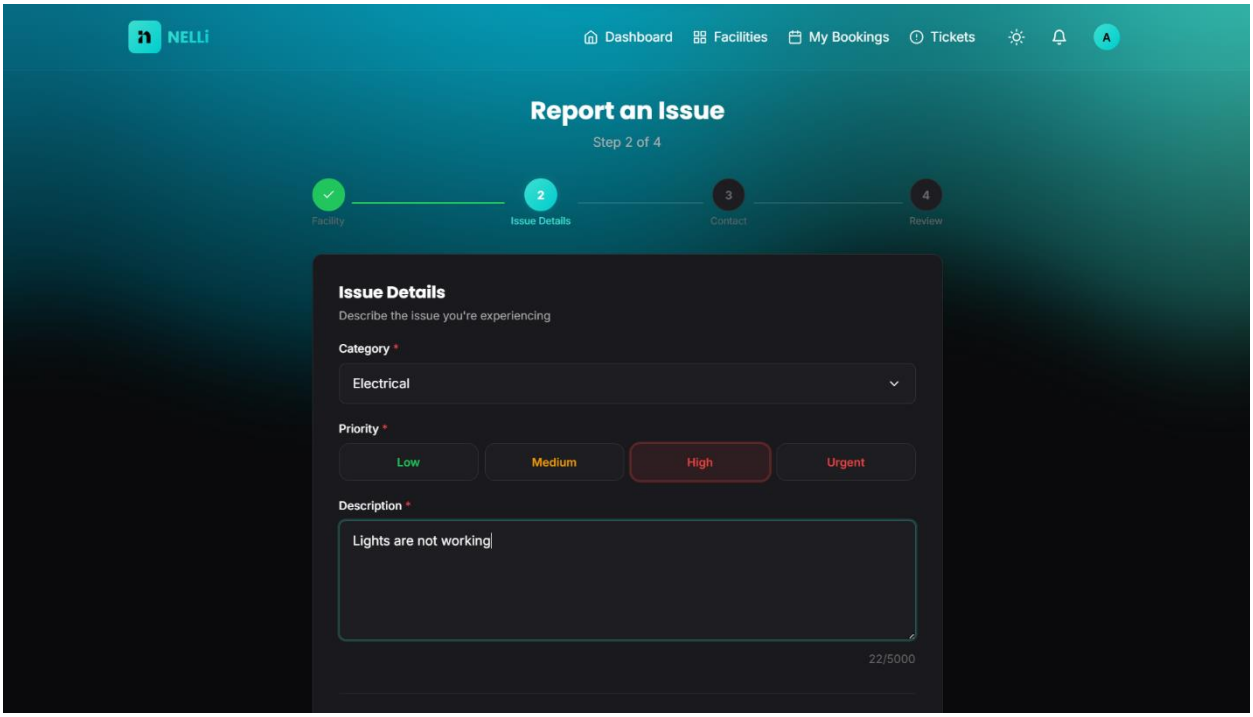

NELLi
Dashboard
Facilities
My Bookings
Tickets
A
Create Booking
Step 3 of 4
Facility
Date & Time
3
Details
4
Review
Booking Details
Provide additional information for your booking
Purpose *
Workshop
8/1000
Expected Attendees
120
← Back
Continue →

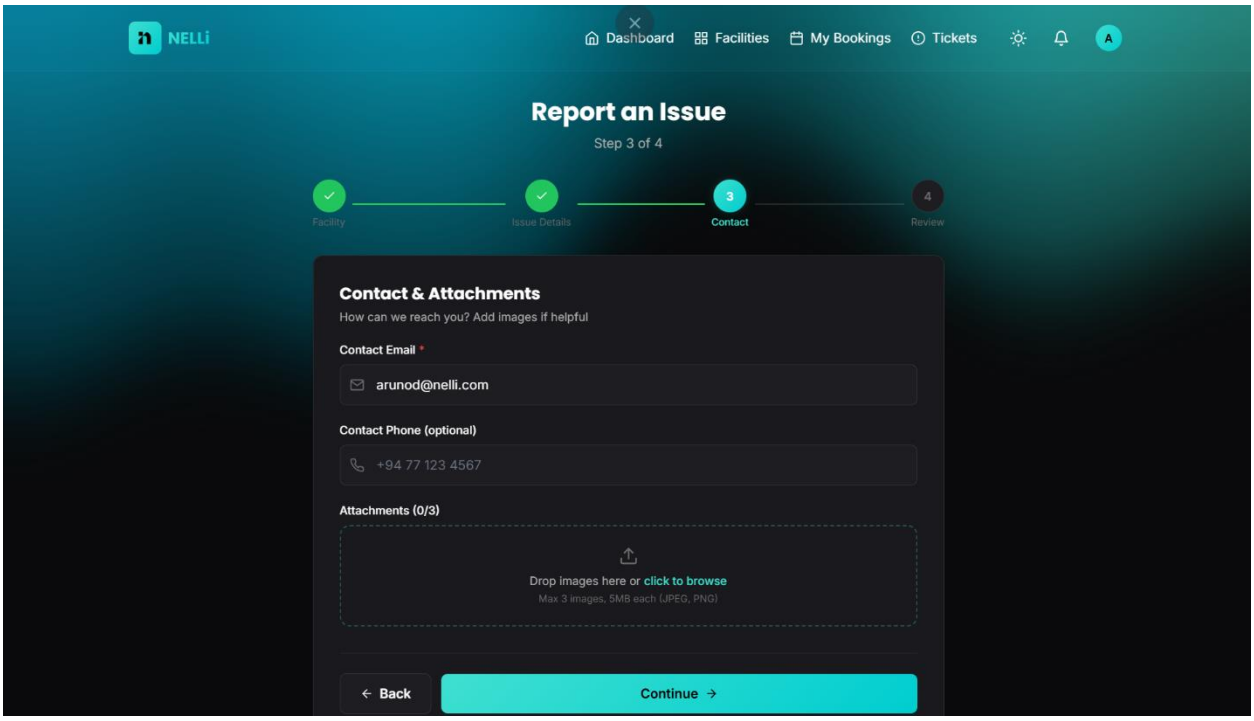

Booking Details
F302
Approved
Date
Apr 24, 2026
Time
11:00 AM — 12:00 PM
Duration
1h
Attendees
12
Purpose
Test
Approved
By 683c9516-826e-4d73-abb5-9718fdd2a5aa
View QR Code
View Facility
Cancel Booking

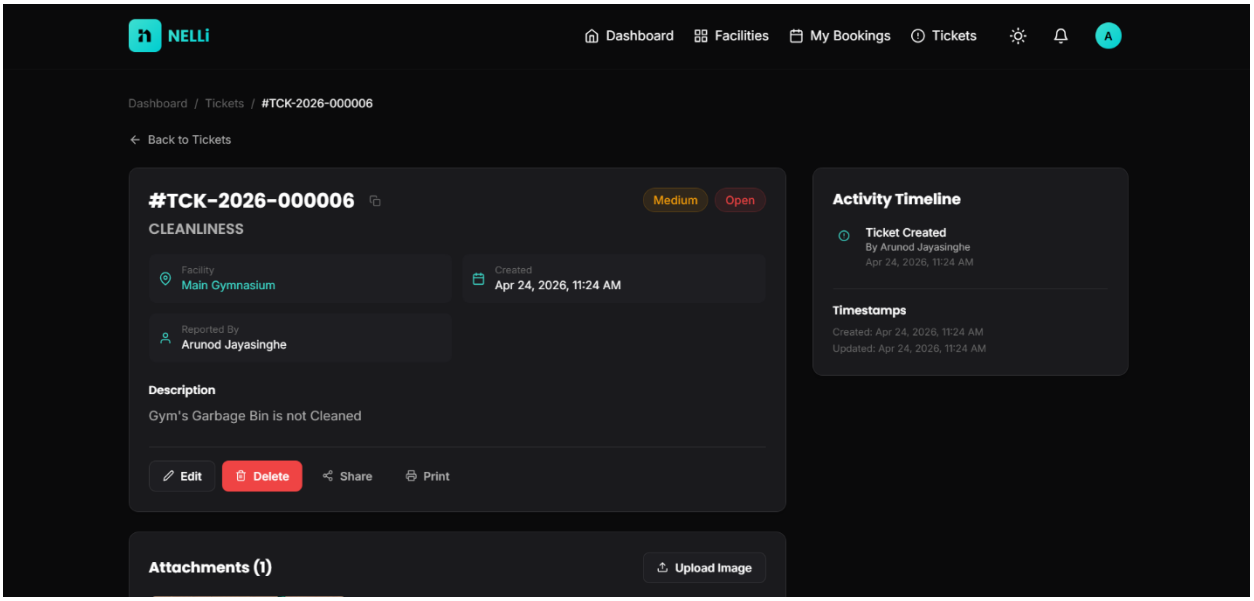
14.6. Ticketing Component

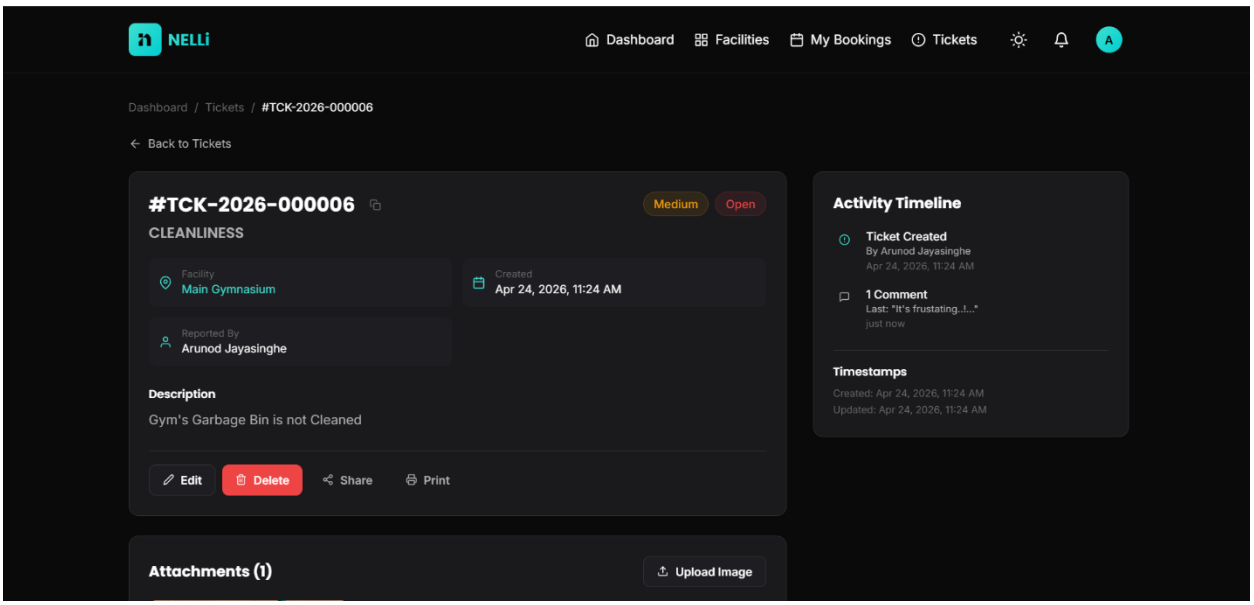




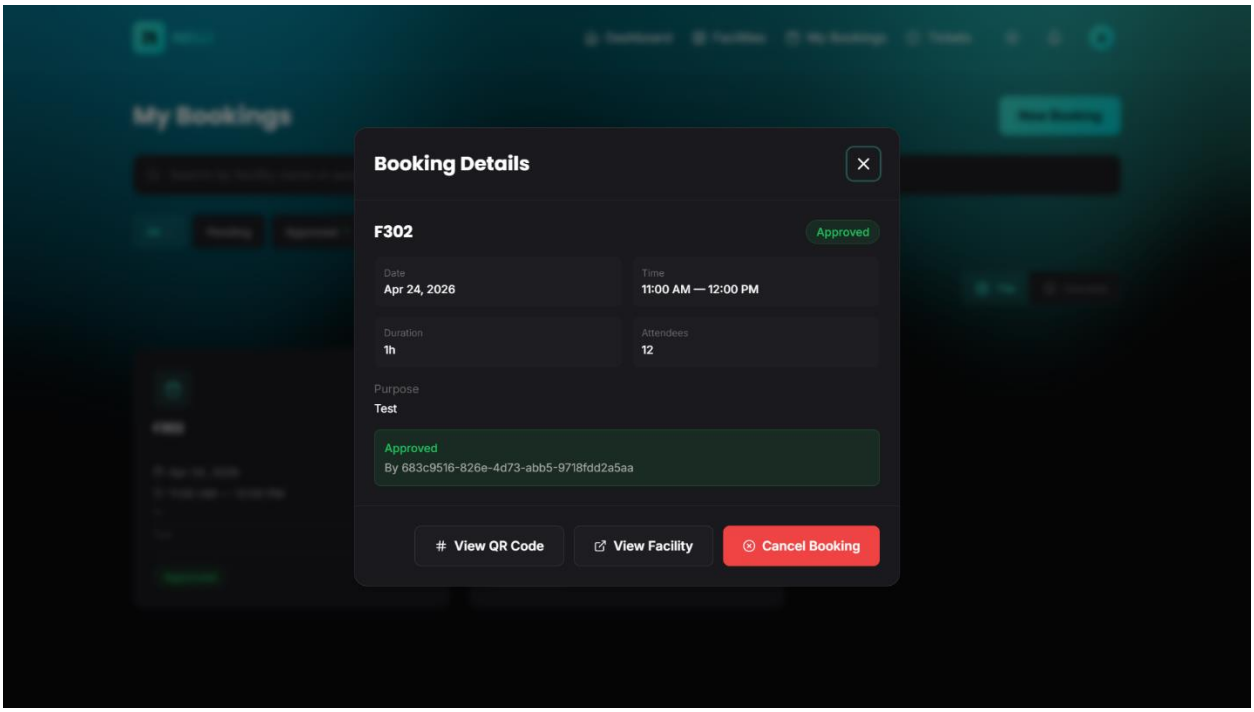

NELLi
Dashboard
Facilities
My Bookings
Tickets
A
Report an Issue
Step 2 of 4
Facility
2
Issue Details
3
Contact
4
Review
Issue Details
Describe the issue you're experiencing
Category *
Electrical
Priority *
Low
Medium
High
Urgent
Description *
Lights are not working
22/5000

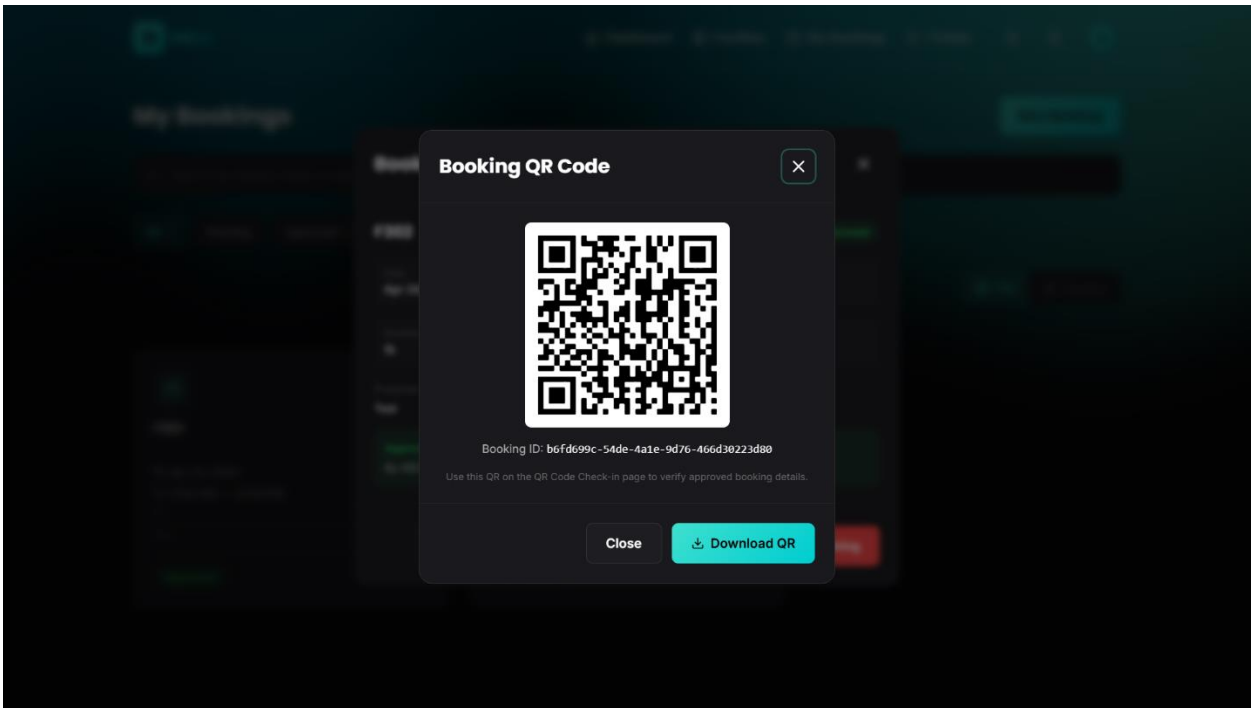

NELLi
Dashboard
Facilities
My Bookings
Tickets
A
Report an Issue
Step 3 of 4
Facility
Issue Details
3
Contact
4
Review
Contact & Attachments
How can we reach you? Add images if helpful
Contact Email *
arunod@nelli.com
Contact Phone (optional)
+94 77 123 4567
Attachments (0/3)
Drop images here or click to browse
Max 3 images, 5MB each (JPEG, PNG)
← Back
Continue →

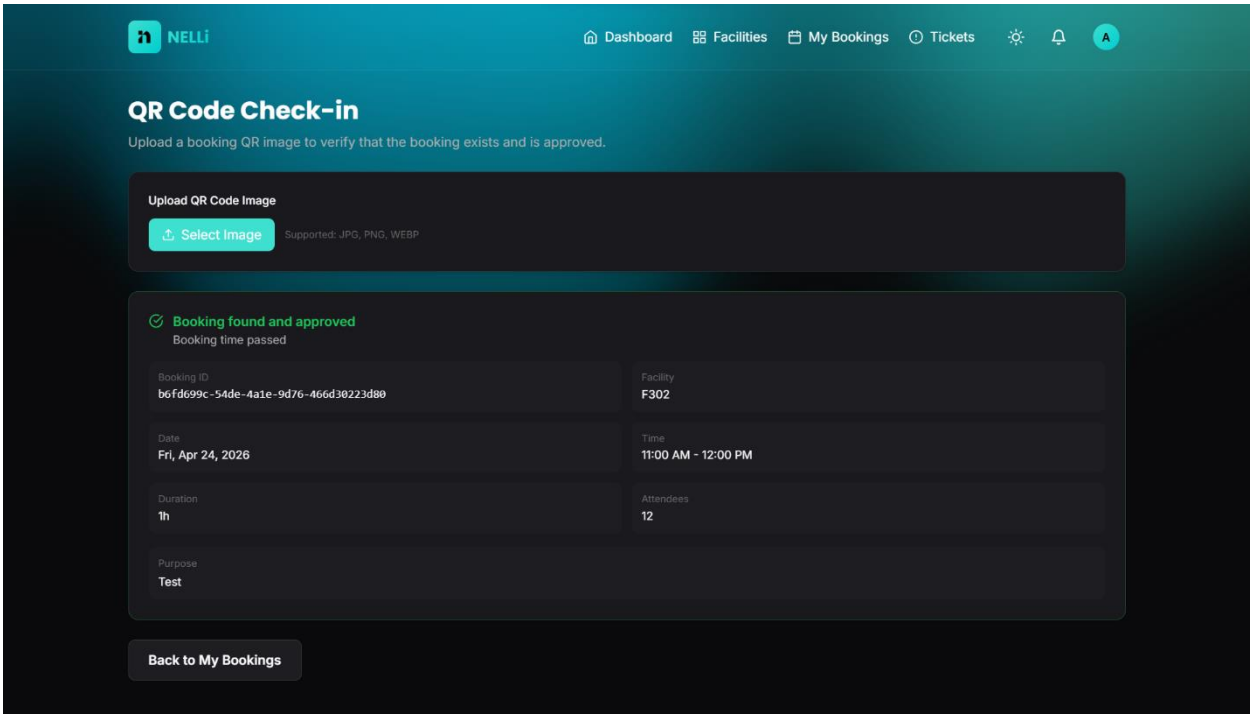

NELLi
Dashboard
Facilities
My Bookings
Tickets
A
Dashboard / Tickets / #TCK-2026-000006
Back to Tickets
#TCK-2026-000006
Medium
Open
CLEANLINESS
Facility
Main Gymnasium
Created
Apr 24, 2026, 11:24 AM
Reported By
Arunod Jayasinghe
Description
Gym's Garbage Bin is not Cleaned
Edit
Delete
Share
Print
Activity Timeline
Ticket Created
By Arunod Jayasinghe
Apr 24, 2026, 11:24 AM
Timestamps
Created: Apr 24, 2026, 11:24 AM
Updated: Apr 24, 2026, 11:24 AM
Attachments (1)
Upload Image


NELLi
Dashboard
Facilities
My Bookings
Tickets
A
Dashboard / Tickets / #TCK-2026-000006
Back to Tickets
#TCK-2026-000006
Medium
Open
CLEANLINESS
Facility
Main Gymnasium
Created
Apr 24, 2026, 11:24 AM
Reported By
Arunod Jayasinghe
Description
Gym's Garbage Bin is not Cleaned
Edit
Delete
Share
Print
Activity Timeline
Ticket Created
By Arunod Jayasinghe
Apr 24, 2026, 11:24 AM
1 Comment
Last: "It's frustating..!..."
just now
Timestamps
Created: Apr 24, 2026, 11:24 AM
Updated: Apr 24, 2026, 11:24 AM
Attachments (1)
Upload Image

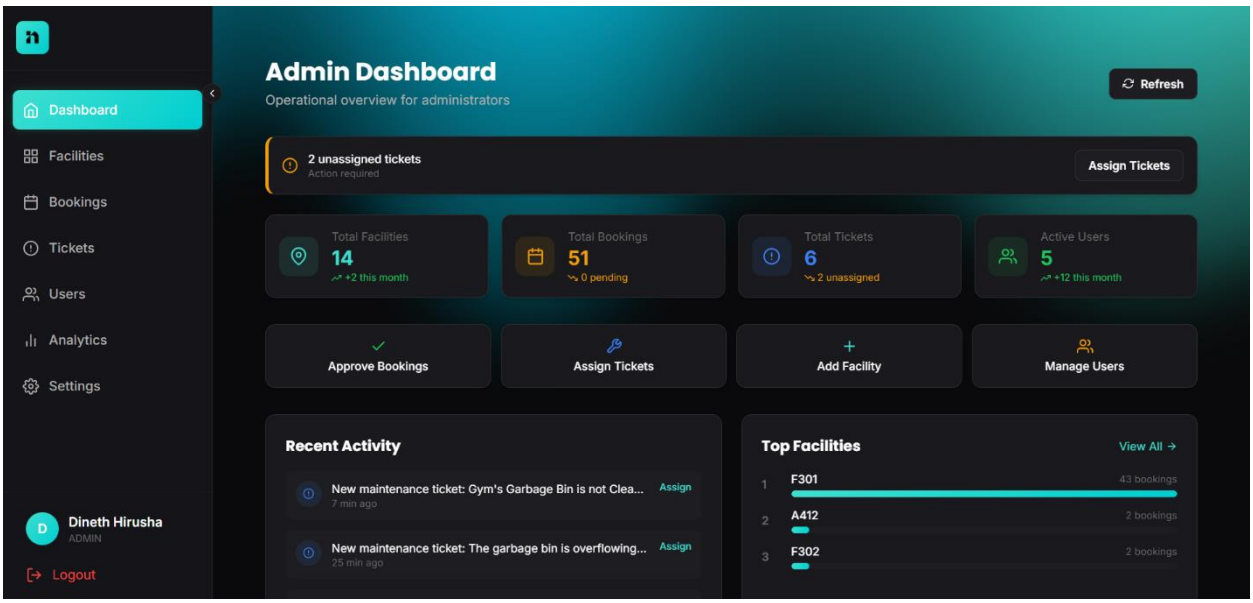
14.7. QR Code Verification Component

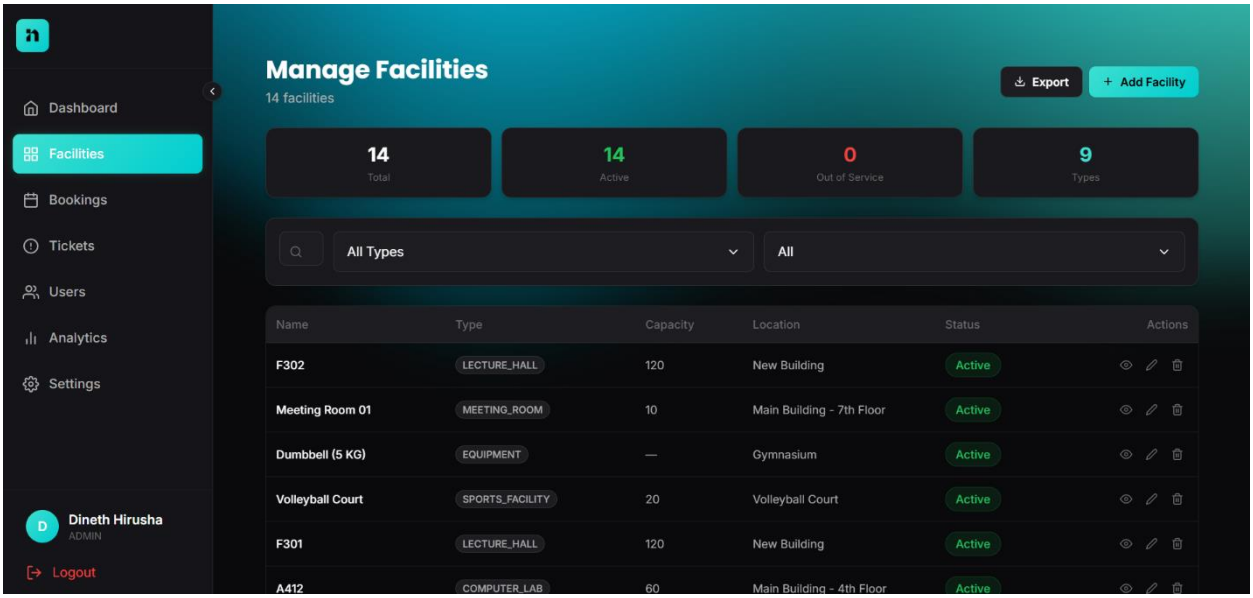


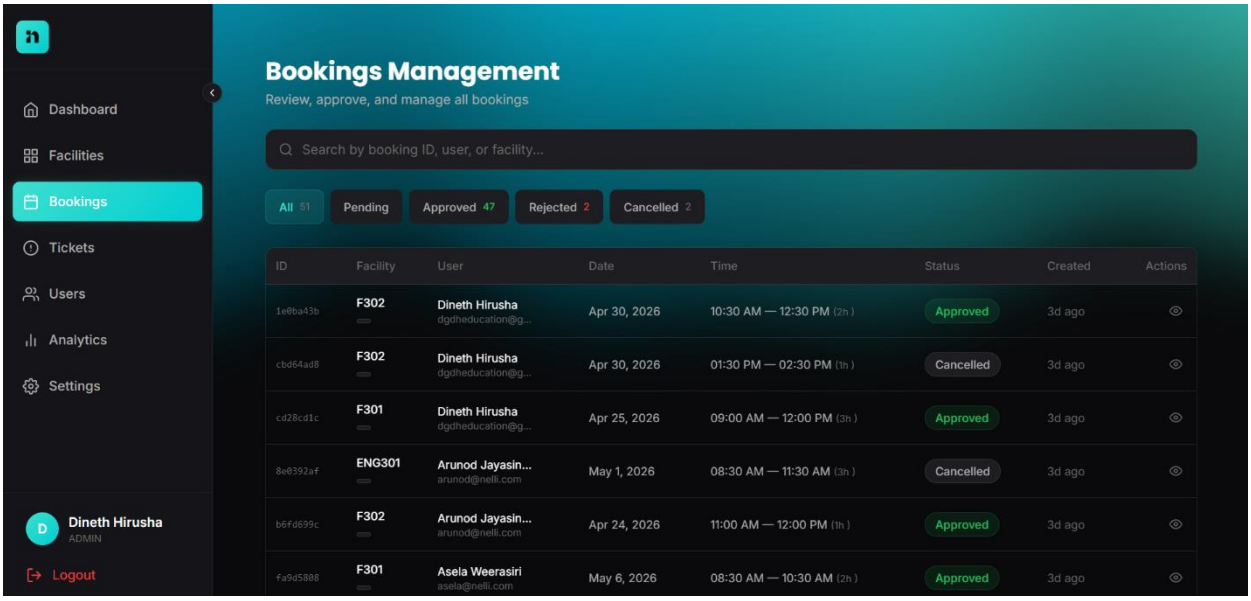


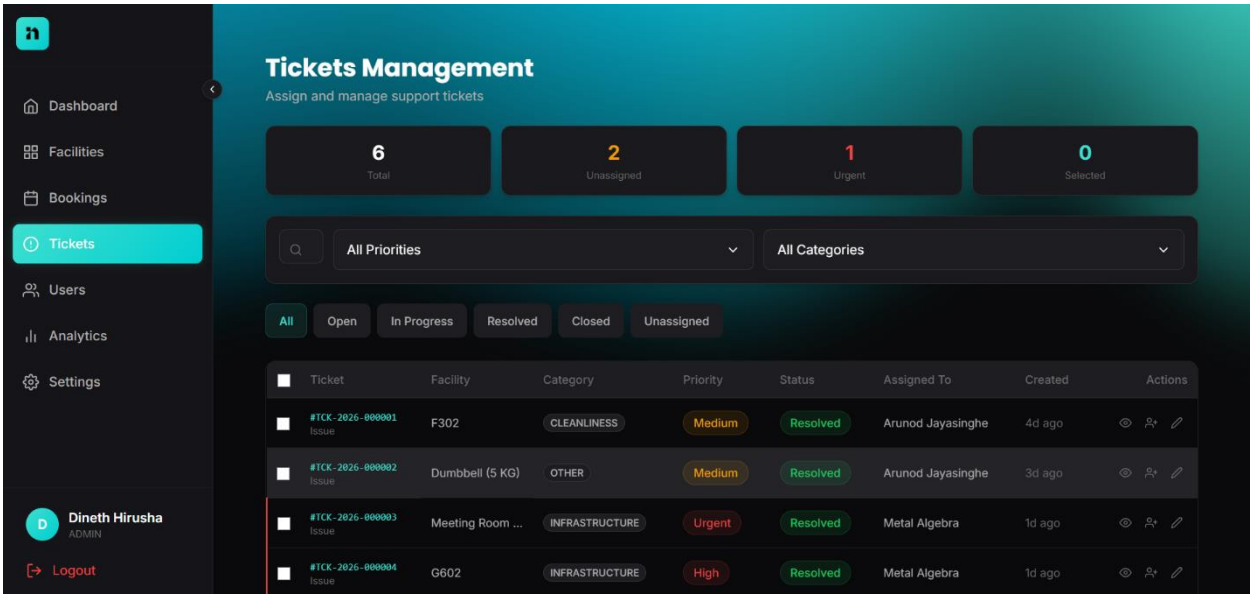


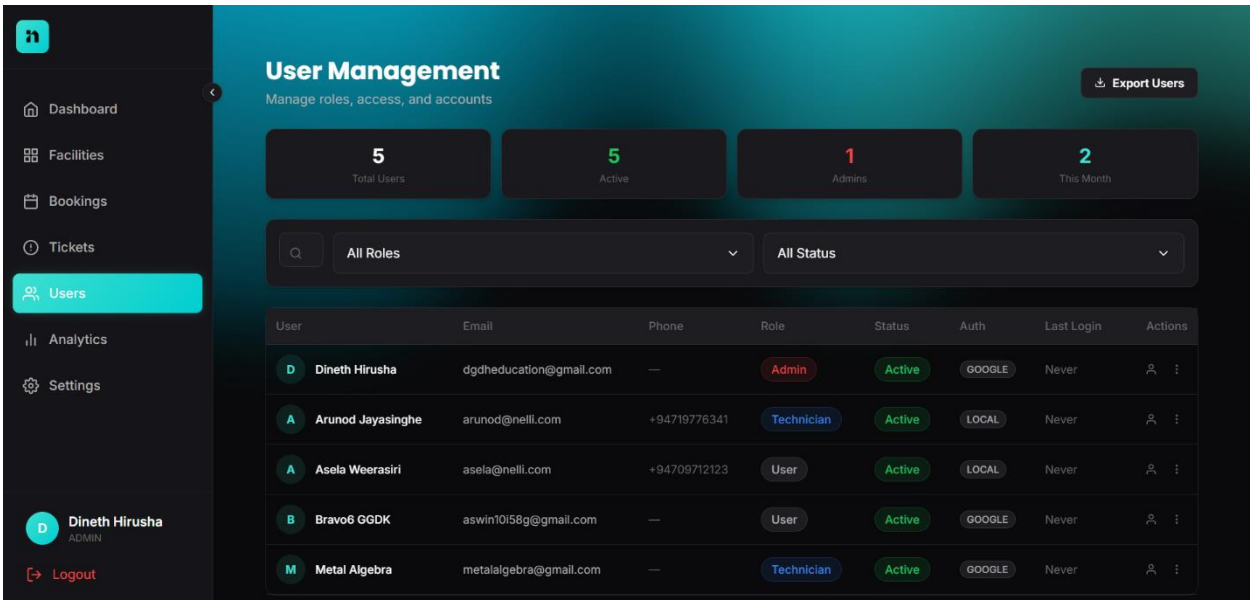
14.8. Admin Management Components




Dashboard
Facilities
Bookings
Tickets
Users
Analytics
Settings
Dineth Hirusha
ADMIN
Logout
Manage Facilities
14 facilities
Export
+ Add Facility
14
Total
14
Active
0
Out of Service
9
Types
All Types
All
Name
Type
Capacity
Location
Status
Actions
F302
LECTURE_HALL
120
New Building
Active
Meeting Room 01
MEETING_ROOM
10
Main Building - 7th Floor
Active
Dumbbell (5 KG)
EQUIPMENT
—
Gymnasium
Active
Volleyball Court
SPORTS_FACILITY
20
Volleyball Court
Active
F301
LECTURE_HALL
120
New Building
Active
A412
COMPUTER_LAB
60
Main Building - 4th Floor
Active

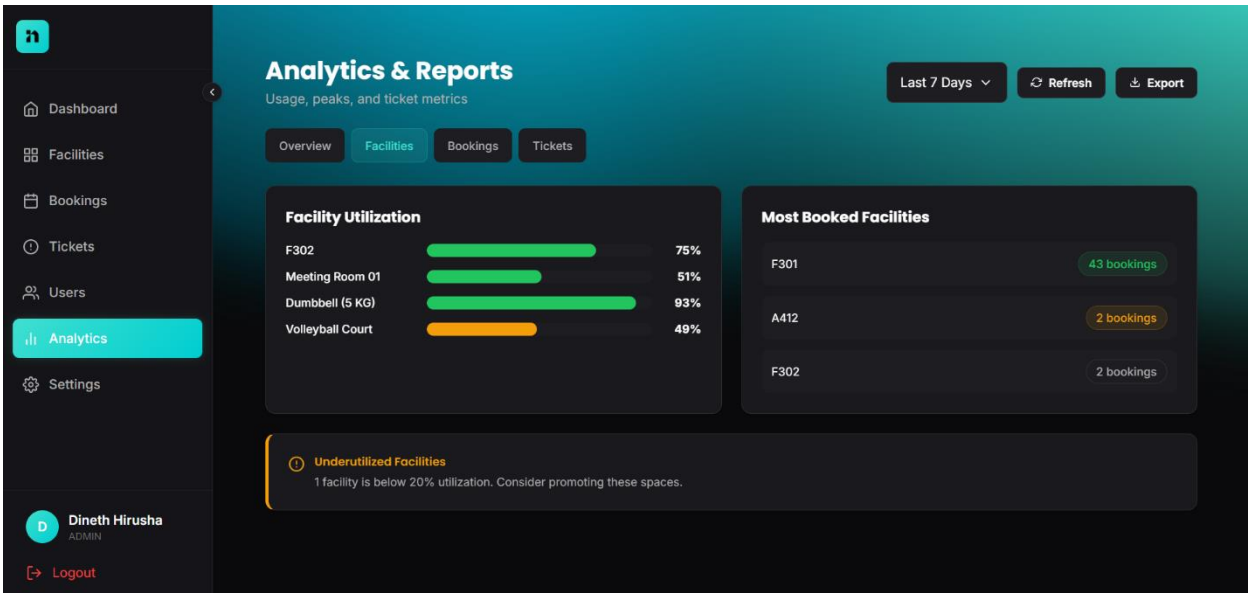

Dashboard
Facilities
Bookings
Tickets
Users
Analytics
Settings
Dineth Hirusha
ADMIN
Logout
Bookings Management
Review, approve, and manage all bookings
Search by booking ID, user, or facility...
All 51
Pending
Approved 47
Rejected 2
Cancelled 2
ID
Facility
User
Date
Time
Status
Created
Actions
1e0ba43b
F302
Dineth Hirusha
dgdheducation@g...
Apr 30, 2026
10:30 AM — 12:30 PM (2h)
Approved
3d ago
cbd64ad8
F302
Dineth Hirusha
dgdheducation@g...
Apr 30, 2026
01:30 PM — 02:30 PM (1h)
Cancelled
3d ago
cd28cd1c
F301
Dineth Hirusha
dgdheducation@g...
Apr 25, 2026
09:00 AM — 12:00 PM (3h)
Approved
3d ago
8e0392af
ENG301
Arunod Jayasin...
arunod@nelli.com
May 1, 2026
08:30 AM — 11:30 AM (3h)
Cancelled
3d ago
b6fd699c
F302
Arunod Jayasin...
arunod@nelli.com
Apr 24, 2026
11:00 AM — 12:00 PM (1h)
Approved
3d ago
fa9d5808
F301
Asela Weerasiri
asela@nelli.com
May 6, 2026
08:30 AM — 10:30 AM (2h)
Approved
3d ago

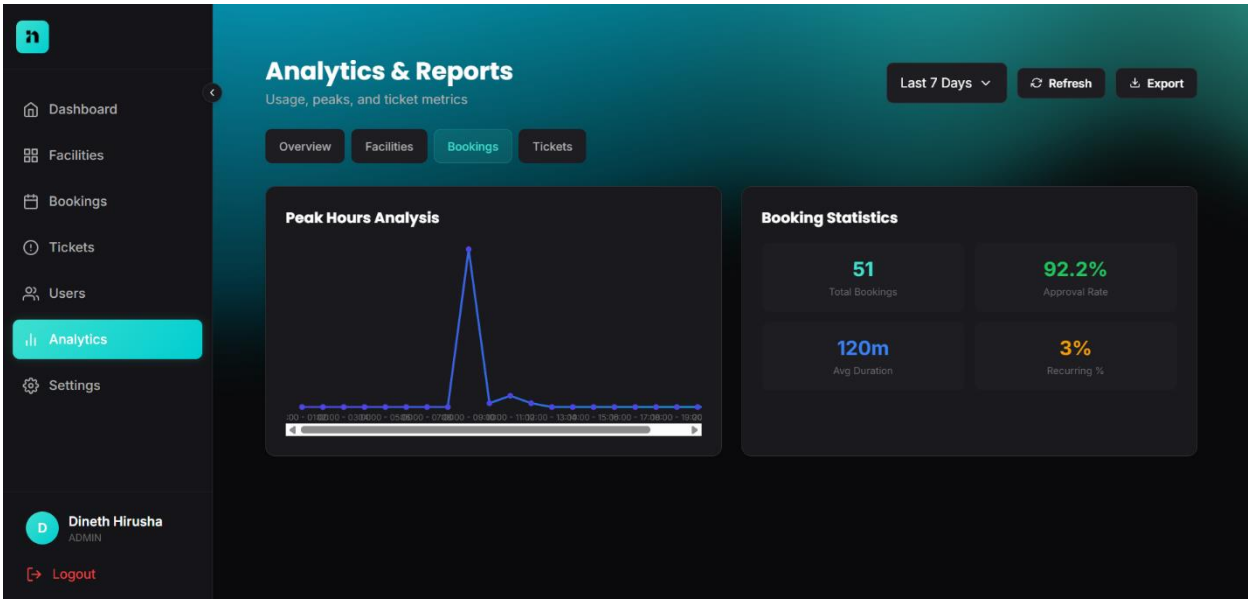

Tickets Management
Assign and manage support tickets
6
Total
2
Unassigned
1
Urgent
0
Selected
All Priorities
All Categories
All
Open
In Progress
Resolved
Closed
Unassigned
Ticket
Facility
Category
Priority
Status
Assigned To
Created
Actions
#TCK-2026-000001
Issue
F302
CLEANLINESS
Medium
Resolved
Arunod Jayasinghe
4d ago
#TCK-2026-000002
Issue
Dumbbell (5 KG)
OTHER
Medium
Resolved
Arunod Jayasinghe
3d ago
#TCK-2026-000003
Issue
Meeting Room ...
INFRASTRUCTURE
Urgent
Resolved
Metal Algebra
1d ago
#TCK-2026-000004
Issue
G602
INFRASTRUCTURE
High
Resolved
Metal Algebra
1d ago
Dashboard
Facilities
Bookings
Tickets
Users
Analytics
Settings
Dineth Hirusha
ADMIN
Logout


User Management
Manage roles, access, and accounts
Export Users
5
Total Users
5
Active
1
Admins
2
This Month
All Roles
All Status
User
Email
Phone
Role
Status
Auth
Last Login
Actions
Dineth Hirusha
dgdheducation@gmail.com
—
Admin
Active
GOOGLE
Never
Arunod Jayasinghe
arunod@nelli.com
+94719776341
Technician
Active
LOCAL
Never
Asela Weerasiri
asela@nelli.com
+94709712123
User
Active
LOCAL
Never
Bravo6 GGDK
aswin10i58g@gmail.com
—
User
Active
GOOGLE
Never
Metal Algebra
metalalgebra@gmail.com
—
Technician
Active
GOOGLE
Never
Dashboard
Facilities
Bookings
Tickets
Users
Analytics
Settings
Dineth Hirusha
ADMIN
Logout

14.9. Admin Analytics Components






Dashboard
Facilities
Bookings
Tickets
Users
Analytics
Settings
Dineth Hirusha
ADMIN
Logout
Analytics & Reports
Usage, peaks, and ticket metrics
Last 7 Days
Refresh
Export
Overview
Facilities
Bookings
Tickets
Peak Hours Analysis
Booking Statistics
51
Total Bookings
92.2%
Approval Rate
120m
Avg Duration
3%
Recurring %


Dashboard
Facilities
Bookings
Tickets
Users
Analytics
Settings
Dineth Hirusha
ADMIN
Logout
Analytics & Reports
Usage, peaks, and ticket metrics
Last 7 Days
Refresh
Export
Overview
Facilities
Bookings
Tickets
6
Total Tickets
14.5h
Avg Response
14.7h
Avg Resolution
66.7%
Resolution Rate
Tickets by Category
CLEANLINESS 3
INFRASTRUCTURE 2
OTHER 1
EQUIPMENT FAILURE 0
ELECTRICAL 0
PLUMBING 0
INTERNET CONNECTI... 0
Technician Performance
Technician
Assigned
Resolved
Avg Time
Arunod Jayasinghe 2 2 29.0h
Metal Algebra 2 2 N/A

15. References

[1] P. Johnson, *Spring Boot: Up and Running*, 1st ed. Sebastopol, CA, USA: O'Reilly Media, 2021.

[2] V. Siva, *Spring Security in Action*, 1st ed. Shelter Island, NY, USA: Manning Publications, 2020.

[3] R. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000.

[4] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.

[5] Meta Open Source, "React – A JavaScript library for building user interfaces," 2026. [Online]. Available: https://react.dev

[6] Spring.io, "Spring Boot Documentation," 2026. [Online]. Available: https://spring.io/projects/spring-boot

[7] Spring.io, "Spring Security Reference," 2026. [Online]. Available: https://spring.io/projects/spring-security

[8] PostgreSQL Global Development Group, "PostgreSQL Documentation," 2026. [Online]. Available: https://www.postgresql.org/docs/

[9] Neon Inc., "Neon Serverless PostgreSQL Platform," 2026. [Online]. Available: https://neon.tech

[10] Google, "OAuth 2.0 Authorization Framework," 2026. [Online]. Available: https://developers.google.com/identity/protocols/oauth2

[11] Auth0 Inc., "JSON Web Tokens (JWT) Introduction," 2026. [Online]. Available: https://jwt.io/introduction

[12] Apache Software Foundation, "Apache Maven Project," 2026. [Online]. Available: https://maven.apache.org

[13] Axios Contributors, "Axios HTTP Client," 2026. [Online]. Available: https://axios-http.com

[14] Tailwind Labs, "Tailwind CSS Documentation," 2026. [Online]. Available: https://tailwindcss.com

[15] Recharts Group, "Recharts – Composable Charting Library for React," 2026. [Online]. Available: https://recharts.org

[16] ZXing Project, "ZXing ('Zebra Crossing') Barcode Scanning Library," 2026. [Online]. Available: https://github.com/zxing/zxing

[17] Google, "Gmail SMTP Server Documentation," 2026. [Online]. Available: https://support.google.com/mail

[18] OpenAPI Initiative, "OpenAPI Specification," 2026. [Online]. Available: https://swagger.io/specification

[19] GitHub Inc., “GitHub Actions Documentation,” 2026. [Online]. Available: https://docs.github.com/actions

[20] Postman Inc., “Postman API Platform Documentation,” 2026. [Online]. Available: https://learning.postman.com